

Содержание

ВВЕДЕНИЕ.....	5
1. ОСНОВНЫЕ ЭТАПЫ ПРОЕКТИРОВАНИЯ ЗАДАЧИ.....	6
2. ОСНОВНЫЕ ЭЛЕМЕНТЫ БЛОК-СХЕМЫ РЕАЛИЗАЦИИ АЛГОРИТМОВ.....	8
3. БЛОК-СХЕМЫ РЕАЛИЗАЦИИ СЛОЖНЫХ УСЛОВНЫХ КОНСТРУКЦИЙ И ЦИКЛОВ	14
4. КРАТКАЯ ИСТОРИЯ ПОЯВЛЕНИЯ ЯЗЫКА C++	21
5. ОСНОВНЫЕ ПАРАДИГМЫ ПРОГРАММИРОВАНИЯ	22
6. ПАРАДИГМА ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ НА ЯЗЫКЕ C++.....	24
7. НЕКОТОРЫЕ ИНСТРУМЕНТЫ ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ	28
7.1. Программа и основная функция main.....	28
7.2. Стандартный ввод и вывод	29
7.3. Переменные и их типы	31
7.4. Константные переменные	36
7.5. Основные арифметические, логические операции и операторы сравнения.....	37
7.6. Условные операторы и циклы	39
7.7. Указатели и ссылки.....	52
7.8. Массивы и его типы.....	59
7.9. Строки и оператор sizeof.....	65
7.10. Функции, перегрузка функций	67
7.11. Структуры.....	75
7.12. Локальные и глобальные переменные.....	79
8. ПАРАДИГМА ОБЪЕКТНО-ОРИЕНТИРОВАННОГО ПРОГРАММИРОВАНИЯ	83

8.1.	Понятие парадигмы объектно-ориентированного программирования.....	83
8.2.	Основные определения объекта и столпы объектно-ориентированного программирования.....	84
8.3.	Время жизни объектов, взаимодействие объектов.....	86
8.4.	Конструктор и деструктор.....	88
8.5.	Описание объектов, модификаторы доступа, константный объект.....	91
8.6.	Указатели на элементы классов.....	95
8.7.	Указатель this.....	97
8.8.	Статические поля и методы класса	99
9.	ОСНОВНЫЕ ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	102
	СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ	106
	СВЕДЕНИЯ ОБ АВТОРАХ	107

ВВЕДЕНИЕ

Настоящее учебно-методическое издание представляет собой учебно-методическое пособие по дисциплине «Объектно-ориентированное программирование» для специальности 09.03.01 «Информатика и вычислительная техника», 09.03.03 «Прикладная информатика», 09.03.04 «Программная инженерия», 01.03.04 «Прикладная математика».

Издание предназначено для помощи студентам первого курса в обучении по данной специальности при изучении теоретических основ и выполнении практических работ, предусмотренных рабочей программой дисциплины. Учебно-методическое пособие поможет обучающимся освоить знания и получить умения и практические навыки в использовании языка C++, что необходимо при выполнении определенных видов работ, связанных с будущей профессиональной деятельностью. Они содержат необходимую часть теоретических сведений, контрольные вопросы и задания по практическим работам, предусмотренные рабочей программой дисциплины «Объектно-ориентированное программирование».

1. ОСНОВНЫЕ ЭТАПЫ ПРОЕКТИРОВАНИЯ ЗАДАЧИ

В мире каждый день решается большое количество проектов, задач, которые каким-либо способом оказывают влияние на жизнь человека. Одними из наиболее интересных, важных и часто встречающихся проектов в настоящее время в информационном обществе являются те, которые в практической основе имеют программный код.

Существует много различных методологий, которые позволяют по-разному организовать работу над проектом. Наиболее известными являются Waterfall и Agile. В составе этих методологий существуют отдельные модели (методики) управления, так, например, в составе методологии Waterfall существуют: каскадная, итеративная и спиралевидная модели управления.

В рамках дисциплины «Объектно-ориентированное программирование» для обучения будет взята методология управления Waterfall, итеративная модель управления проектом, которая представлена на рисунке 1.

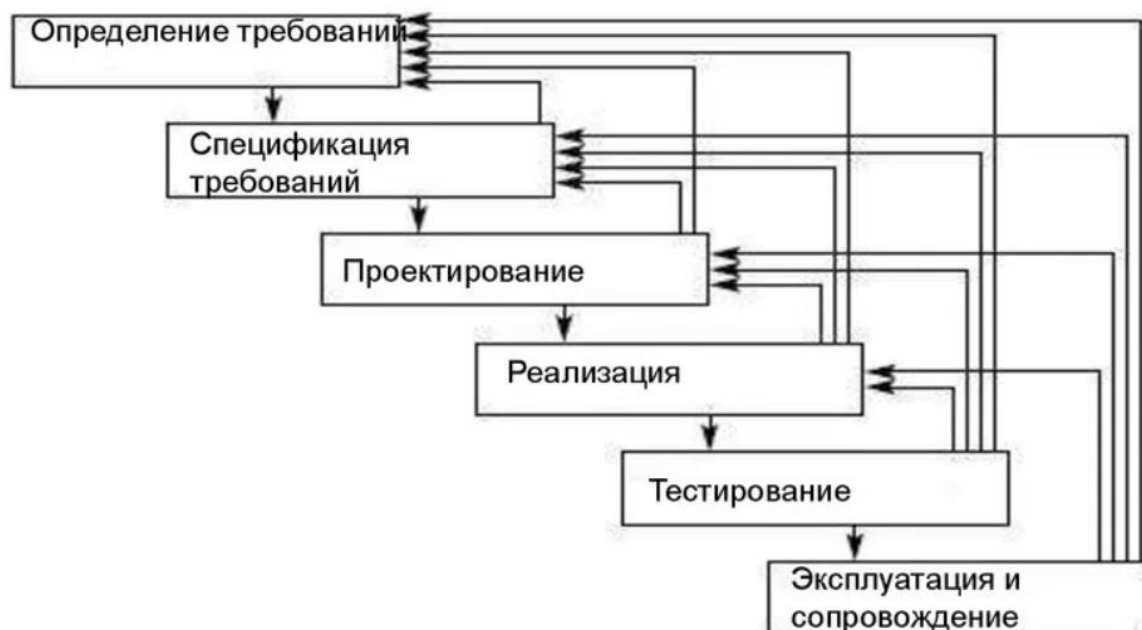


Рисунок 1. Итеративная модель управления проектом

Любые проекты всегда подразделяются на задачи, которые требуется отдельно решить. В рамках последовательного решения задачи, на основе итеративной модели управления, требуется по очереди выполнить несколько пунктов:

- Изучить постановку задачи. В постановке задачи, как правило, описываются основные требования, которые ОБЯЗАТЕЛЬНЫ к

исполнению в рамках выполнения данного проекта. Аналогом в жизни постановки задачи являются требования заказчика, оформленные в виде технического задания или приложений к техническому заданию;

- Продумать концепцию инструментария решения. На основе требований заказчика на данном этапе требуется понять, какие инструменты (переменные, операторы, массивы, функции и т.д.) потребуются для реализации задачи;
- Продумать и реализовать алгоритмы решения задачи и ее элементов. Благодаря алгоритму можно однозначно понять зависимость и последовательность действий друг от друга. Одним из наиболее распространенных способов написания алгоритмов являются блок-схемы. Язык описания алгоритма является универсальным и никогда не привязывается к определенному языку программирования;
- На основе реализованных алгоритмов реализовать программный код на языке программирования, используя его синтаксические конструкции. Важно, что программный код базируется и, как правило, полностью повторяет алгоритм решения;
- После реализации программного кода на основе разработанных алгоритмов идет квалифицированное тестирование, цель которого проверить работоспособность программного кода с помощью нескольких наборов входных и ожидаемых выходных параметров;

В процессе реализации последовательности действий допускается возможность перехода и возврата из одного пункта в другой.

2. ОСНОВНЫЕ ЭЛЕМЕНТЫ БЛОК-СХЕМЫ РЕАЛИЗАЦИИ АЛГОРИТМОВ

При реализации алгоритма решения (в данном случае алгоритм представлен будет в виде блок-схемы) каждый программный продукт всегда имеет одно начало и один конец. В блок-схеме начало или конец реализуется с помощью блока начало/конец — овала в которых пишется «начало» или «конец» [1] (рисунки 2, 3).

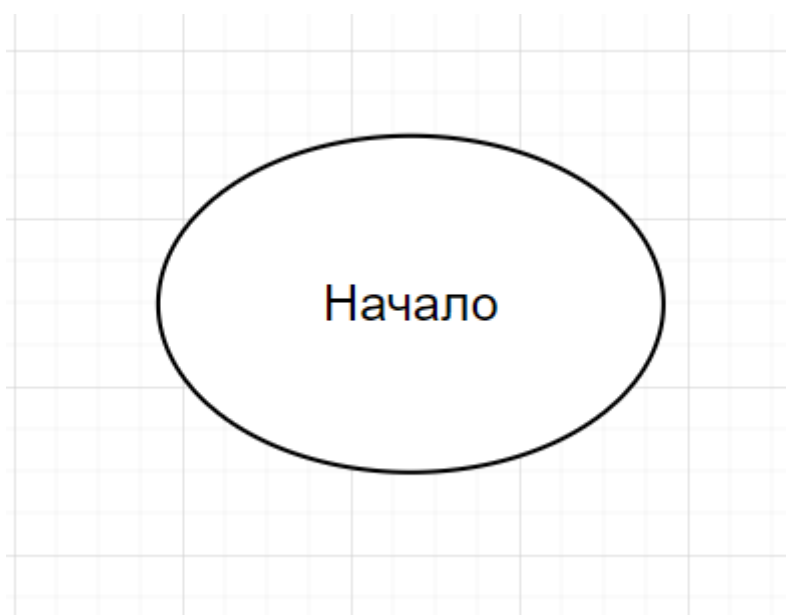


Рисунок 2. Блок начало/конец «Начало»

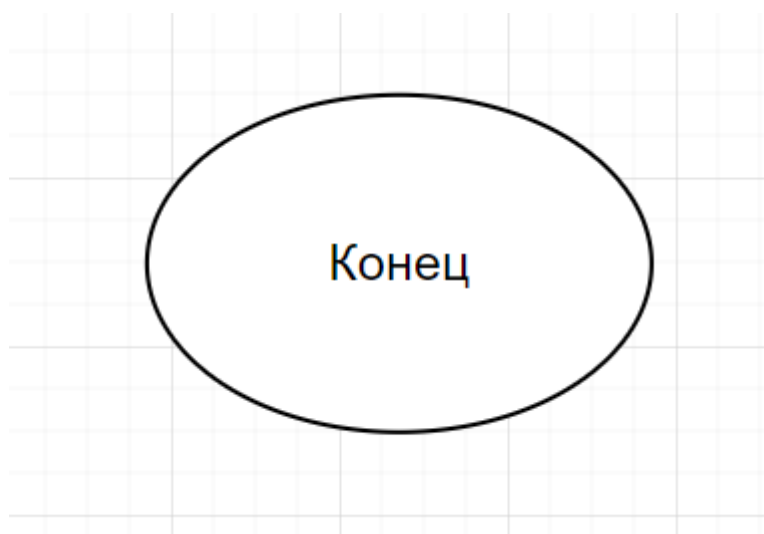


Рисунок 3. Блок начало/конец «Конец»

В случае описания работы модулей программы, где используются подпрограммы, например, функции (методы в ООП) используется также блок начало/конец. Однако наименование внутри овала будет, например, следующее: «Старт работы функции func» или «Окончание работы функции func» (рисунок 4, 5).



Рисунок 4. Блок начало/конец «Старт работы функции func»



Рисунок 5. Блок начало/конец «Окончание работы функции func»

Если при решении задачи требуется ввести данные или вывести их, то при описании алгоритма в блок-схеме используется блок ввода/вывода — параллелограмм, в котором пишется, например: «ввод значения переменной а» или «вывод значения переменной а» (рисунок 6, 7).

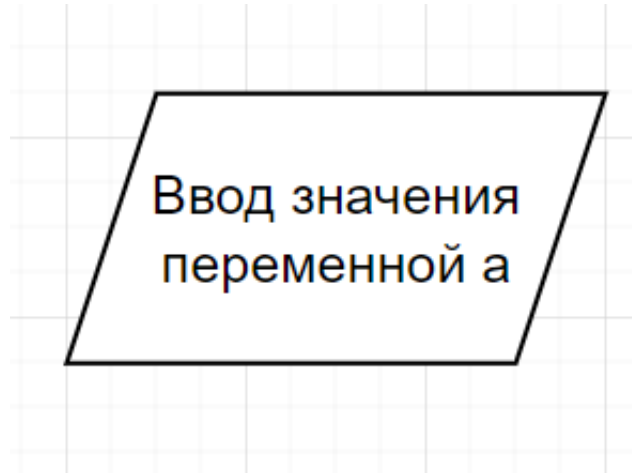


Рисунок 6. Блок ввода/вывода «Ввод значения переменной а»

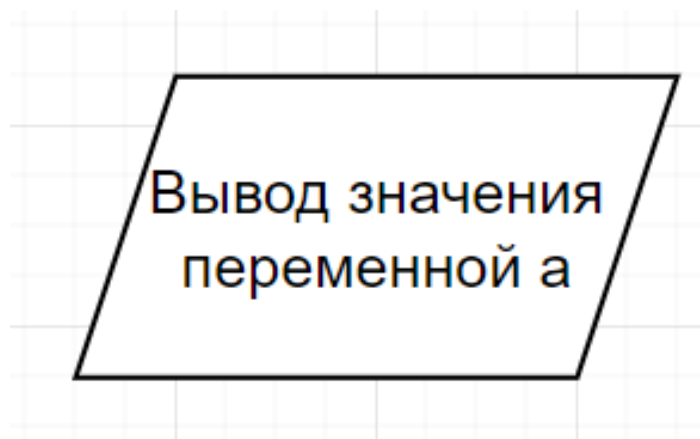


Рисунок 7. Блок ввода/вывода «Вывод значения переменной а»

Наиболее часто встречающимся элементом во многих блок-схемах является блок процесса, который изображается в виде прямоугольника. В данном блоке, как правило, пишется конечный процесс, который не подразумевает деление на подпроцессы. Примером процесса может служить объявление переменной а целочисленного типа (рисунок 8).

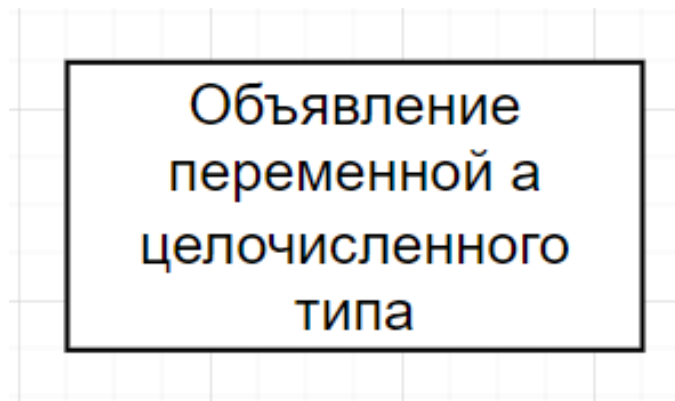


Рисунок 8. Блок процесса «Объявление переменной а целочисленного типа»

В случае если процесс не является конечным, то есть он в себе имеет некий подпроцесс, описанный разработчиком, то тогда он обозначается с помощью прямоугольника, имеющего два отрезка внутри прямоугольника, параллельных ширине фигуры. При этом сами действия в подпроцессе выносятся в отдельную блок-схему. Примером процесса с подпроцессом может являться вызов функции func (рисунок 9).

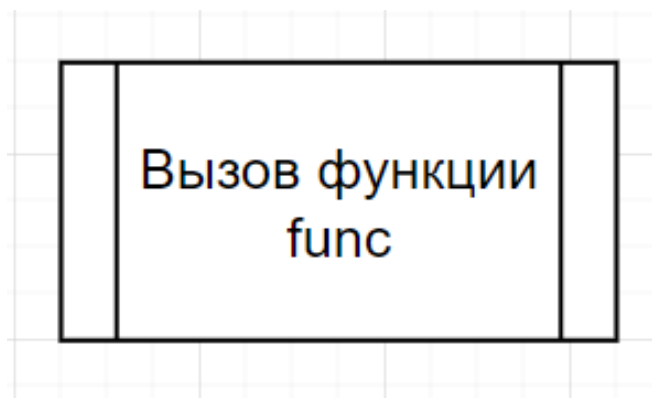


Рисунок 9. Блок процесса с подпроцессом «Вызов функции func»

Если в программе требуется реализовать условие, которое предполагает два или несколько действий, то тогда используется в блок условие, который изображается с помощью ромба (рисунок 10).

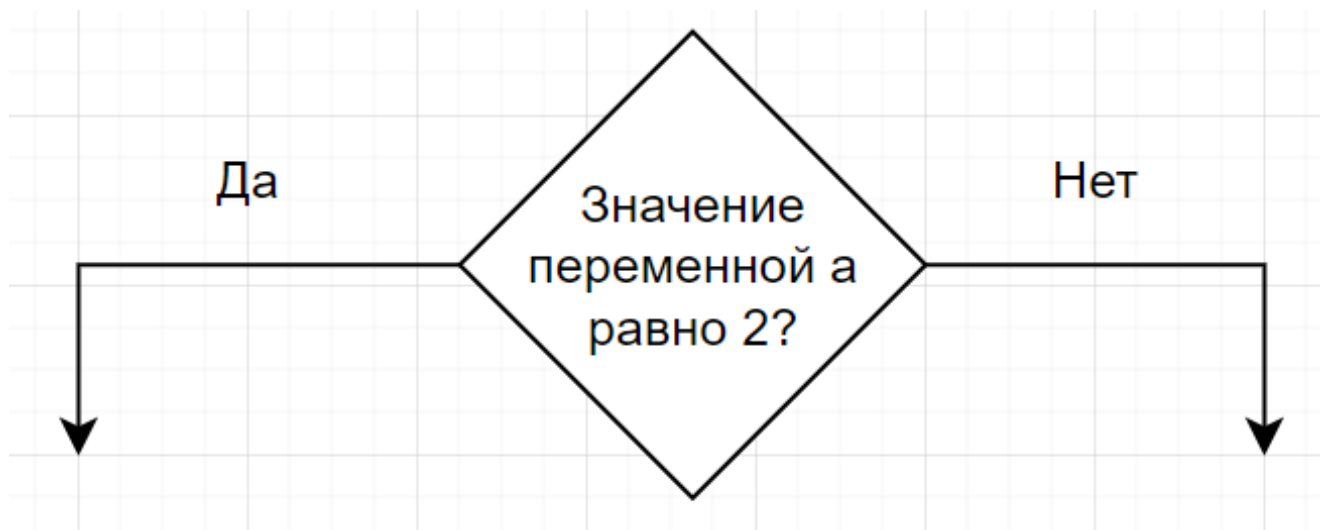


Рисунок 10. Блок условия «Значение переменной a равно 2?»

Отдельно рассматривается в блок-схеме элемент, реализующий цикл for (рисунок 11), который изображается с помощью шестиугольника. Внутри блока, пишется, как правило, начальное и конечное значения цикла, а также шаг.

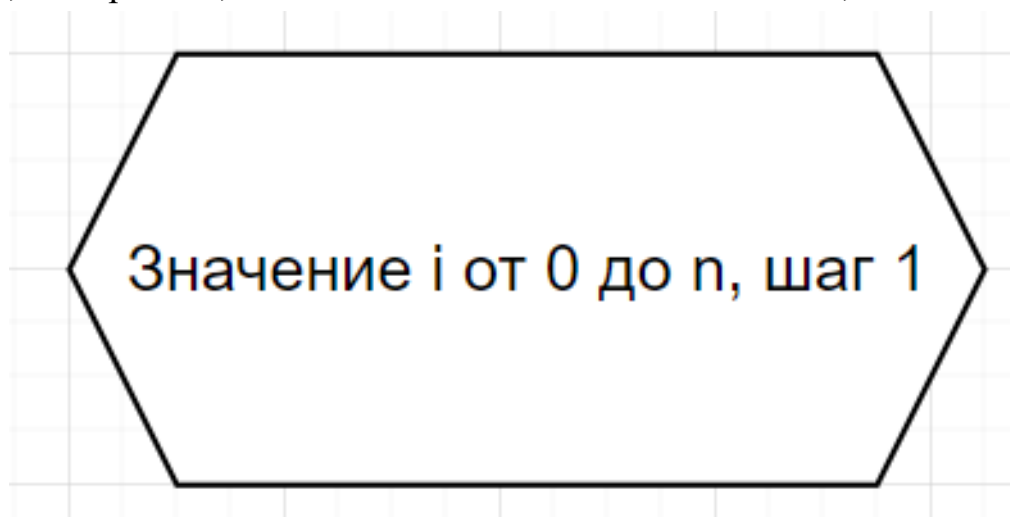


Рисунок 11. Блок цикла for «Значение i от 0 до n , шаг 1»

Пример задачи 1: реализовать программу, которая будет проверять вводимое пользователем целочисленное значение на четность или нечетность.

Алгоритм решения данной задачи с помощью блок-схемы представлен на рисунке 12.

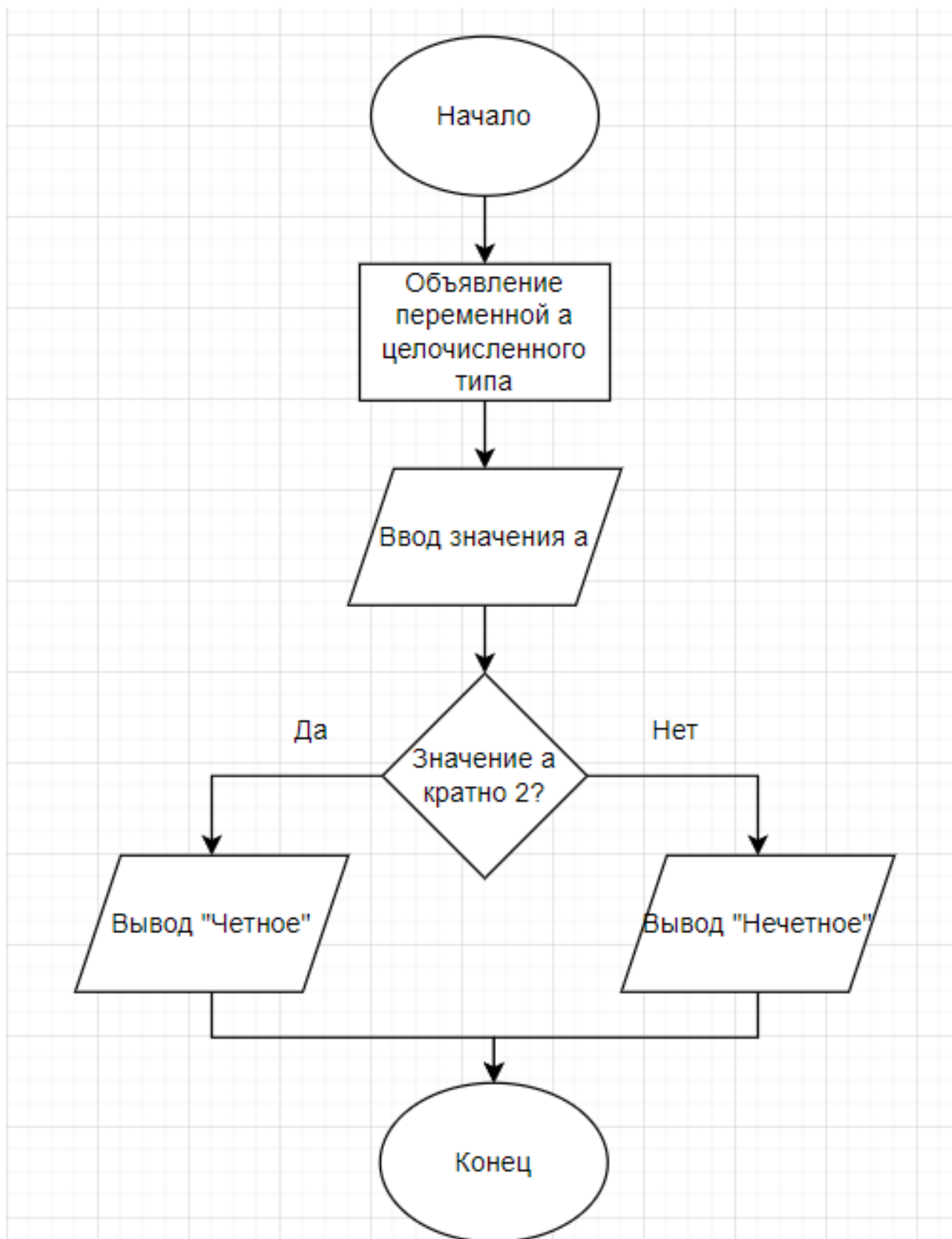


Рисунок 12. Алгоритм решения задачи по проверке целочисленного значения на четность или нечетность

3. БЛОК-СХЕМЫ РЕАЛИЗАЦИИ СЛОЖНЫХ УСЛОВНЫХ КОНСТРУКЦИЙ И ЦИКЛОВ

Часто в задачах требуется сделать большее количество веток, чем, как было изначально в операторе if две ветки. Для реализации подобных действий используются конструкции «if/else if» и «switch/case/default» [2].

Обе конструкции, как правило, предполагают повторное использование блока условия, в связи с чем и увеличивается количество путей. Пример реализации «if/else if» с помощью блок-схемы показан на рисунке 13.

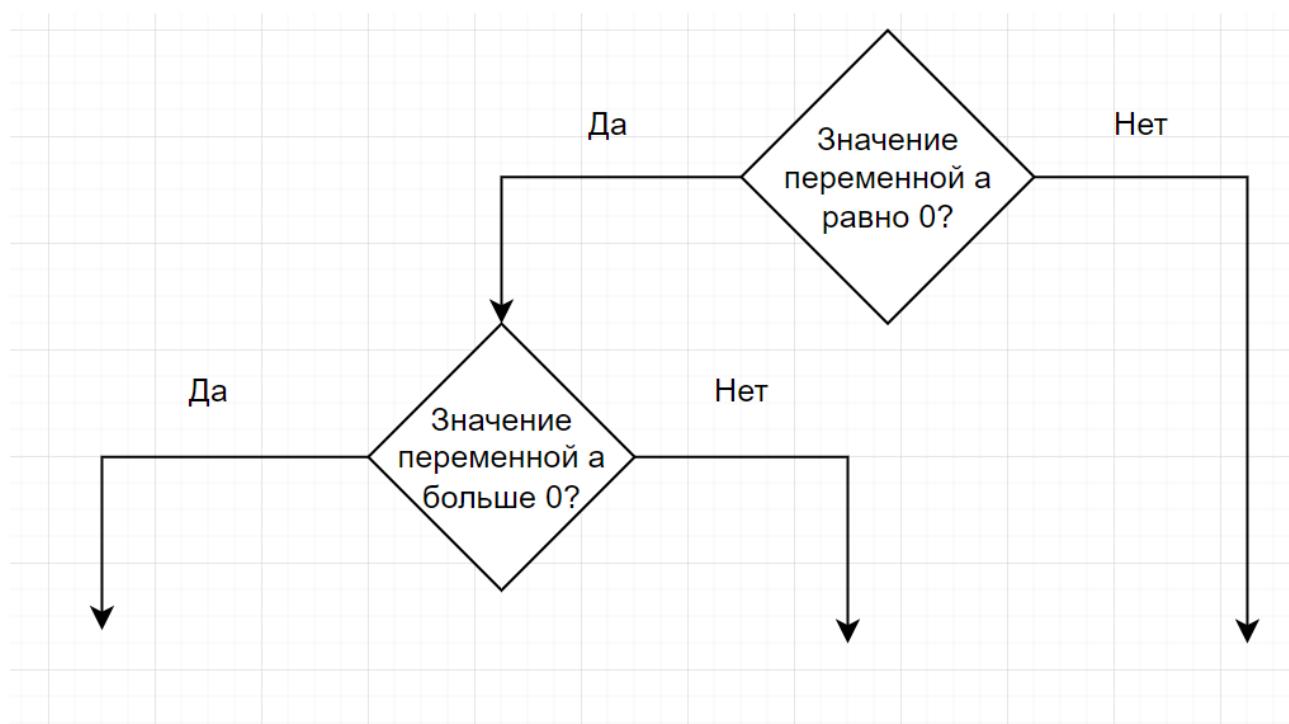


Рисунок 13. Блок-схема конструкции «if/else if» с помощью блоков условия

Пример задачи 2: реализовать программу, которая будет проверять вводимое пользователем целочисленное значение на четность, нечетность или нуль. Алгоритм решения в виде блок-схемы показан на рисунке 14.

Отдельного внимания заслуживает также конструкция «switch/case/default». Ее смысл заключается в проверке поступающего значения на switch. В случае (здесь используется case) если полученное значения равно какому-то из заранее заданных условий, то будет выполняться определённый набор операций. После успешного выполнения набора операций для выхода из конструкции используется оператор break. Если же полученное значение не подошло ни к одному из

заранее заданных условий, то будет выполняться процесс или набор процессов по умолчанию (default).

Аналогом в реальной жизни описанной конструкции выступает звонок в компанию, когда пользователю отвечает робот и говорит, например, что если пользователь хочет связаться с отделом продаж, пусть нажмет кнопку 1, если пользователь хочет связаться с менеджером компании, пусть нажмет кнопку 2, если хочет оставить отзыв, пусть нажмет кнопку 3 и т.д.

Примеры реализации конструкции switch/case/default с помощью блок-схем показаны на рисунках 15 и 16.

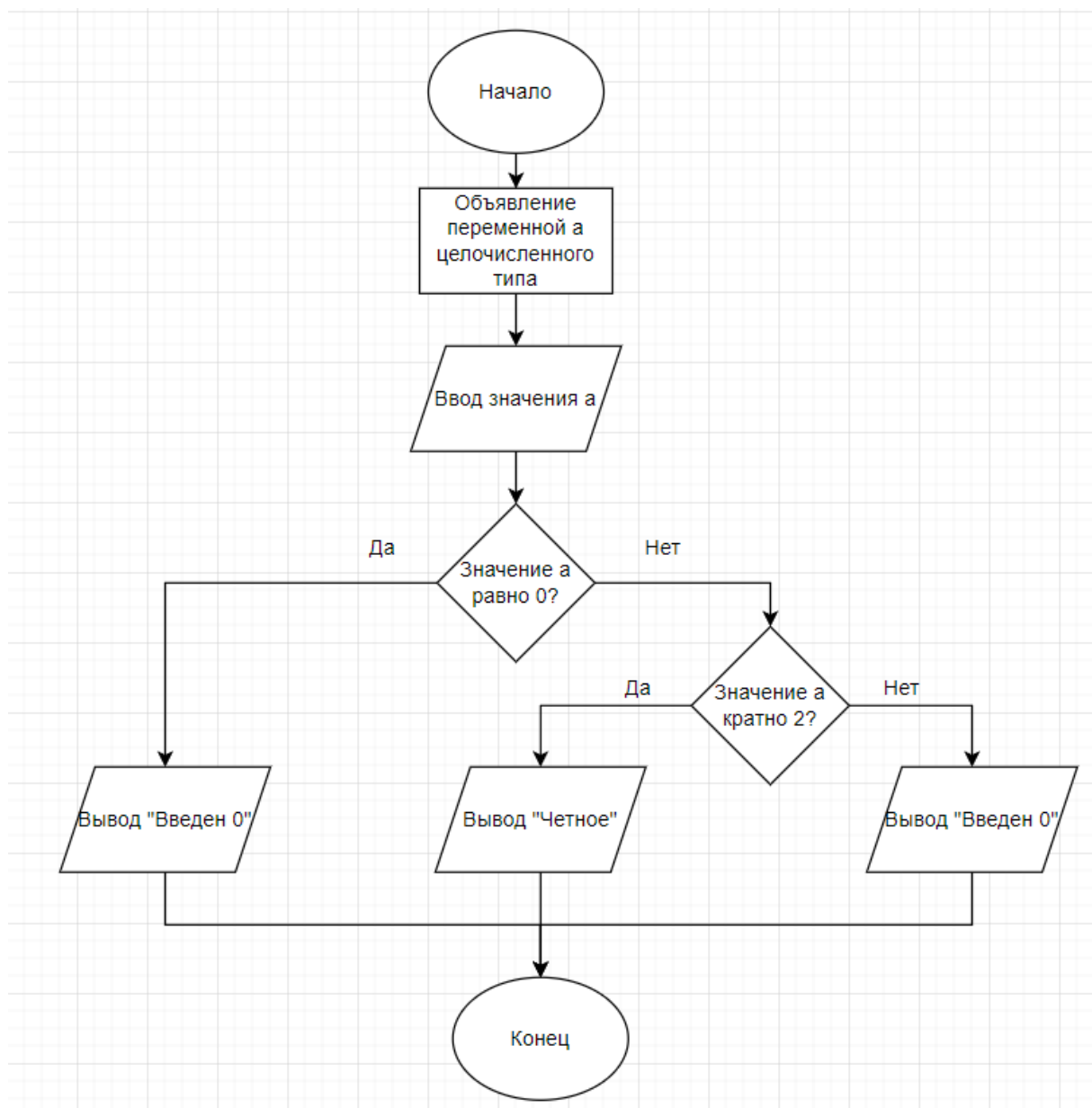


Рисунок 14. Блок-схема конструкции алгоритма решения задачи с использованием конструкции «if/else if»

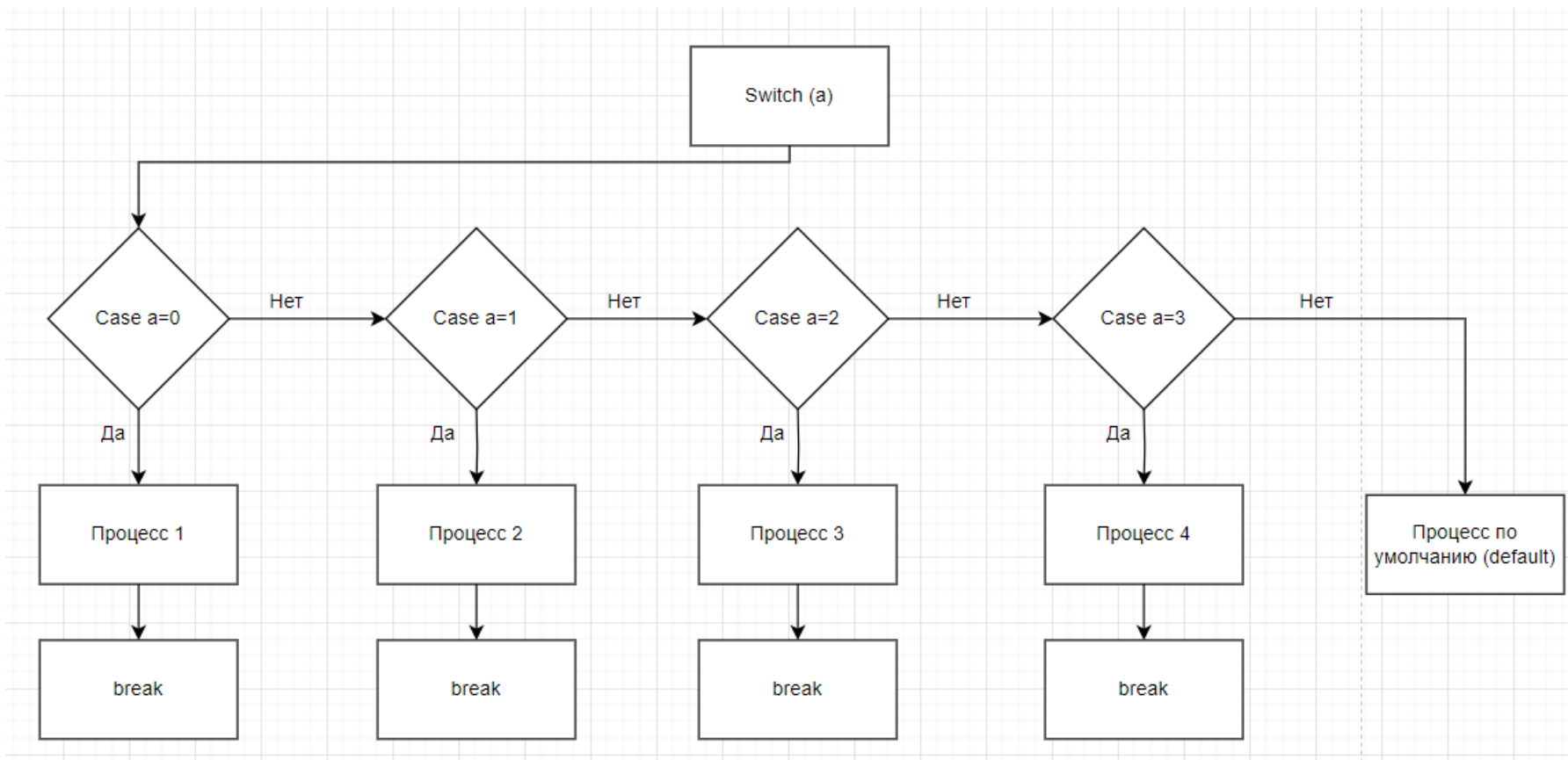


Рисунок 15. Блок-схема конструкции «switch/case/default» по горизонтали

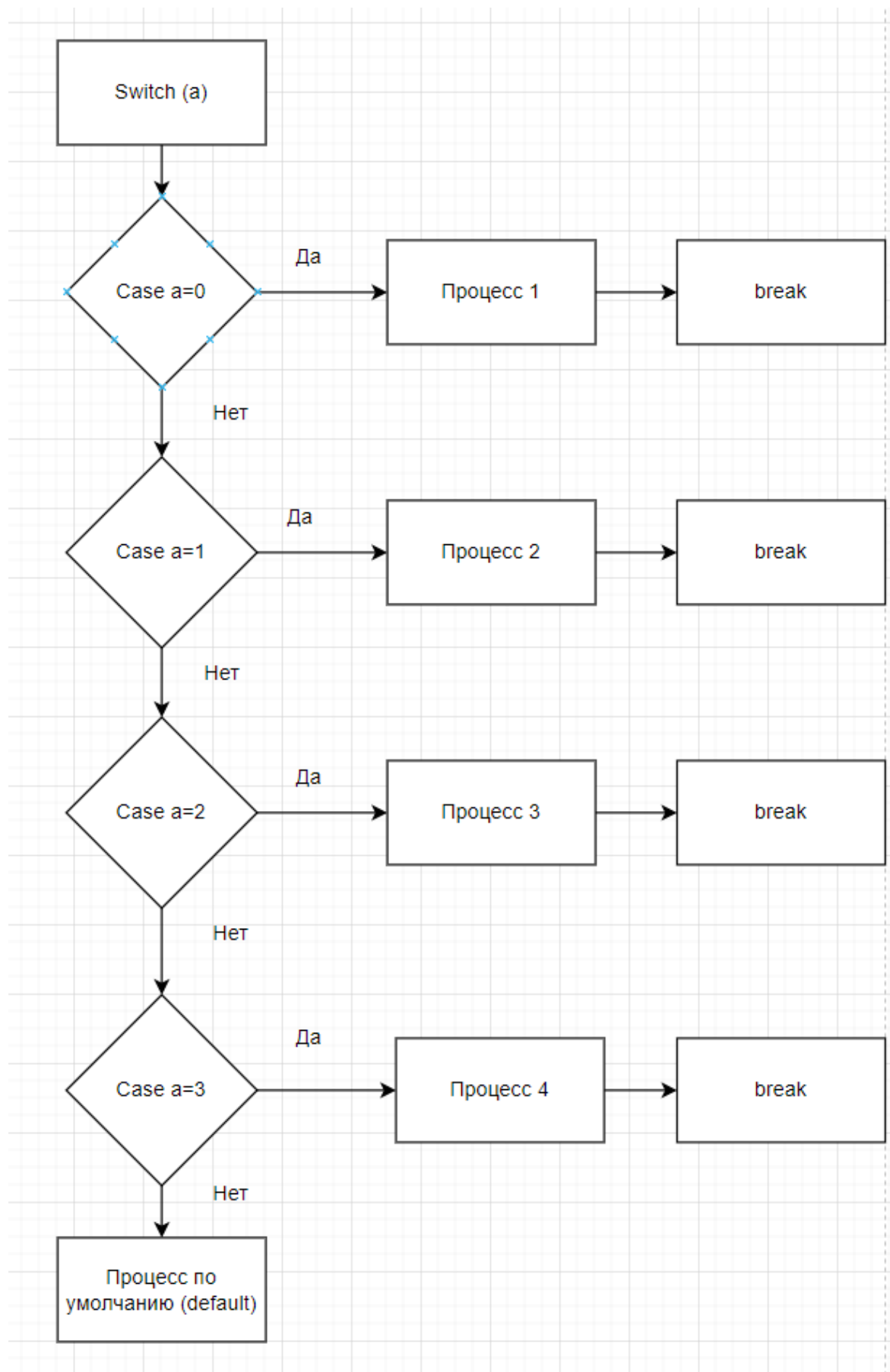


Рисунок 16. Блок-схема конструкции «switch/case/default» по вертикали

Помимо сложных условных конструкций используются также циклы. Цикл, по сути, реализует некоторый набор процессов, которые находятся в его теле и повторяются в заданной последовательности до тех пор, пока не будет осуществлен выход из него. Наиболее известными являются циклы `for`, `while` и `do while`.

Цикл со счетчиком `for`, как правило, используется, когда известны начальное и конечное значение счета и шаг счета. Реализация блок-схемы данного цикла предполагает два варианта. Первым выступает шестиугольник, в котором пишется стартовое и конечное значение счетчика, а также шаг (Рисунок 17). Другая реализация предполагает построение цикла `for` через блок процесса и блок условия. В блоке процесса, объявляется переменная счетчик (может инициализироваться уже стартовым значением счетчика). В блоке условия идет сравнение с конечным значением счета. Внутри тела цикла в блоке процесса пишется шаг счетчика. Данная реализация является более масштабной, но в то же время более полной (Рисунок 18).

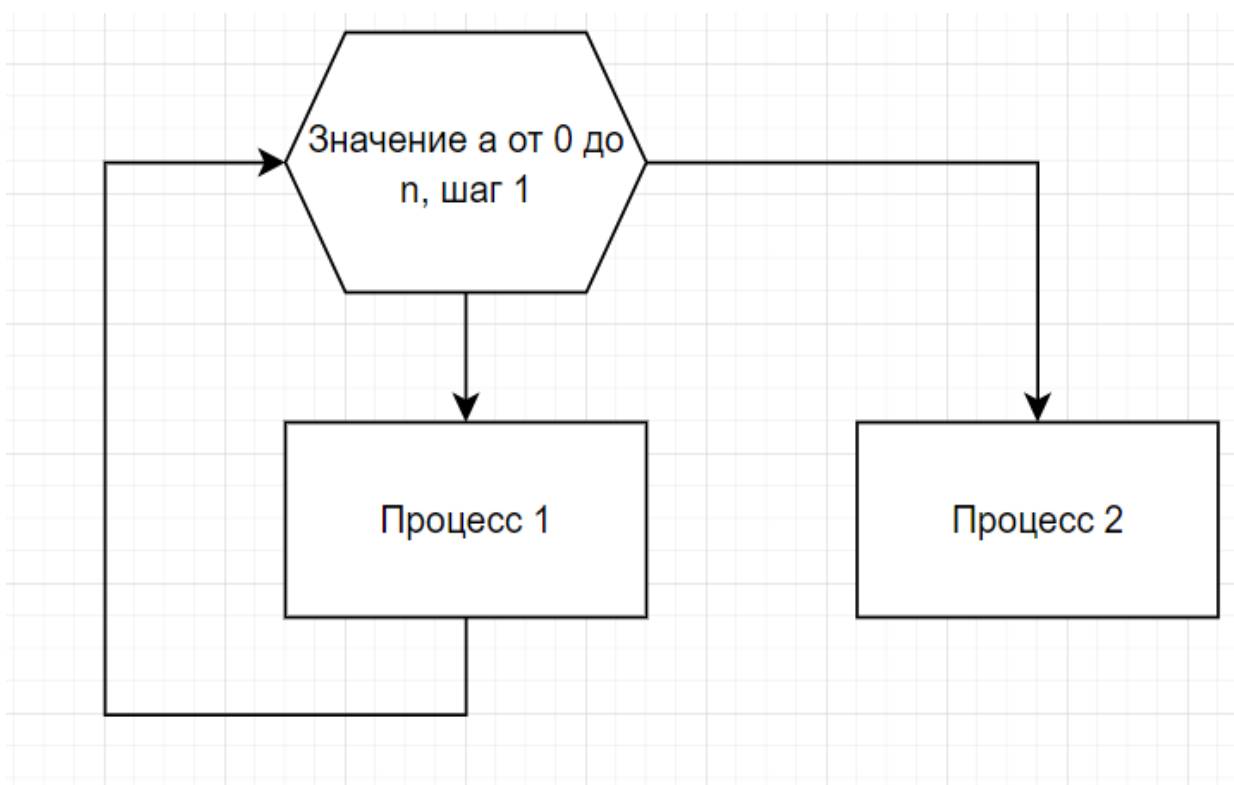


Рисунок 17. Блок-схема цикла `for` с помощью блока цикла `for`

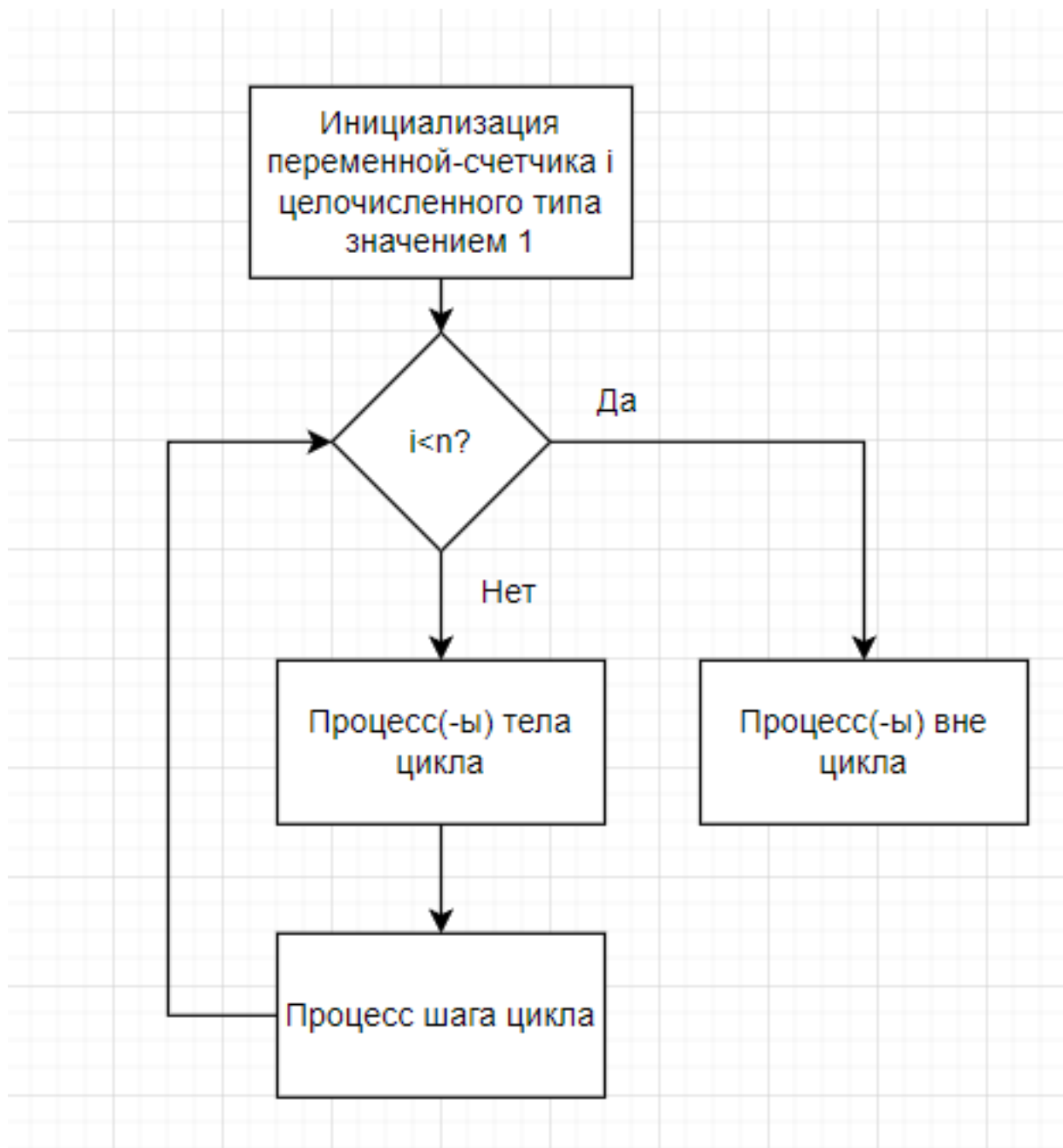


Рисунок 18. Блок-схема цикла for с помощью блоков процесса и условия

Циклы while и do while используются тогда, когда неизвестно конечное количество итераций повторяющихся процессов тела цикла, однако известно начальное значение итераций, шаг и конечный результат, который требуется получить. Отличие циклов while и do while друг от друга, главным образом, заключается в том, что в цикле while сначала идет проверка условия цикла, а затем идет процесс или набор процессов цикла, в do while наоборот.

Примеры реализации циклов while и do while представлены на рисунках 19 и 20.

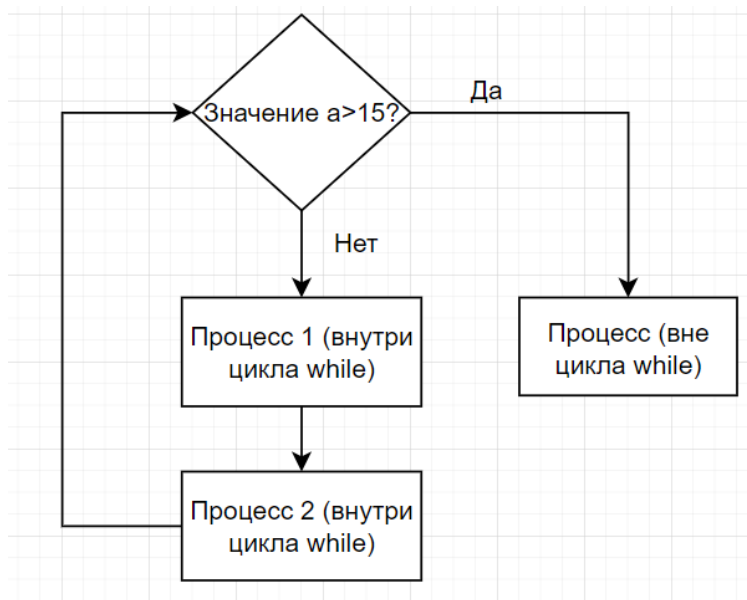


Рисунок 19. Блок-схема цикла `while`

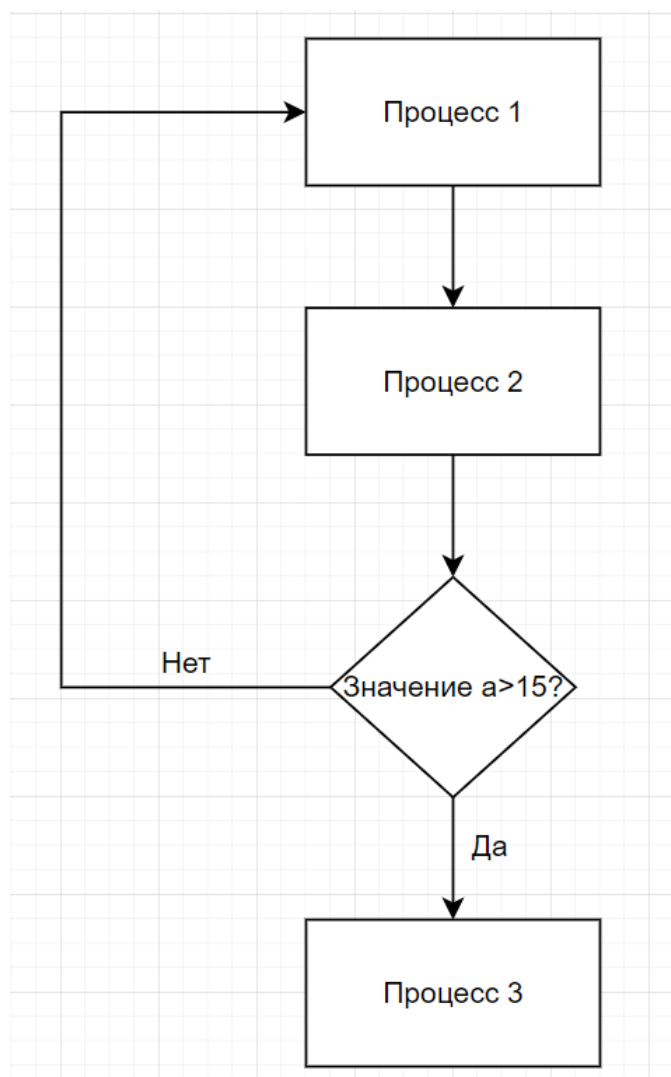


Рисунок 20. Блок-схема цикла `do while`

4. КРАТКАЯ ИСТОРИЯ ПОЯВЛЕНИЯ ЯЗЫКА C++

Изначально C++ был задуман для того, чтобы автору Б. Страуструпу и его друзьям не надо было программировать на ассемблере, С (Си) или других на тот момент современных языках высокого уровня. Основное его предназначение - упростить и сделать более приятным процесс программирования для отдельного программиста. Проектирование, реализация и документирование шли параллельно [3].

C++ многим обязан языку С, который сохраняется как его подмножество. Сохранены все свойственные языку С средства низкого уровня, предназначенные для решения самых насущных задач системного программирования.

Развитие языка C++ происходило на базе языка С, и, за небольшим исключением, С был сохранен в качестве подмножества C++. Базовый язык С был спроектирован таким образом, что имеется очень тесная связь между типами, операциями, операторами и объектами, с которыми непосредственно работает машина, т.е. числами, символами и адресами. За исключением операций new, delete и throw, а также проверяемого блока, для выполнения операторов и выражений C++ не требуется скрытой динамической аппаратной или программной поддержки.

В C++ используется та же (или даже более эффективная) последовательность команд для вызова функций и возврата из них, что и в С [3].

Язык программирования C++ задумывался как язык, который будет:

- лучше языка С;
- поддерживать абстракцию данных;
- поддерживать объектно-ориентированное программирование.

В рамках дисциплины «Объектно-ориентированное программирование» язык программирования C++ был выбран по следующим причинам:

- Возможность поддержки объектно-ориентированного программирования.
- Язык C++ кроссплатформенный и подходит для любых целей.
- Возможность работы на различных (в т.ч. старых) устройствах, так как язык не утяжеляет программы.
- Благодаря своей вычислительной мощности язык обеспечивает высокую скорость исполнения кода.
- Возможность работы с низкоуровневыми системами.

5. ОСНОВНЫЕ ПАРАДИГМЫ ПРОГРАММИРОВАНИЯ

Парадигмой программирования называют набор идей и понятий, которые определяют стиль написания программ, подход к программированию. Существует множество разных парадигм, но чаще всего используются *императивная* и *декларативная*, практически противоположные друг другу [4].

В *императивной* (англ. *imperative* — *приказ*) парадигме разработчик пишет для компьютера инструкции, которым тот следует. Инструкции могут быть следующие:

- сложи два числа;
- если а, то сделай б, а иначе сделай в;
- отправь запрос на сервер;
- открой файл;
- выведи строку.

То есть программист говорит, что нужно сделать и в каком порядке, а компьютер выполняет приказы.

Отличительная черта императивной парадигмы — понятие состояния компьютера или программы. Состояние — это совокупность всех данных в определённый момент времени: переменных, массивов, счётчиков и так далее.

К императивной парадигме относятся следующие виды программирования:

- процедурное;
- структурное;
- объектно-ориентированное и другие.

Также императивную парадигму программирования можно считать более низкоуровневой, потому что программисту нужно знать, как работают программы.

В *декларативной* парадигме разработчик описывает проблему и ожидаемый результат, но не пишет никаких инструкций. В декларативном программировании отсутствуют переменные, состояние и прочие понятия, которые свойственны императивной парадигме.

К декларативной парадигме относятся функциональное и логическое программирование.

Представим, что есть массив N, состоящий из переменных целочисленного типа от 1 до 1000, требуется найти все значения этого массива, которые будут больше 50, но меньше 200. В декларативном программировании, на примере языка SQL это будет выглядеть так:

SELECT num FROM N WHERE num > 50 AND num < 200

В императивном программировании требуется самостоятельно написать код, который выполнял бы подобную задачу, предварительно выбрав инструменты (методы) решения задачи и разработав алгоритм.

Для решения данной задачи потребуются массивы, операторы сравнения, оператор цикла for, оператор условия if, целочисленные переменные, оператор инкремента.

В императивном программировании реализация на примере языка C++ будет выглядеть следующим образом (таблица 1).

Таблица 1. Пример листинга кода императивного программирования

```
#include <iostream>
using namespace std;
int main()// точка входа в программу
{
    int n = 0;// инициализация переменной n значением 0
    int array [10] = {40,78,79,800,810,250,456,137,128,122}; // инициализация массива
array целочисленных переменных на 10 значений
    int *p= new int[n];// инициализация динамического массива целочисленных пере-
менных p
    for (int i=0; i<10; i++)// установка условий работы цикла for и его старт
    {
        if (array [i]>50 && array [i]<200)// установка правил работы условия с помощью
условного оператора if
        {
            n++;// инкремент значения индекса динамического массива
            p[i] = array[i];// элементу массива p присвоить значение элемента массива
array под индексом i
            cout<<p[i]<< endl;// вывод текущего значения массива p
        }
    }
    return 0;// возврат нуля, индикация корректности завершения программы
}
```

Итак, в декларативном программировании разработчик говорит, что ему нужно. А в императивном — как это можно получить.

6. ПАРАДИГМА ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ НА ЯЗЫКЕ C++

Первоначальной (и, возможно, наиболее используемой) парадигмой программирования было: определите, какие процедуры вам нужны; используйте лучшие из известных вам алгоритмов [3].

Ударение делалось на обработку данных с помощью алгоритма, производящего нужные вычисления. Для поддержки этой парадигмы языки предоставляли механизм передачи параметров и получения результатов функций.

Примером понимания парадигмы может служить вызов нескольких различных функций. В примере ниже отражено три функции, которые вызываются в основной функции main и возвращают разный результат. Блок-схемы функций, а также результат тестирования программы представлен на рисунках 21-25. Код решения задачи на основе разработанных блок-схем показан в таблице 2.

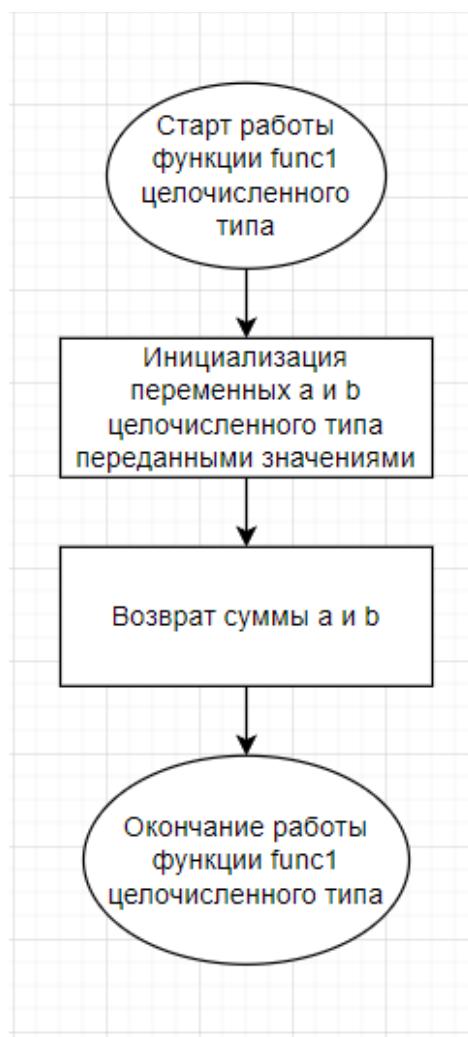


Рисунок 21. Блок-схема функции func1

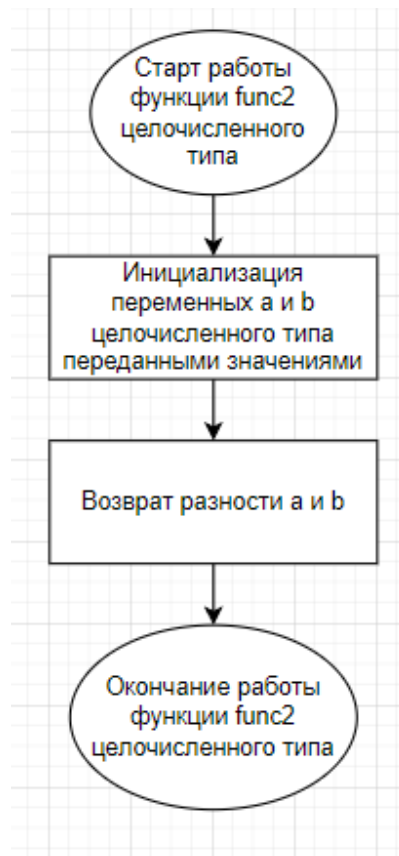


Рисунок 22. Блок-схема функции `func2`

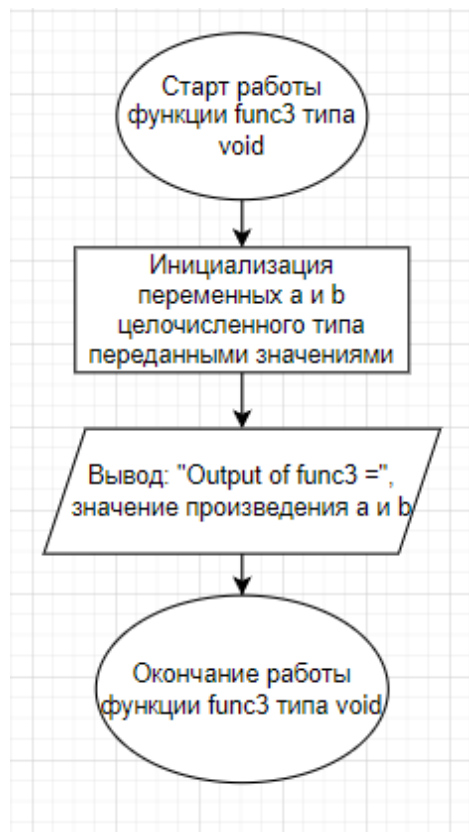


Рисунок 23. Блок-схема функции `func3`

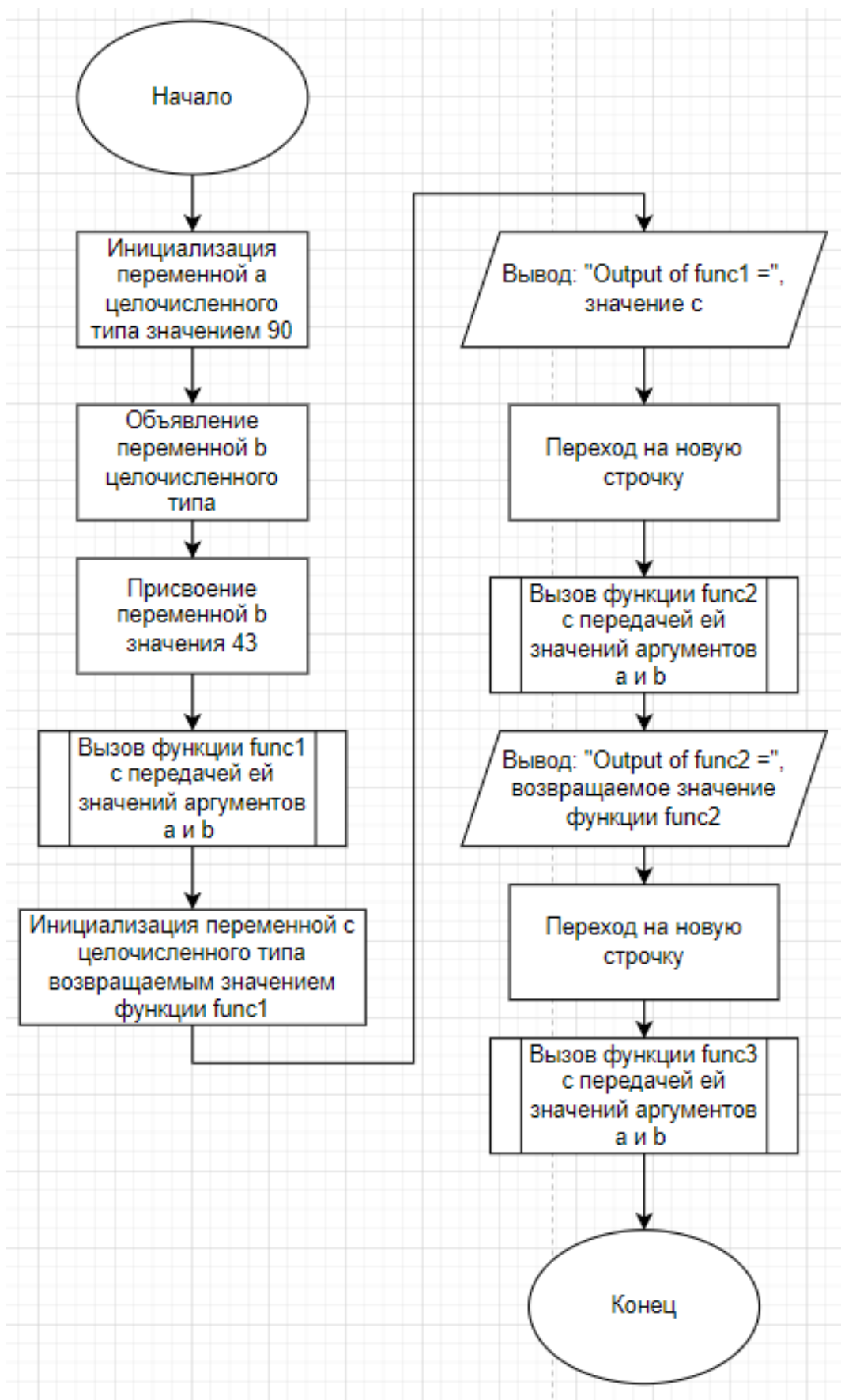


Рисунок 24. Блок-схема функции *main* задачи

Таблица 2. Листинг программы с использованием функций

```
#include <iostream>
using namespace std;

// объявляем несколько функций с разными именами
int func1(int a, int b)
{
    return a + b; // вернет значение суммы параметров 'a' и 'b'
}

int func2(int a, int b)
{
    return a - b; // вернет значение разности параметров 'a' и 'b'
}

void func3(int a, int b) // данная функция объявлена как void, то есть она ничего не воз-
вращает, ключевое слово 'return', в таких функциях не требуется
{
    cout << "Output of func3 = " << a * b; //вывод в консоль значения произведения па-
раметров 'a' и 'b'
}

int main()
{
    int a = 90; // инициализация целочисленной переменной 'a' значением 90
    int b; // объявление целочисленной переменной 'b'
    b = 43; // присвоение переменной 'b' значения 43
    int c = func1(a, b); // инициализация целочисленной переменной 'c' значением, кото-
рое вернет функция 'func1' с аргументами 'a' и 'b'
    cout << "Output of func1 = " << c << "\n"; // вывод в консоль значения переменной 'c'
    cout << "Output of func2 = " << func2(a, b) << "\n"; // вывод в консоль значения кото-
рое вернет функция 'func2' с аргументами 'a' и 'b'
    func3(a, b); // вызов функции func3 с аргументами 'a' и 'b'
    return 0;
}
```

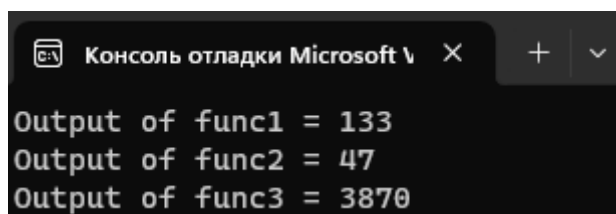


Рисунок 25. Тестирование программного кода со значением

При такой организации программы функции вносят определенный порядок в хаос различных алгоритмов.

7. НЕКОТОРЫЕ ИНСТРУМЕНТЫ ПРОЦЕДУРНОГО ПРОГРАММИРОВАНИЯ

7.1. Программа и основная функция main

Решение задачи с использованием того или иного языка программирования предполагает написание программы. *Программа* — это описание на языке программирования структур данных и алгоритма решения задачи, автоматически переводимое, при помощи специальной программы-транслятора (компилятора или интерпретатора), на язык команд компьютера для последующего выполнения [5].

Иначе говоря, компьютерная программа – один из способов реализации понятия алгоритма, а язык программирования – средство описания алгоритмов.

Основной точкой входа в программу является функция main. В каждой программе на языке C++ должна быть своя функция main(). Функция main вызывается при старте программы после инициализации нелокальных объектов со статическим классом памяти [6]. Это точка входа в программу, которая исполняется в гостевом окружении (то есть с операционной системой). Точки входа в автономные программы (загрузчики, ядра ОС, и т.п.) зависят от реализации.

Параметры функции main в варианте с двумя параметрами позволяют передать произвольные многобайтовые строки из среды выполнения (обычно известных как аргументы командной строки), указатели argv[1] .. argv[argc-1] ссылаются на первые символы этих строк. argv[0] (если не нуль) это указатель на первый символ многобайтовой строки с завершающим нулём, которая содержит имя, используемое при вызове программы (или пустую строку "", если это не поддерживается средой выполнения). Эти строки изменяемые, хотя их изменения не распространяются назад в среду выполнения. Размер массива, на который указывает argv, равен, по меньшей мере, argc+1, и последний элемент массива argv[argc] гарантированно является пустым указателем [7].

Самая короткая программа на языке C++ представлена в таблице 3.

Таблица 3. Листинг самой короткой программы на языке C++

```
void main (void)
{
}
```

В этой программе определяется функция, называемая main, которая не имеет параметров и ничего не делает. Фигурные скобки «{}» используются в

C++ для группирования операторов. В данном случае они обозначают начало и конец тела функции `main`.

Однако в вышеприведенном случае, нет точки выхода программы, поэтому не каждый компилятор может понять и принять данную запись.

7.2. Стандартный ввод и вывод

Поточный ввод-вывод в C++ выполняется с помощью функций сторонних библиотек. В C++, как и в C, нет встроенных в язык средств ввода-вывода. В языке программирования C++ для реализации ввода-вывода данных разработана библиотека `iostream`.

Библиотека `iostream` определяет три стандартных потока:

- `cin` — объект стандартного входного потока;
- `cout` — объект стандартного выходного потока;
- `cerr` — объект стандартного потока вывода сообщений об ошибках;

Для выполнения операций ввода-вывода переопределены две операции поразрядного сдвига:

- `>>` — получить из входного потока;
- `<<` — поместить в выходной поток;

Для вывода информации используется следующая конструкция (Рисунок 26):

`cout << значение;`

Рисунок 26. Конструкция для вывода данных с помощью объекта `cout`

Здесь значение преобразуется в последовательность символов и выводится в выходной поток. Возможно многократное использование потоков. В таком случае конструкция может выглядеть следующим образом (Рисунок 27):

`cout << 'значение1' << 'значение2' << ... << 'значение n';`

Рисунок 27. Конструкция для многократного вывода данных с помощью объекта `cout`

Для вывода информации используется следующая конструкция (Рисунок 28):

cin >> идентификатор;

Рисунок 28. Конструкция для ввода данных с помощью объекта cin

При этом из входного потока читается последовательность символов до пробела, затем эта последовательность преобразуется к типу идентификатора, и получаемое значение помещается в идентификатор.

Также возможно многократное назначение потоков. В таком случае конструкция будет иметь следующий вид (Рисунок 29).

cin >> переменная1 >> переменная2 >>...>> переменнаяn;

Рисунок 29. Конструкция для многократного ввода данных с помощью объекта cout

Также для работы с входными и выходными данными могут использоваться манипуляторы потока. В C++ наиболее используемыми являются манипуляторы потока представленные в таблице 4.

Таблица 4. Манипуляторы потока в языке C++

Манипулятор	Описание
endl	Помещение в выходной поток символа конца строки '\n'
dec	Установка основания 10-ой системы счисления
oct	Установка основания 8-ой системы счисления
hex	Установка основания 16-ой системы счисления
setbase	Вывод базовой системы счисления
width(ширина)	Устанавливает ширину поля вывода
fill('символ')	Заполняет пустые знакоместа значением символа

Манипулятор	Описание
precision(точность)	Устанавливает количество значащих цифр в числе (или после запятой) в зависимости от использования fixed
fixed	Показывает, что установленная точность относится к количеству знаков после запятой
showpos	Показывает знак + для положительных чисел
scientific	Выводит число в экспоненциальной форме
get()	Ожидает ввода символа
getline(указатель, количество)	Ожидает ввода строки символов. Максимальное количество символов ограничено полем количество

7.3. Переменные и их типы

Часто в программах используются переменные и их описание.

Переменная — это именованная область памяти. **Описание** — это оператор, который вводит имя в программу. В описании указывается тип имени. Тип, в свою очередь, определяет, как правильно использовать имя или выражение.

В языке C++ все переменные имеют определенный тип данных. Тип данных присваивается переменной при ее объявлении или инициализации.

Объявление переменной — это действие, в котором сначала указывается тип данных, а затем название этой переменной.

Присвоение значения переменной — это действие, в котором ранее объявленной переменной присваивается некоторое значение (значение записывается в именованную область памяти).

Инициализация переменной — это действие, в котором одновременно происходит и объявление переменной, и присвоение ей значения.

При реализации алгоритма в виде блок-схемы объявление, присвоение и инициализация, как правило, пишутся в блоке процесса. Примеры объявления, инициализации и присвоения значений переменной представлены в таблице 5.

Таблица 5. Листинг кода с примерами объявления, присвоения и инициализации переменной

```
#include <iostream>
using namespace std;

int main()
{
    int a; // пример объявления переменной
    int b = 0; // пример инициализации переменной значением ноль
    a = 999; // пример присвоения некоторого значения, существующей переменной
    c = 10; //присвоение значения несуществующей переменной невозможно, так как она
    не была объявлена ранее
    return 0;
}
```

Основные типы переменных, их размер и диапазон значений представлен в таблице 6.

Таблица 6. Типы, размеры и диапазон значений переменных

Тип	Размер в битах	Диапазон
char	8	-128 — 127
unsigned char	8	0 — 255
signed char	8	-128 — 127
int	16	-32768 — 32767
unsigned int	16	0 — 65535
signed int	16	-32768 — 32767
short int	16	-32768 — 32767
unsigned short int	16	0 — 65535
signed short int	16	-32768 — 32767
long int	32	-2 147 483 648 — 2 147 483 647
signed long int	32	-2 147 483 648 — 2 147 483 647
unsigned long int	32	0 — 4 294 967 295
float	32	3,4E-38 — 3,4E+38
double	64	1,7E-308 — 1,7E+308
long double	80	3,4E-4932 — 1,1E+4932

Часто в программах требуется использовать типы переменных, которые обозначают целые значения или значения с плавающей точкой.

Среди данных типов целые значения представляют следующие:

- char
- short
- int
- long

Типы, которые представляют значения с плавающей точкой:

- float
- double
- long double

Логический тип переменной представляется bool, Тип переменной перечислителя называется enum.

Ниже приведен пример задачи, где буду рассмотрена работа с различными типами переменных. Блок-схема и результат работы программы представлен на рисунках 30, 31, листинг кода на основе разработанной блок-схемы представлен в таблице 7.

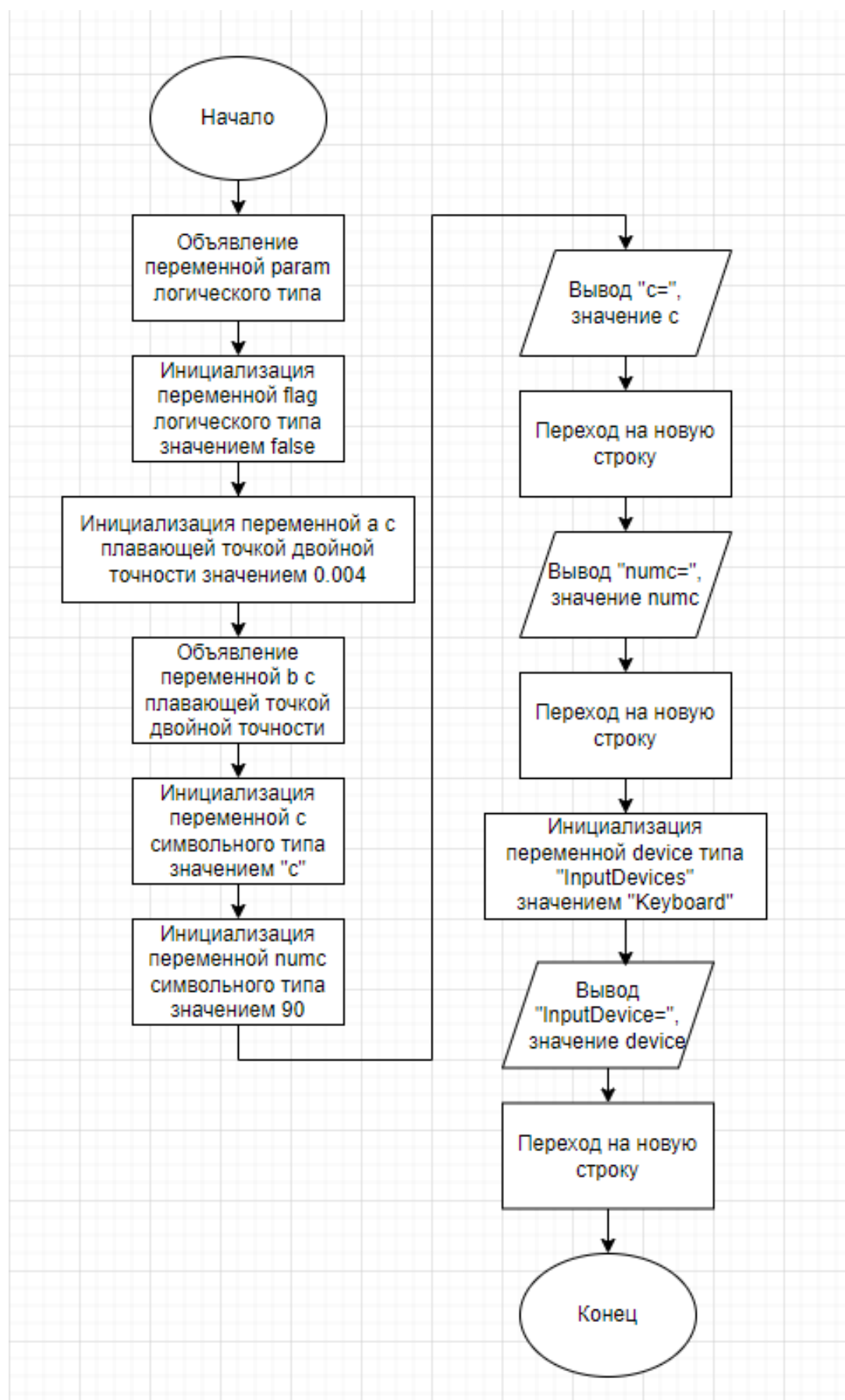


Рисунок 30. Блок-схема алгоритма решения задачи реализации работы с разными типами переменных

Таблица 7. Листинг программы, показывающей работу с различными типами переменных

```
#include <iostream>
using namespace std;

enum InputDevices // объявляем перечисление, с названием InputDevices
{
    Keyboard, // начальный номер данного устройства 0 (значение по умолчанию)
    Mouse,
    Gamepad,
    SteeringWheel,
    Joystick,
};
int main()
{
    bool param; //пример объявления переменной логического типа
    bool flag = false; //пример инициализации переменной логического типа значением false

    double a = 0.004; //пример инициализации переменной с плавающей точкой двойной точности
    double b; //пример объявления переменной с плавающей точкой двойной точности

    char c = 'c'; //пример инициализации переменной символьного типа значением 'c'
    char numc = 90; //пример инициализации переменной символьного типа значением '90',
    при выводе в консоль это значение превратится в букву 'Z'

    cout << "c = " << c << "\n"; // вывод в консоль значения переменной c
    cout << "numc = " << numc << "\n"; // вывод в консоль значения переменной numc
    InputDevices device = Keyboard; // инициализация переменной device типа "InputDevices"
    значением "Keyboard"
    cout << "InputDevice = " << device << "\n"; // вывод в консоль значения переменной
    device, в консоль будет выведено значение 0, так как в предыдущей строке мы инициализи-
    ровали эту переменную значением Keyboard
    return 0;
}
```

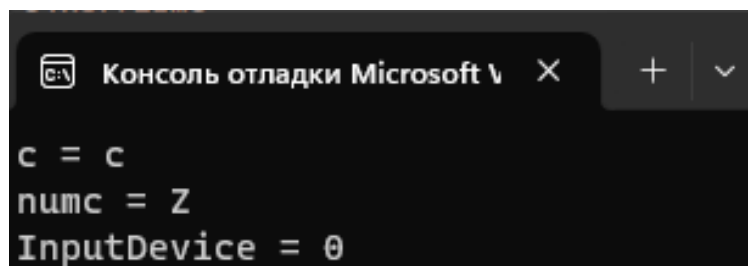


Рисунок 31. Тестирование программного кода со значением

7.4. Константные переменные

Константы обычно нужны, чтобы хранить какие-то постоянные величины, которые в рамках программы никак не будут меняться. Целые константы могут появляться в четырех обличиях: десятичные, восьмеричные, шестнадцатеричные и символьные.

Примеры десятичной константы выглядят следующим образом: 0, 12345, 5674839201, 1234567890 и т.д.

Десятичная константа, имеющая тип `int`, должна уместиться в память, в случае если этого не происходит транслятор должен предупреждать об этом и тип должен быть изменен на `long int`.

Константа, которая начинается с комбинации `0x` – является **шестнадцатеричной константой**. Буквы A, B, C, D, E, F используются для представления чисел 10, 11, 12, 13, 14 и 15 соответственно. Примером шестнадцатеричной константы могут быть следующие значения: `0x0`, `0x3f`, `0x23`, `0x45`. Их десятичным эквивалентом будут соответственно: 0, 63, 35, 69.

Константа, которая начинается с комбинации `0` – является **восьмеричной константой**. Ее примером могут быть следующие значения: `02`, `027`, `045`, `0123`. Их десятичным эквивалентом будут соответственно: 2, 23, 37, 83.

Символьной константой является символ, заключенный в одинарные кавычки, например `'a'` или `'0'`. Символьные константы можно считать константами, которые дают имена целым значениям символов из набора, принятого на машине, на которой выполняется программа. Так, например, если на машине используется набор символов ASCII, то значение `'0'` равно 48, а если, например, используется набор символов EBCDIC, то значение будет равно 240.

Использование символьных констант вместо их десятичного эквивалента повышает переносимость программ. Существуют некоторые специальные комбинации символов, начинающиеся с обратной дробной черты, имеют стандартные названия (рисунок 32).

Константы, если они задаются значениями с плавающей точкой, могут иметь тип `double` или `float`. Примеры задания констант типа `double` выглядят следующим образом: `1.0`, `1.23`, `0.23`, `.23` и т.д.

В случае если требуется задать константу типа `float`, то в таком случае нужно дописать в окончании значения букву `<f>`. В таком случае примеры будут выглядеть следующим образом: `2.0f`, `1.23f`, `2.9994455f` и т.д.

Конец строки	NL (LF)	\n
Горизонтальная табуляция	HT	\t
Вертикальная табуляция	VT	\v
Возврат	BS	\b
Возврат каретки	CR	\r
Перевод формата	FF	\f
Сигнал	BEL	\a
Обратная дробная черта	\	\\
Знак вопроса	?	\?
Одиночная кавычка	'	\'
Двойная кавычка	"	\"
Нулевой символ	NUL	\0
Восьмеричное число	ooo	\ooo
Шестнадцатеричное число	hhh	\xhhh

Рисунок 32. Некоторые специальные комбинации символов

7.5. Основные арифметические, логические операции и операторы сравнения

Существует следующий набор арифметических операций, который может производиться над любым сочетанием типов:

- + (плюс, унарный или бинарный)
- - (минус, унарный или бинарный)
- * (умножение)
- / (деление)
- % (остаток от деления)

Существует также набор логических операторов, позволяющих проводить логические действия:

- && (Логическое И)
- || (Логическое ИЛИ)
- ! (Отрицание)

Также для любых сочетаний типов могут производиться следующие операции сравнения:

- == (равно)
- != (не равно)
- < (меньше чем)
- <= (меньше или равно)
- >= (больше или равно)

Для операций присваивания, где символ "=" означает присваивание, и арифметических операций в C++ выполняются все осмысленные преобразования основных типов, чтобы их можно было неограниченно использовать, любые их сочетания.

Ниже приведен пример задачи, которая показывает в рамках работы условных операторов, работу некоторых арифметических, логических операторов, а также неявное приведение типов. Блок-схема и результат выполнения программы показан на рисунках 33 и 34, а листинг кода на основе разработанной блок-схемы представлен в таблице 8.

В данном примере также используются условные оператор if, а также цикл for, речь о которых пойдет в следующем разделе.

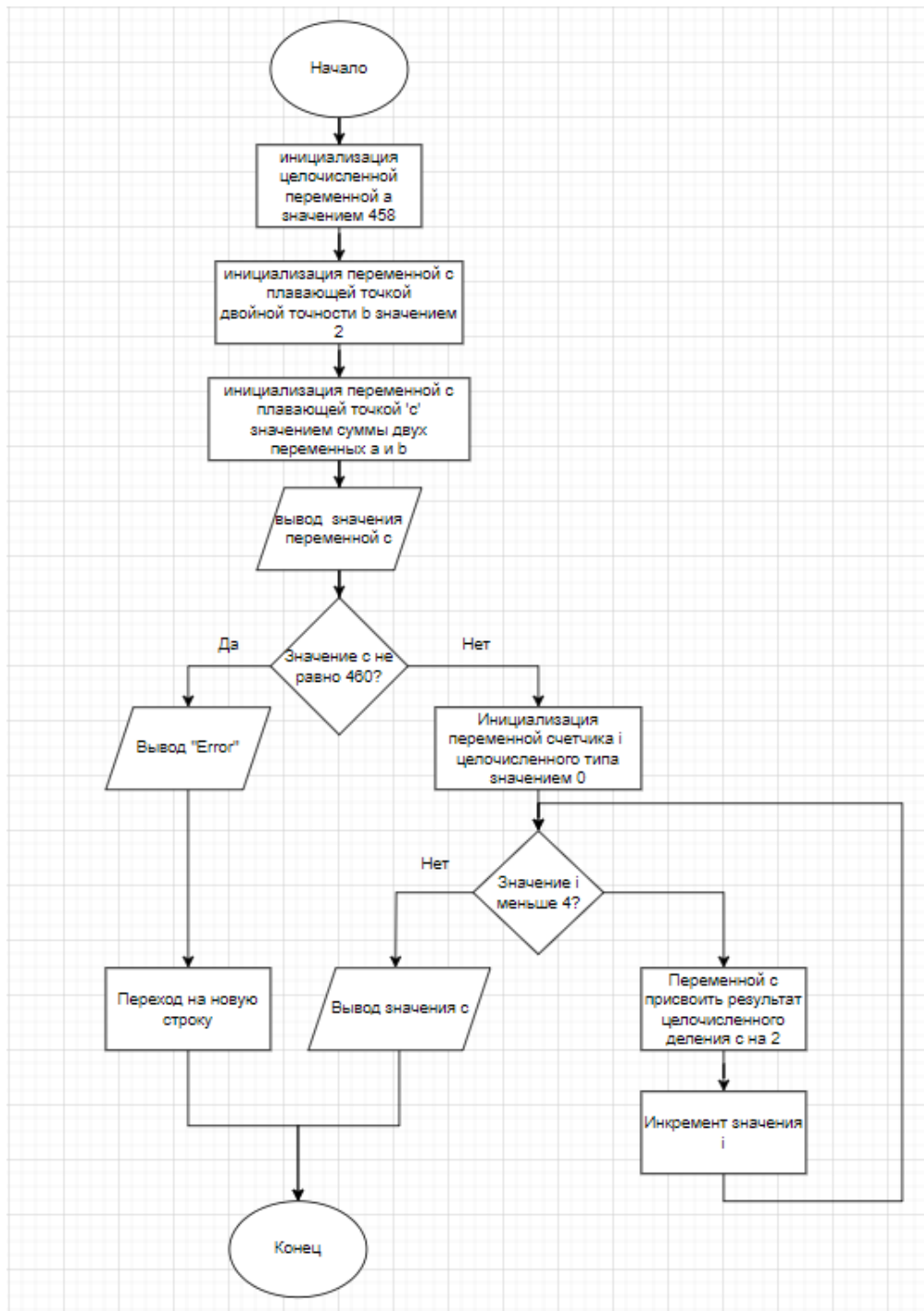


Рисунок 33. Блок-схема решения задачи

Таблица 8. Листинг программы, показывающей работу различных типов операторов и неявное приведение типов

```
#include <iostream>
using namespace std;

int main()
{
    int a = 458; // инициализация целочисленной переменной 'a' значением 458
    double b = 2; // инициализация переменной с плавающей точкой двойной точности 'b'
    // значением 2

    float c = a + b; // инициализация переменной с плавающей точкой 'c' значением суммы
    // двух предыдущих
    cout << c; // вывод в консоль значения переменной 'c'

    if (c != 460) cout << "Error!" << "\n"; // Операции сравнения '!=' (не равно) удобно использовать
    // в условиях
    for (int i = 0; i < 4; i++) // объявляем цикл for, устанавливаем начальное значение счет-
    // чика равное нулю, цикл будет выполняться пока счетчик i '<' меньше 4, увеличение счет-
    // чика идет с помощью операции инкремент '++'
    {
        c = c / 2;
    }
    cout << c; // вывод в консоль значения переменной 'c'

    return 0;
}
```

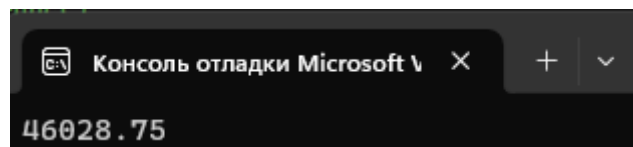


Рисунок 34. Тестирование программного кода со значением

7.6. Условные операторы и циклы

В языке программирования C++ в качестве основных используемых **условных операторов** — операторов, которые указывают, следует ли выполнять какие-то связанные с ними инструкции, выступают: if, switch.

В качестве примера реализации оператора if будет взята следующая задача: требуется проверить на четность или нечетность вводимое пользователем число. В случае, когда введенная переменная, при делении по модулю 2 будет иметь нулевой остаток, означая что число четное, то вывести на экран слово «even», в

противном случае вывести слово «uneven», что будет означать, что число не четное.

Решение текущей задачи предполагает разработку алгоритма решения задачи, которая будет представлена в виде блок-схемы (рисунок 35), написание программного кода на языке C++ (таблица 9) и его тестирование (рисунок 36)

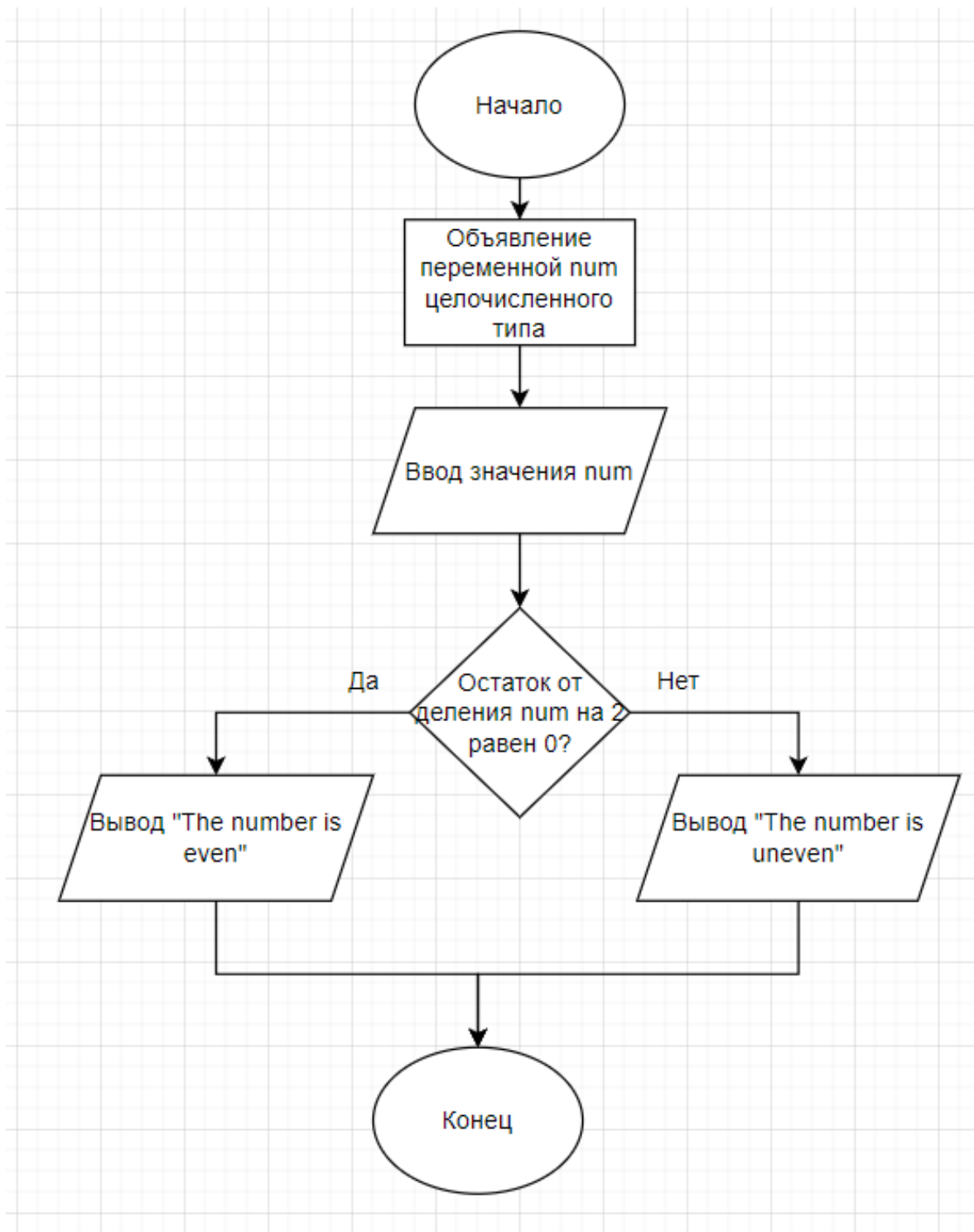


Рисунок 35. Блок-схема алгоритма решения задачи проверки числа на четность

Таблица 9. Листинг программы решения задачи на проверку числа на четность

```
#include <iostream>
using namespace std;

int main()
{
    int num; // объявление целочисленной переменной 'num'
    cout << "Enter the number: "; // вывод сообщения о вводе значения переменной 'num'
    cin >> num; // ввод значения переменной 'num'
    if (num % 2 == 0) cout << "The number is even!"; // условие, если остаток от деления
на 2 равен 0, то вывод сообщения, что число четное
    else cout << "The number is uneven!"; // в противном случае, вывод на экран сообще-
ния, что число не четное
    return 0;
}
```

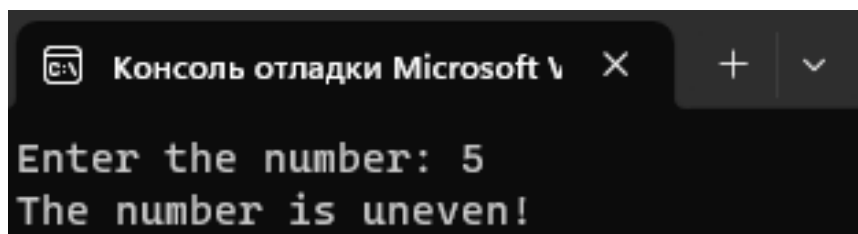


Рисунок 36. Тестирование программного кода со значением

В случае если нужно проверить более двух путей условия используется оператор else if.

Примером его реализации может выступить такое условие задачи: требуется проверить вводимое пользователем букву английского алфавита на гласность или согласность. В случае если гласная вывести «vowel», иначе вывести «consonant». В случае если введено что-то другое вывести «Not letter»

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 37 и 38. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 10.

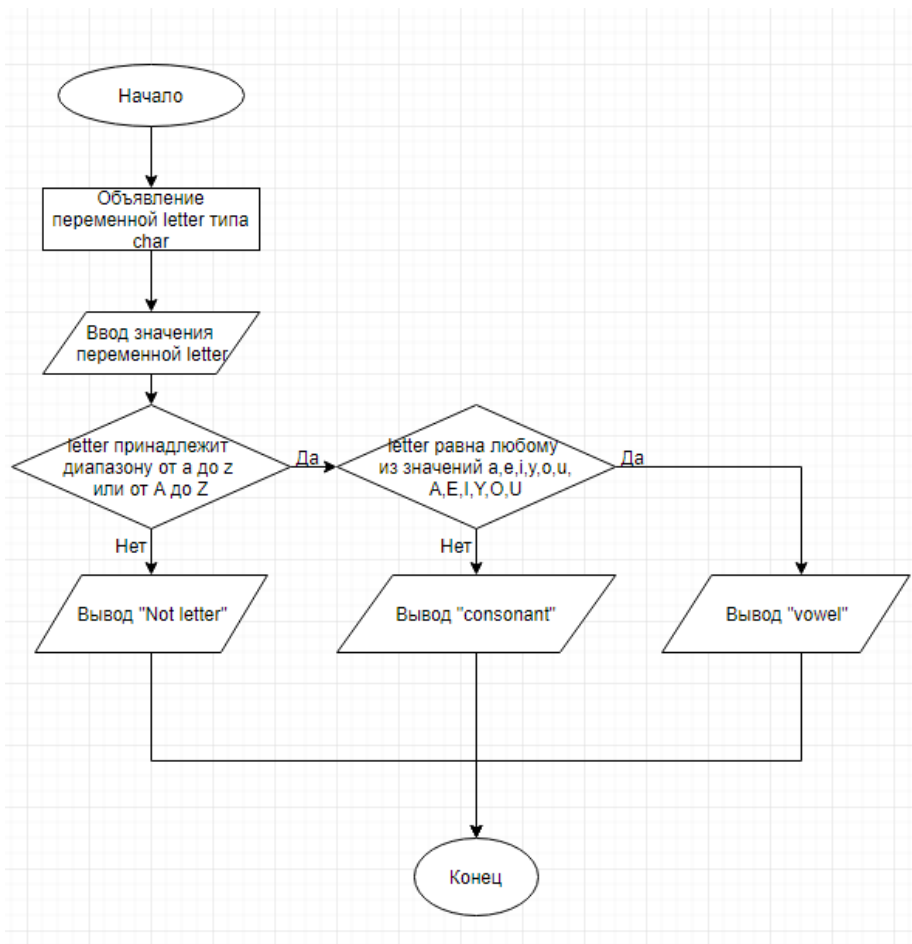


Рисунок 37. Блок-схема алгоритма решения задачи

Таблица 10. Листинг программы решения задачи на проверку буквы на гласность или согласность

```

#include <iostream>
using namespace std;

int main()
{
    char letter; // объявление символьной переменной letter
    std::cin >> letter; // Ввод значения переменной letter
    if ((letter >= 'a' && letter <= 'z') || (letter >= 'A' && letter <= 'Z')) // Условие, в котором идет проверка принадлежности введенного символа буквам английского алфавита
    {
        if (letter == 'a' || letter == 'e' || letter == 'i' || letter == 'o' || letter == 'u' || letter == 'y') // второе условие, которое проверяет принадлежность введенного символа малым гласным буквам английского алфавита
            std::cout << "vowel"; // вывод на экран надписи 'vowel'
        else if (letter == 'A' || letter == 'E' || letter == 'I' || letter == 'O' || letter == 'U' || letter == 'Y') // третье условие, которое проверяет принадлежность введенного символа заглавным гласным буквам английского алфавита
            std::cout << "vowel"; // вывод на экран надписи 'vowel'
        else std::cout << "consonant"; // вывод на экран надписи 'consonant'
    }
}
  
```



```

    }
    else
        std::cout << "Not letter"; // Если первое условие не выполнилось, то выводим
        на экран сообщение, что введенный символ не принадлежит буквам английского алфавита
        return 0;
}

```

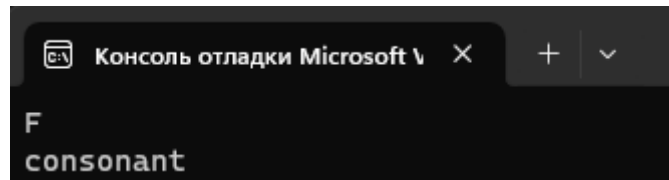


Рисунок 38. Тестирование программного кода со значением

Оператор `switch` в C++ — это оператор управления потоком, который используется для выполнения различных блоков процессов на основе значения данного выражения (переключатель). Иначе говоря, он сравнивает полученное значение с набором констант.

В качестве примера реализации оператора `switch` будет разобрана такая задача: необходимо организовать ввод с клавиатуры двух целочисленных переменных, а также знака математической операции. Для математических операций используем стандартный набор: «+» (плюс), «-» (минус), «*» (умножить), «/» (поделить). Вывести на экран значение результата выбранной математической операции между двумя целочисленными переменными.

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 39 и 40. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 11.

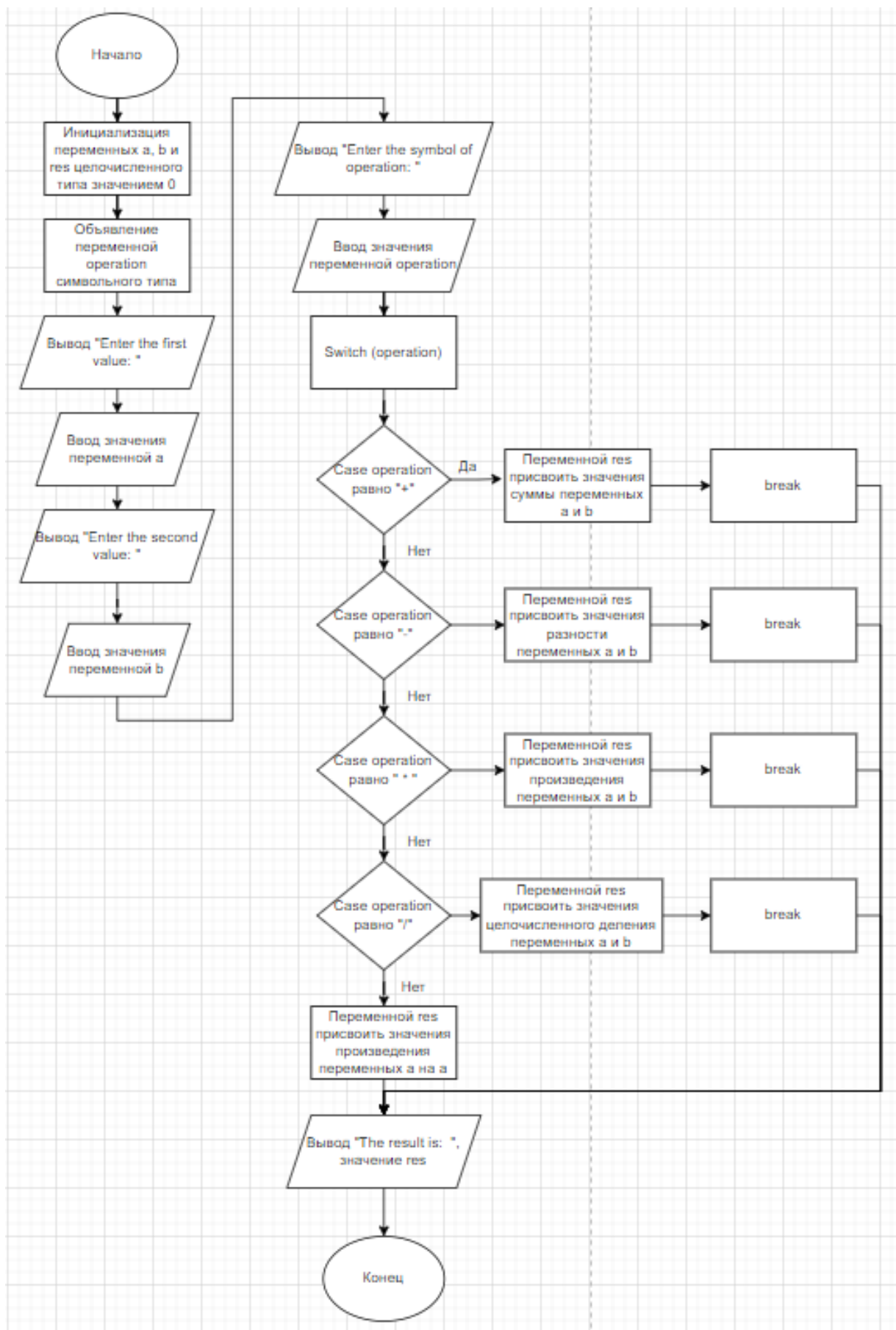


Рисунок 39. Блок-схема алгоритма решения задачи

Таблица 11. Листинг программы решения задачи с использованием
switch/case/default

```
#include <iostream>
using namespace std;

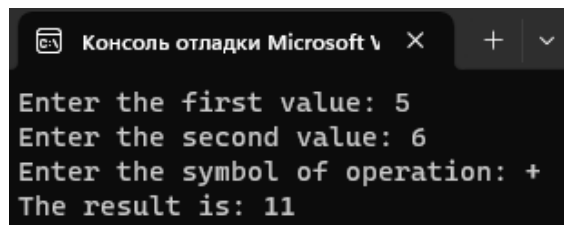
int main()
{
    int a, b, res = 0; // инициализируем три целочисленные переменные значением нуля
    char operation; // объявляем переменную, которая будет отвечать за знак операции
    cout << "Enter the first value: "; // вывод сообщения с просьбой ввода значения для первой
    переменной

    cin >> a; // ввод значения первой переменной
    cout << "Enter the second value: "; // Вывод сообщения с просьбой ввода значения для
    второй переменной
    cin >> b; // ввод значения второй переменной
    cout << "Enter the symbol of operation: "; // вывод сообщения с просьбой ввода значения
    знака операции

    cin >> operation; // ввод знака операции
    switch (operation) // объявление оператора switch, с условием 'operation'
    {
        case ('+'): // в случае если введенный знак операции это '+', то вычислить сумму двух вве-
        денных переменных
            res = a + b;
            break;
        case ('-'): // в случае если введенный знак операции это '-', то вычислить разность двух
        введенных переменных
            res = a - b;
            break;
        case ('*'): // в случае если введенный знак операции это '*', то вычислить произведение
        двух введенных переменных
            res = a * b;
            break;
        case ('/'): // в случае если введенный знак операции это '/', то вычислить частное двух вве-
        денных переменных

            res = a / b;
            break;
        default: // в случае если введен некорректное значение знака операции, то вычислить
        квадрат значения первой переменной

            res = a * a;
            break;
    }
    cout << "The result is: " << res; // вывод на экран значения переменной 'res'
    return 0;
}
```



```
Консоль отладки Microsoft V  X + v
Enter the first value: 5
Enter the second value: 6
Enter the symbol of operation: +
The result is: 11
```

Рисунок 40. Тестирование программного кода со значением

Оператор break в данном случае используется для выхода из переключателя. За пример возьмем предыдущую реализацию, но с небольшими изменениями. Уберем в случаях с плюсом и минусом операторы break. В результате отсутствия этого оператора, выполнение оператора case не будет остановлено, и программа продолжит выполнение дальше, пока не встретит оператор break. Листинг кода показан в таблице 12. Результат показан на рисунке 41.

Таблица 12. Листинг программы решения задачи с использованием switch/case/default при неиспользовании оператора break

```
#include <iostream>
using namespace std;

int main()
{
    int a, b, res = 0; // объявляем две целочисленные переменные, и инициализируем переменную результата значением нуля
    char operation; // объявляем переменную, которая будет отвечать за знак операции
    cout << "Enter the first value: "; // вывод сообщения с просьбой ввода значения для первой переменной
    cin >> a; // ввод значения первой переменной
    cout << "Enter the second value: "; // Вывод сообщения с просьбой ввода значения для второй переменной
    cin >> b; // ввод значения второй переменной
    cout << "Enter the symbol of operation: "; // вывод сообщения с просьбой ввода значения знака операции
    cin >> operation; // ввод знака операции
    switch (operation) // объявление оператора switch, с условием 'operation'
    {
        case ('+'): // в случае если введенный знак операции это '+', то вычислить сумму двух введенных переменных
            res = a + b;

        case ('-'): // в случае если введенный знак операции это '-', то вычислить разность двух введенных переменных
            res = a - b;
```

```

    case ('*'): // в случае если введенный знак операции это '*', то вычислить произведение
двух введенных переменных
        res = a * b;
        break;
    case ('/'): // в случае если введенный знак операции это '/', то вычислить частное двух вве-
денных переменных
        res = a / b;
        break;
    default: // в случае если введен некорректное значение знака операции, то вычислить квад-
рат значения первой переменной
        res = a * a;
        break;
}
cout << "The result is: " << res; // вывод на экран значения переменной 'res'
return 0;
}

```

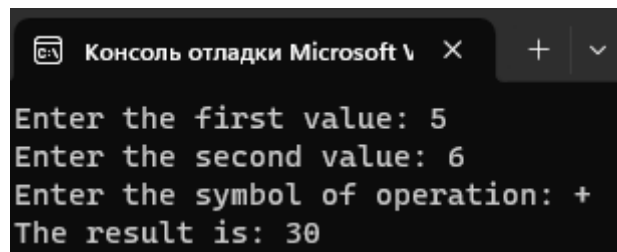


Рисунок 41. Тестирование программного кода со значением

В результате можно увидеть, что при выбранной операции «+» (сложение), мы получили число 30, что соответствует произведению двух чисел.

Цикл — это повторение одного и того же участка кода в программе. Последовательность действий, которые повторяются, называют **телом цикла**. Один проход цикла — это **шаг** или **итерация**. Переменные, изменяющиеся внутри цикла и влияющие на его окончание, называются **параметрами цикла**. В языке программирования C++ предусмотрены три оператора, реализующих циклический процесс: `while`, `do while` и `for`.

Особенность цикла со счетчиком `for` заключается в том, что он выполняет инструкции внутри своего тела в течение определенного количества итераций или до тех пор, пока данное условие не станет ложным.

Примером реализации цикла `for` может служить следующая задача: требуется вывести сумму всех натуральных чисел от 1 до N, значение N вводит пользователь.

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 42 и 43. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 13.

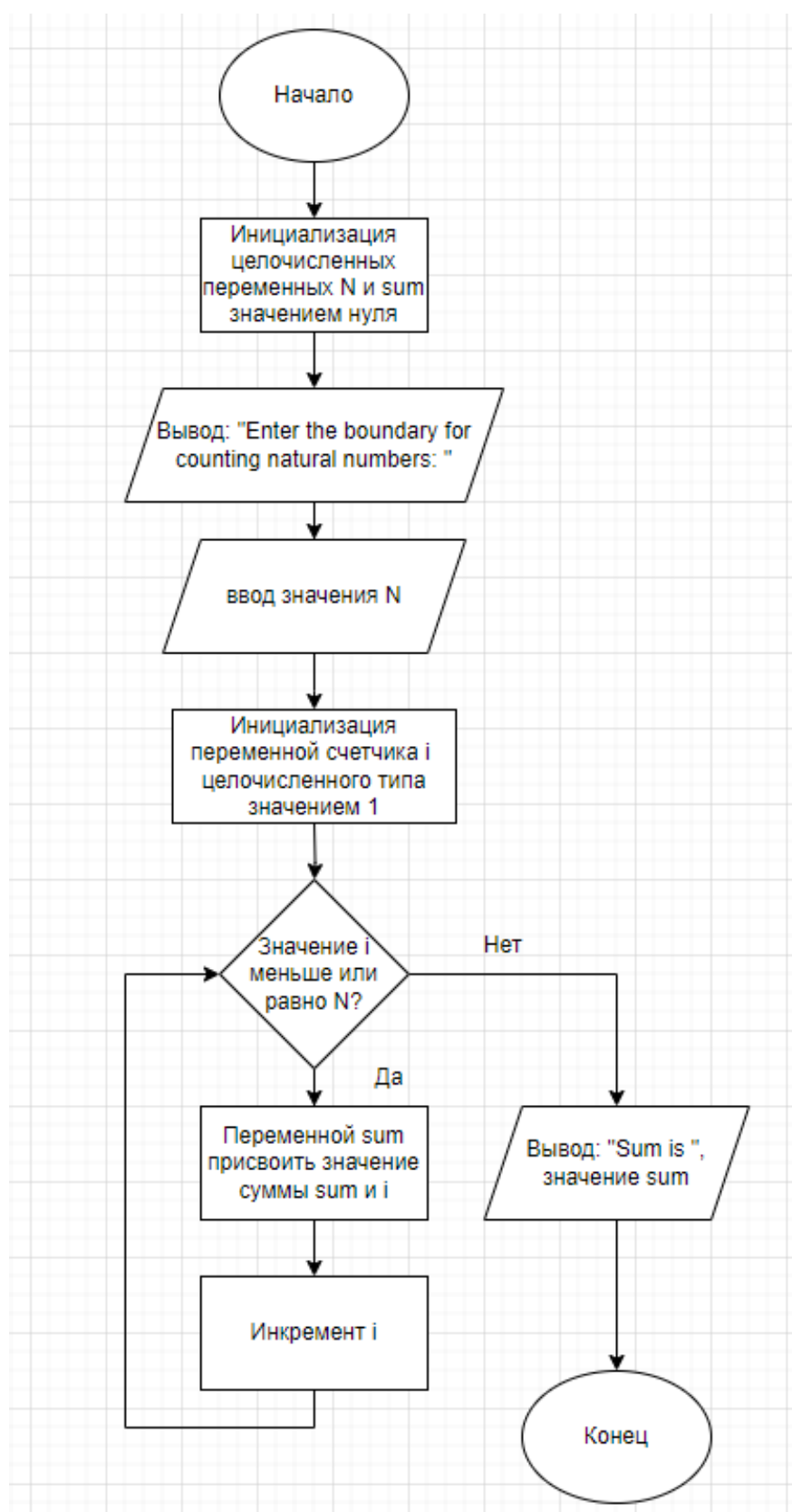


Рисунок 42. Блок-схема алгоритма решения задачи

Таблица 13. Листинг программы решения задачи с использованием цикла for

```
#include <iostream>
using namespace std;

int main()
{
    int N, sum = 0; // Инициализация целочисленных переменных N и sum значением
    нуля
    cout << "Enter the boundary for counting natural numbers: "; // вывод на экран сооб-
    щения об ожидании ввода от пользователя
    cin >> N; // ввод значения переменной N с клавиатуры
    for (int i = 1; i <= N; i++) // описание цикла for, в качестве счетчика инициализируем
    целочисленную переменную i значением единицы, цикл будет выполняться пока значения
    счетчика i меньше значения N, после прохода тела цикла увеличиваем счетчик на единицу

    {
        sum += i; // с помощью двойного оператора суммируем натуральные числа
    }
    cout << "Sum is: " << sum; // вывод на экран значения суммы натуральных чисел от 0
    до N
    return 0;
}
```

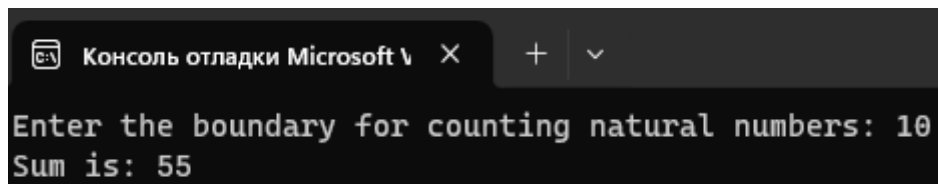


Рисунок 43. Тестирование программного кода со значением

Если посчитать сумму натуральных чисел от 1 до 10, то действительно получится значение 55.

Особенность цикла с предусловием while заключается в том, что перед очередной итерацией проверяется условие, и если оно истинно, то цикл продолжается.

Примером реализации цикла while может служить следующая задача: требуется вывести конечное значение итерации, на которой сумма всех натуральных чисел будет равна 21.

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 44 и 45. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 14.

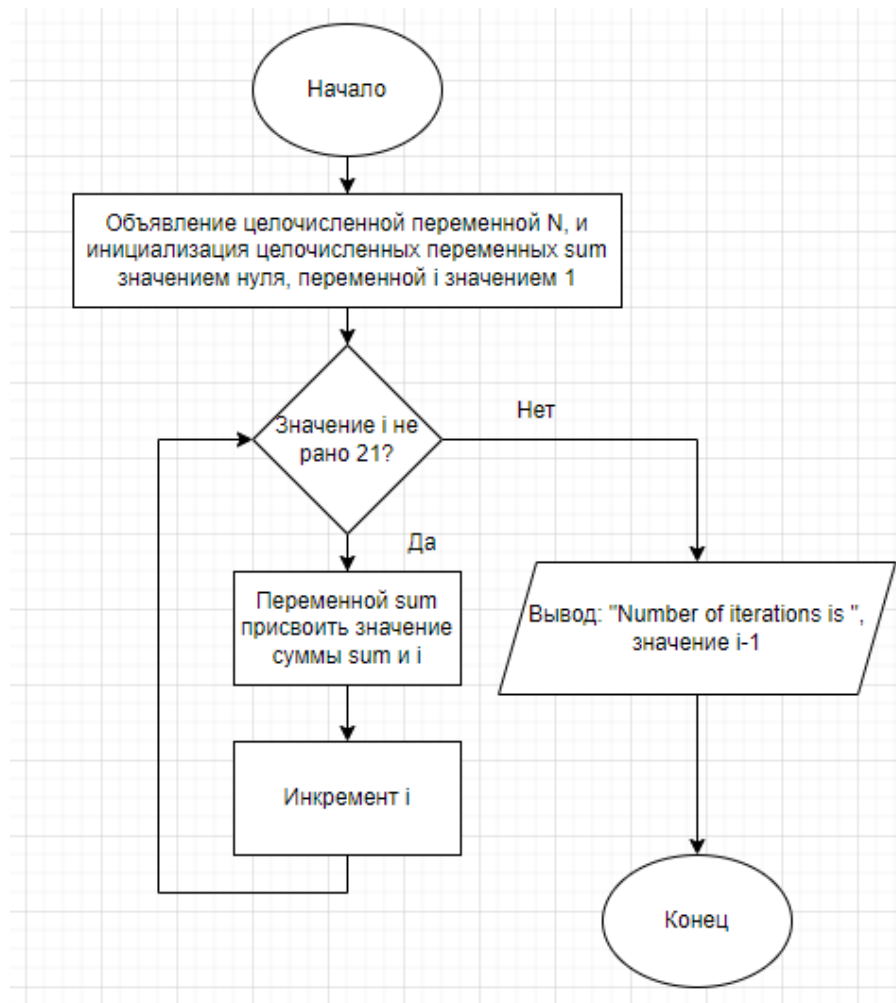


Рисунок 44. Блок-схема алгоритма решения задачи

Таблица 14. Листинг программы решения задачи с использованием цикла while

```

#include <iostream>
using namespace std;
int main()
{
    int N, sum = 0, i = 1; // объявление целочисленной переменной N, и инициализация
    переменных sum значением нуля, переменной i значением 1
    while (sum != 21) // объявляем цикл while, пока условие sum != 21 не станет ложью
    {
        sum += i;
        i++;
    }
    cout << "Number of iterations is: " << i - 1; // вывод на экран значения суммы нату-
    ральных чисел от 0 до N
    return 0;
}
  
```

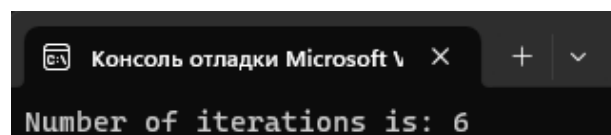


Рисунок 45. Тестирование программного кода со значением

Особенность цикла с постусловием `do while` заключается в том, что сначала производится итерация и в конце проверяется условие, если оно истинно, то цикл продолжается.

Примером реализации цикла `do while` может служить предыдущая задача: требуется вывести конечное значение итерации, на которой сумма всех натуральных чисел будет равна 21.

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 46 и 47. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 15.

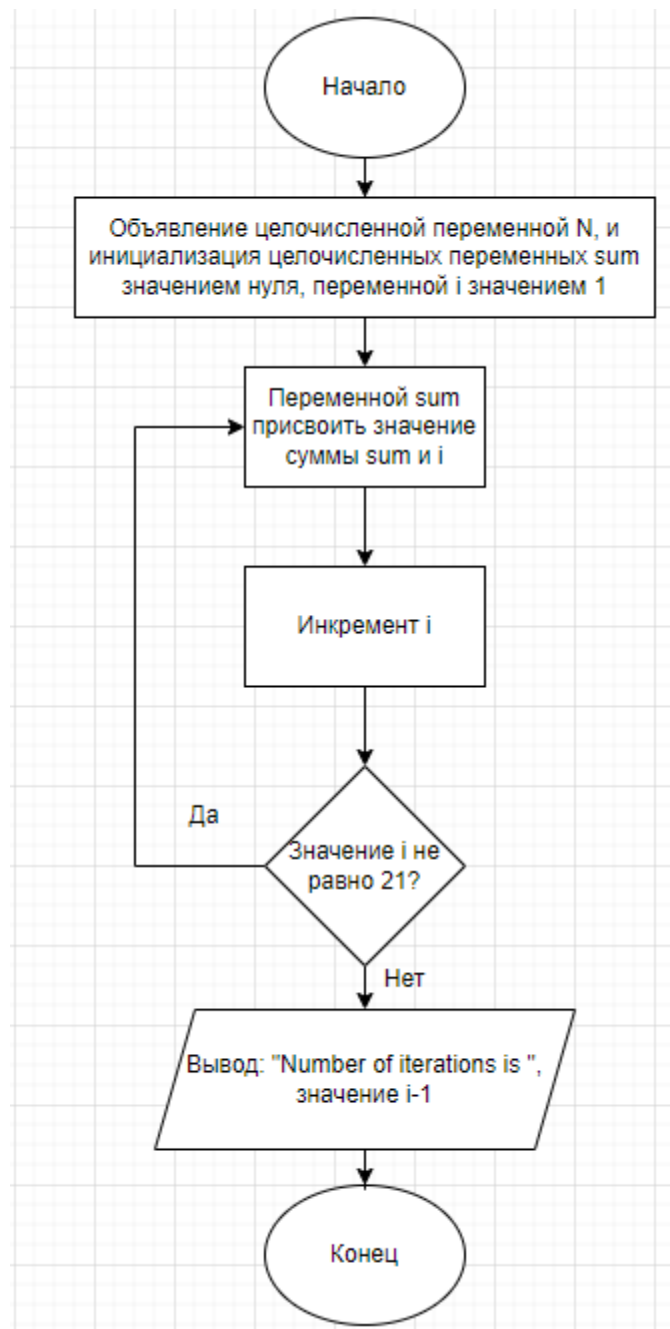


Рисунок 46. Блок-схема алгоритма решения задачи

Таблица 15. Листинг программы решения задачи с использованием цикла do while

```
#include <iostream>
using namespace std;

int main()
{
    int N, sum = 0, i = 1; // объявление целочисленной переменной N, и инициализация
    переменных sum значением нуля, переменной i значением 1
    do // ставим оператор 'do', который ознаменует начало цикла
    {
        sum += i;
        i++;

    } while (sum != 21); // после первого прохода цикла будет проверено заданное усло-
    вие, после чего цикл либо продолжится, либо оборвется
    cout << "Number of iterations is: " << i - 1; // вывод на экран значения суммы нату-
    ральных чисел от 0 до N
    return 0;
}
```

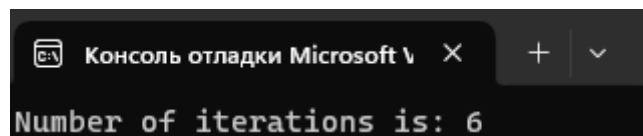


Рисунок 47. Тестирование программного кода со значением

7.7. Указатели и ссылки

Указатель — это самостоятельный тип данных, который может хранить адреса других переменных в памяти. Указатель должен быть разыменован с помощью оператора *, чтобы получить доступ к ячейке памяти, на которую он указывает.

Для указателя может происходить объявление и инициализация следующими способами, показанными на рисунках 48-49.

```
int a = 10;
int *p = &a;
```

Рисунок 48. Вариант инициализации указателя p

```
int *p;  
p = &a;
```

Рисунок 49. Вариант объявления указателя

Указатель может быть переназначен. Это свойство полезно для реализации структур данных, таких как связанный список, дерево и т.д. Пример переназначения указателя представлен на рисунке 50.

```
int a = 5;  
int b = 6;  
int *p;  
p = &a;  
p = &b;
```

Рисунок 50. Вариант переназначения указателя

В качестве примера работы указателя будет рассмотрена задача, где будет инициализирована переменная целочисленного типа некоторым значением, затем инициализирован указатель и будет выведено значение, на которое показывает указатель.

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 51 и 52. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 16.

Таблица 16. Листинг программы решения задачи с использованием указателя

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int a = 100; // инициализация целочисленной переменной 'a' значением 100  
    int* ptr; // объявление указателя 'ptr'  
    ptr = &a; // присвоение указателю 'ptr', адреса переменной 'a'  
    cout << "Pointer is pointing to: " << *ptr << "\n"; // вывод значения, на которое указывает  
    // указатель 'ptr' (100)  
}
```

```

int b = 255;
ptr = &b;
cout << "Pointer is pointing to: " << *ptr; // вывод значения, на которое указывает указатель 'ptr' (255)
delete ptr; // удаление указателя из памяти
return 0;
}

```

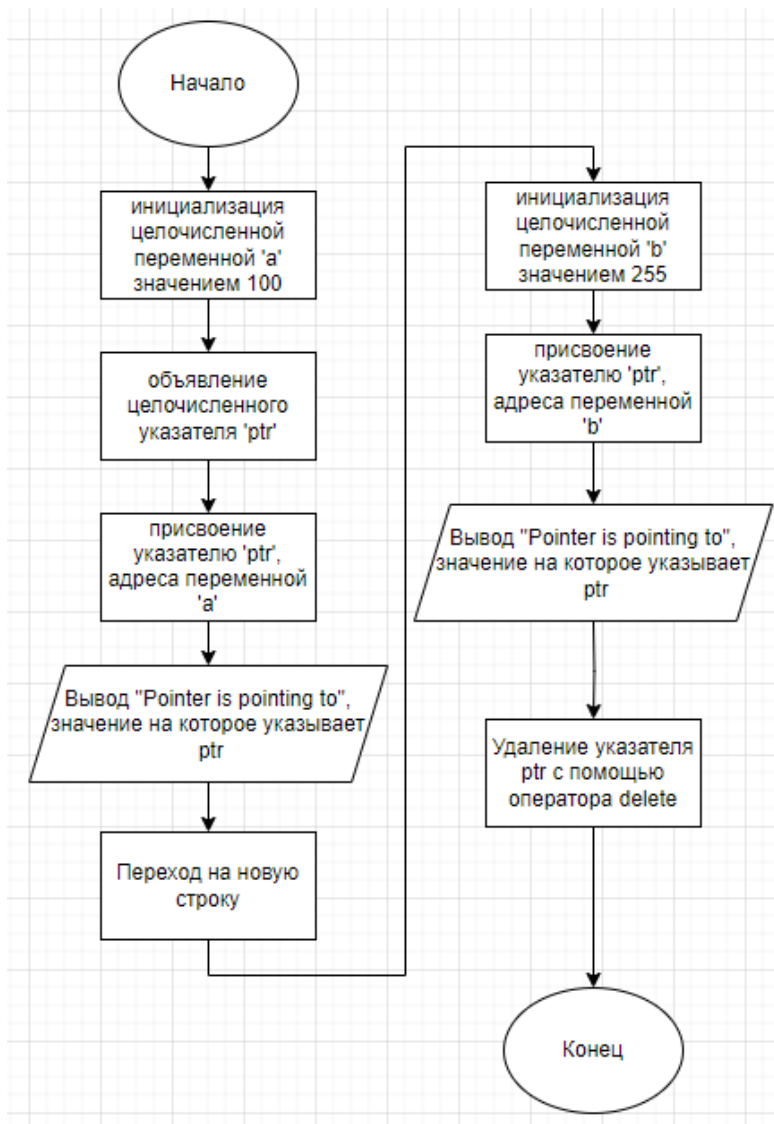


Рисунок 51. Блок-схема алгоритма решения задачи

```

Консоль отладки Microsoft V
Pointer is pointing to: 100
Pointer is pointing to: 255

```

Рисунок 52. Тестирование программного кода со значением

Отдельно следует сказать, что у указателя может быть несколько уровней косвенности. Может быть двойной указатель (указатель на указатель), как показано на рисунке 53.

```
int a = 10;
int *p;
int **q; // It is valid.
p = &a;
q = &p;
```

Рисунок 53. Тестирование программного кода со значением

В качестве примера работы будет рассмотрена следующая задача, где будет инициализирована переменная целочисленного типа некоторым значением, затем инициализирован указатель на эту переменную и двойной указатель на адрес первого указателя.

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 54 и 55. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 17.

Таблица 17. Листинг программы решения задачи с использованием двойного указателя

```
#include <iostream>
using namespace std;

int main()
{
    int a = 100; // инициализация целочисленной переменной 'a' значением 100
    int* ptr; // объявление указателя 'ptr'
    ptr = &a; // присвоение указателю 'ptr', адреса переменной 'a'
    cout << "Pointer is pointing to: " << *ptr << "\n"; // вывод значения, на которое указывает
    // указатель 'ptr' (100)
    cout << "The adress is: " << ptr << "\n"; // вывод значения адреса переменной 'a'
    int** ptr2 = &ptr; // инициализируем двойной указатель значением адреса первого указа-
    // теля
    cout << "Pointer is pointing to: " << **ptr2 << "\n"; // вывод значения, на которое указы-
    // вает указатель 'ptr' (100)
    cout << "The adress is: " << *ptr2 << "\n"; // вывод значения адреса переменной 'a'
    delete ptr, ptr2; // удаление указателей из памяти
    return 0;
}
```

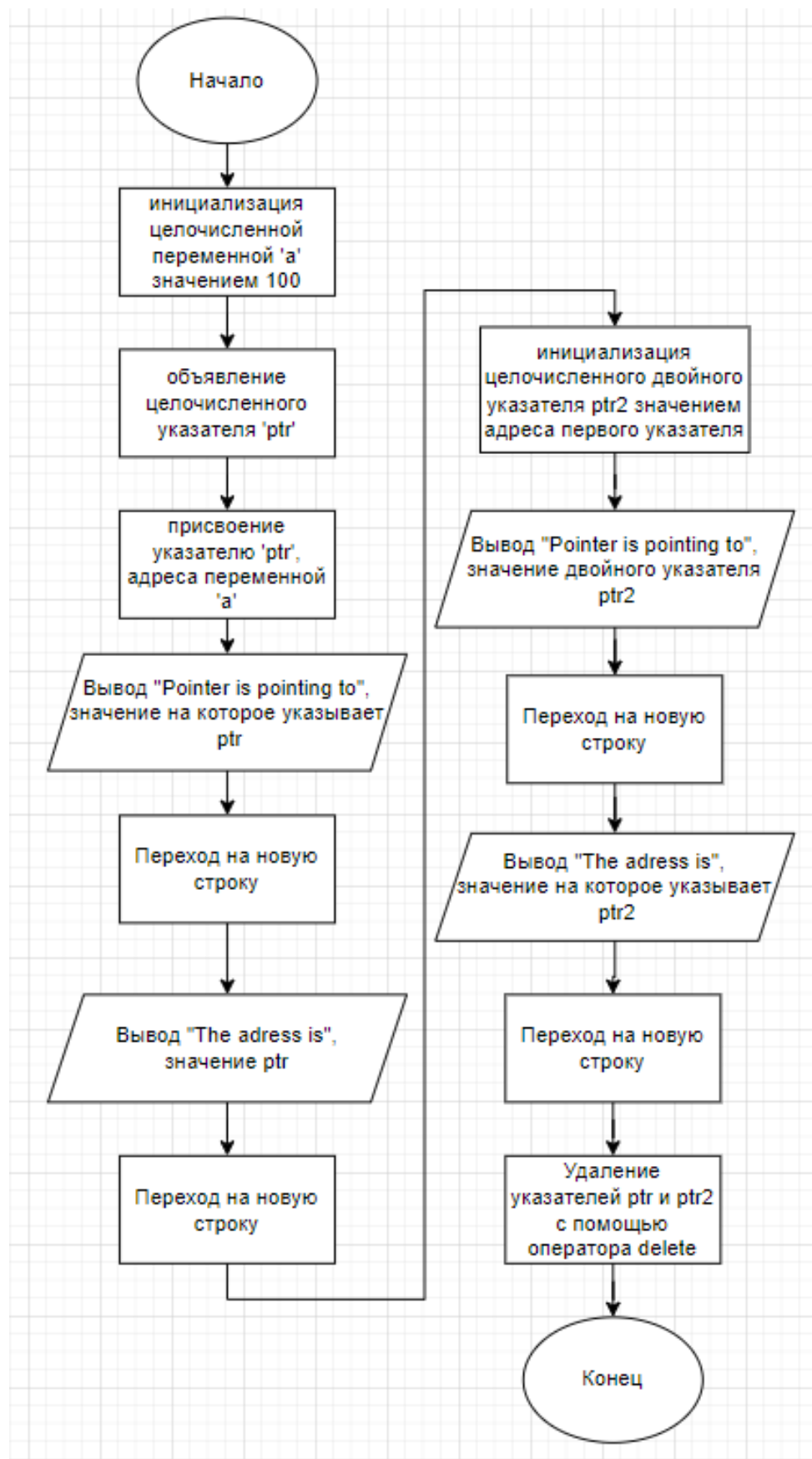


Рисунок 54. Блок-схема алгоритма решения задачи

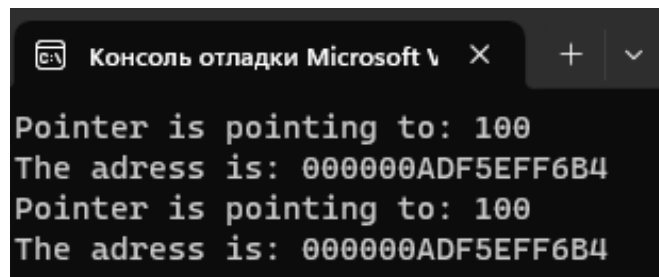


Рисунок 55. Тестирование программного кода со значением

Ссылочная переменная (ссылка) — это псевдоним, то есть другое имя для уже существующей переменной. Ссылка, как и указатель, также реализуется путем сохранения адреса объекта. В отличие от указателя, она может быть только инициализирована (рисунок 56).

```
int a = 10;
int &p = a;
```

Рисунок 56. Вариант инициализации ссылки

В качестве примера работы будет рассмотрена следующая задача, где будет инициализирована переменная целочисленного типа некоторым значением, затем инициализирован указатель на эту переменную и двойной указатель на адрес первого указателя.

Блок-схема алгоритма решения задачи и тестирование приведены на рисунках 57 и 58. Листинг программного кода на основе разработанной блок-схемы представлен в таблице 18.

Таблица 18. Листинг программы решения задачи с использованием ссылки

```
#include <iostream>
using namespace std;

int main()
{
    char symbol = 'V'; // инициализируем символьную переменную 'symbol' значением символа 'V'
    char& ref = symbol; // инициализируем ссылку 'ref', значением переменной 'symbol'
    cout << "The reference equals: " << ref << " And the original symbol is: " << symbol << "\n";
    // вывод на экран значений ссылки и основной переменной
    symbol = 'B'; // присвоение переменной нового значения
```

```

cout << "The reference equals: " << ref << " And the original symbol is: " << symbol << "\n";
// вновь вывод значений ссылки и основной переменной
ref = 'M'; // меняем значение самой ссылки, не меняя при этом значения основной пере-
менной
cout << "The reference equals: " << ref << " And the original symbol is: " << symbol << "\n";
// вновь вывод значений ссылки и основной переменной
return 0;
}

```

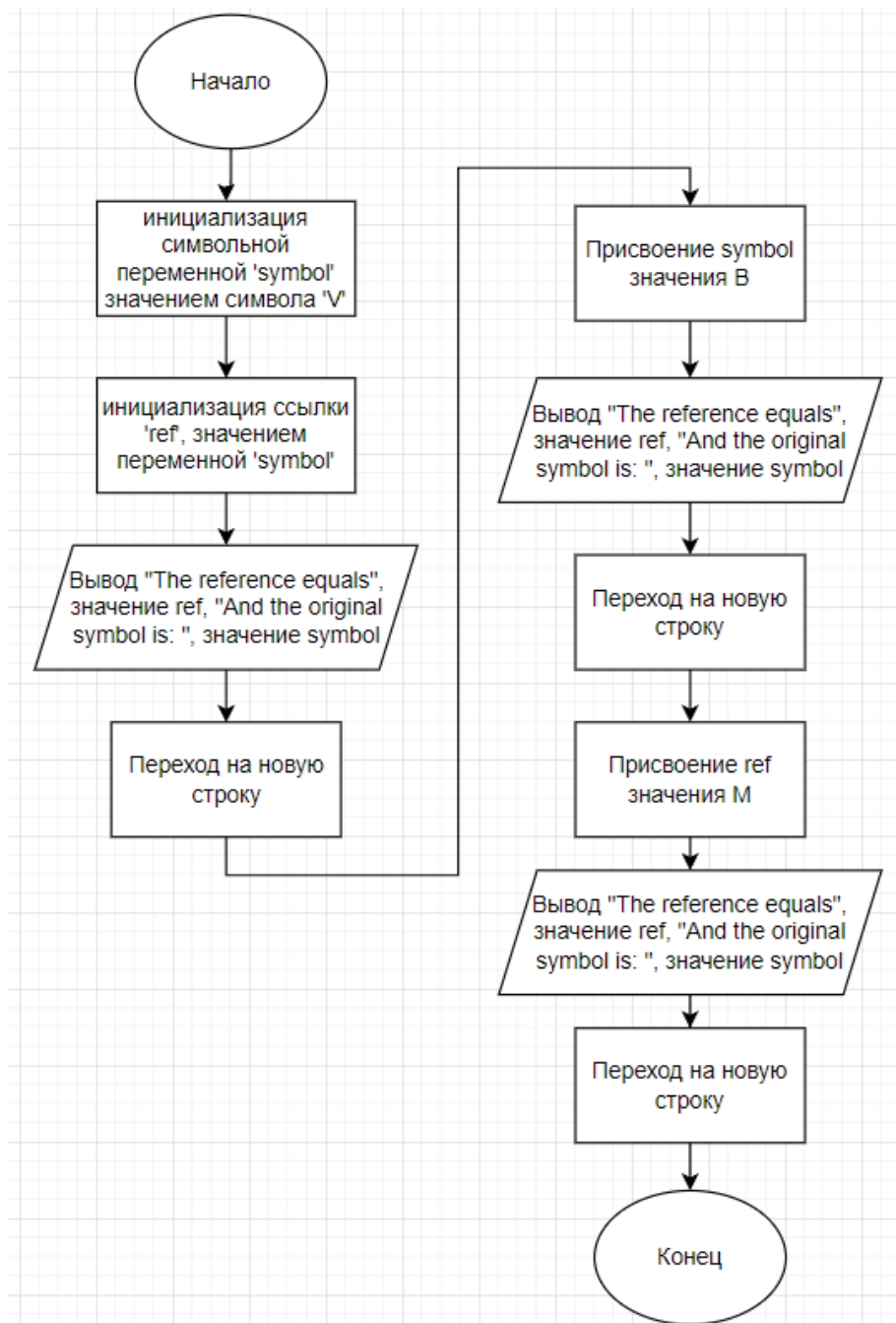


Рисунок 57. Блок-схема алгоритма решения задачи


```

Консоль отладки Microsoft V  X  +  v
The reference equals: V And the original symbol is: V
The reference equals: B And the original symbol is: B
The reference equals: M And the original symbol is: M

```

Рисунок 58. Тестирование программного кода со значением

В отличие от указателя ссылка не может быть переназначена и должна быть назначена при инициализации. Ссылка имеет один уровень косвенности в отличие от указателя. Другие различия между указателями и ссылкой представлены на рисунке 59.

	Ссылки	Указатели
Переназначение	Переменная не может быть переназначена в ссылке.	Переменная может быть переназначена в указателях.
Адрес памяти	У него тот же адрес, что и у исходной переменной.	Указатели имеют свой собственный адрес в памяти.
Работа	Это относится к другой переменной.	Он хранит адрес переменной.
Нулевое значение	Он не имеет нулевого значения.	Ему может быть присвоено значение null.
Аргументы	На эту переменную ссылается метод, передаваемый по значению.	Указатель выполняет свою работу с помощью метода, известного как передача по ссылке.

Рисунок 59. Различия между указателем и ссылкой

7.8. Массивы и его типы

Массив – это набор однотипных переменных, к которому можно обратиться по одному имени, но они имеют разные индексы.

Массивы в зависимости от размерности могут быть одномерные, двумерные и т.д.

Массивы, которые объявляются с указанием конкретных значений диапазонов своих индексов называются *статическими*.

Ниже приведен пример задачи реализации различных типов массивов: объявление одномерного массива, инициализация одномерного массива, способы заполнения массива, а также заполнение таблицы истинности демультиплексора, где входы А, В и С адресными (А – старший разряд, С – младший разряд), D – информационный вход, а элементы от 0 до 7 являются выходами.

Листинг кода приведен в таблице 19, а результаты тестирования на рисунке 60.

Таблица 19. Листинг программы решения задачи с использованием статических массивов

```
#include <iostream>
using namespace std;

int main()
{
    int Array_Example[5]; // пример объявления одномерного статического целочисленного массива
    int Array_Example2[5] = { 1,2,3,4,5 }; // пример инициализации одномерного статического целочисленного массива

    for (int i = 0; i < 5; i++) // еще один способ заполнения массива, это использование циклов, счетчик в данном случае не должен выходить за пределы размера массива
    {
        Array_Example[i] = 0; // присваиваем i-ому элементу массива значение нуль
    }

    cout << "First Array: ";

    for (int i = 0; i < 5; i++) // цикл вывода массива аналогичен вводу, только само действие изменено
    {
        cout << Array_Example[i]; // вывод i-ого элемента массива на экран
    }
    cout << "\n";

    cout << "Second Array: ";

    for (int i = 0; i < 5; i++) // вывод второго массива на экран
    {
        cout << Array_Example2[i]; // вывод i-ого элемента массива на экран
    }
    cout << "\n";

    cout << "DMUX table: \n";
```

```

char Array_Example3[9][12]; // объявляем двумерный целочисленный массив, в котором
отразим таблицу истинности трехразрядного демультимплексора

Array_Example3[0][0] = 'A'; // Для изменения значения конкретной ячейки, необходимо
указать две координаты, и присвоить нужное значение этой ячейке
Array_Example3[0][1] = 'B';
Array_Example3[0][2] = 'C';
Array_Example3[0][3] = 'D';
Array_Example3[0][4] = '0';
Array_Example3[0][5] = '1';
Array_Example3[0][6] = '2';
Array_Example3[0][7] = '3';
Array_Example3[0][8] = '4';
Array_Example3[0][9] = '5';
Array_Example3[0][10] = '6';
Array_Example3[0][11] = '7';

for (int i = 1; i < 9; i++) // более правильным способом изменения значений в двумерном
массиве будет использование циклов, данный цикл будет использован для заполнения
старшего разряда
{
    if (i < 5) Array_Example3[i][0] = '0'; // при условии, когда счетчик меньше '5', в '0' стол-
бец будет установлено значение нуль
    else Array_Example3[i][0] = '1'; // в случае, когда условие не выполняется будет значе-
ние единица
    Array_Example3[i][3] = '1'; // в '3' столбец установим значение 1 без условий
}

for (int i = 1; i < 9; i++) // Второй цикл будет использован для заполнения второго разряда
{
    if (i == 1 || i == 2 || i == 5 || i == 6) Array_Example3[i][1] = '0';
    else Array_Example3[i][1] = '1';
}

for (int i = 1; i < 9; i++) // третий цикл будет использован для заполнения младшего раз-
ряда
{
    if (i%2 == 1) Array_Example3[i][2] = '0'; // при условии, что номер строки нечетный бу-
дет заносить в ячейку значение нуль
    else Array_Example3[i][2] = '1'; // когда номер строки четный, будем ставить значение
'1'
}

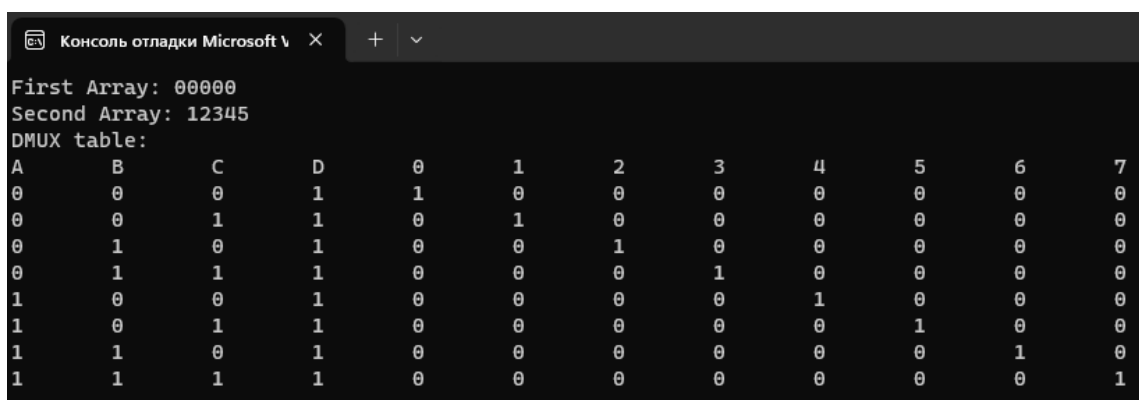
for (int i = 1; i < 9; i++) // данный цикл будет использован для заполнения выходных дан-
ных демультимплексора
{
    for (int j = 4; j < 12; j++)
    {
        if (i == j-3) Array_Example3[i][j] = '1';
        else Array_Example3[i][j] = '0';
    }
}

```

```

}
for (int i = 0; i < 9; i++) // в этом цикле организуем вывод двумерного массива в консоль
{
    for (int j = 0; j < 12; j++)
    {
        cout << Array_Example3[i][j] << "\t"; // оператор \t используется для установки знака
табуляции, для лучшей читаемости таблицы истинности
    }
    cout << "\n";
}
return 0;
}

```



```

First Array: 00000
Second Array: 12345
DMUX table:
A   B   C   D   0   1   2   3   4   5   6   7
0   0   0   1   1   0   0   0   0   0   0   0
0   0   1   1   0   1   0   0   0   0   0   0
0   1   0   1   0   0   1   0   0   0   0   0
0   1   1   1   0   0   0   1   0   0   0   0
1   0   0   1   0   0   0   0   1   0   0   0
1   0   1   1   0   0   0   0   0   1   0   0
1   1   0   1   0   0   0   0   0   0   1   0
1   1   1   1   0   0   0   0   0   0   0   1

```

Рисунок 60. Тестирование программного кода со значением

Динамическим называется такой массив, размер которого меняется во время исполнения программы. Для выделения памяти под динамический массив также используется оператор `new`, после которого в квадратных скобках указывается, сколько массив будет содержать объектов. Для удаления динамического массива и освобождения его памяти применяется специальная форма оператора `delete`. Пример кода, который показывает создание, изменение и удаление динамического массива приведен в таблице 20 и результат работы на рисунке 61.

Таблица 20. Листинг программы решения задачи с использованием динамических

массивов

```

#include <iostream>
using namespace std;

void bubbleSort(int Array[], int size) // объявляем функцию bubbleSort с параметрами массив
и его длина, которая будет сортировать массив в порядке возрастания
{
    while (size--)

```

```

    {
        bool swapped = false;
        for (int i = 0; i < size; i++)
        {
            if (Array[i] > Array[i + 1])
            {
                swap(Array[i], Array[i + 1]);
                swapped = true;
            }
        }

        if (swapped == false)
            break;
    }
}

int main()
{
    int size; // объявляем целочисленную переменную, которая будет использоваться для за-
    дания размера будущего массива
    cout << "Enter the size of the array value: ";
    cin >> size;
    int* Dynamic_Array = new int[size]; // объявляем динамический массив размера 'size'

    for (int i = 0; i < size; i++) // объявляем цикл for, для заполнения динамического массива
    случайными числами от 0 до 1000
    {
        Dynamic_Array[i] = 0 + rand() % 1000;
    }

    cout << "The Array before sorting : \n";

    for (int i = 0; i < size; i++) // организуем еще один цикл для вывода получившегося мас-
    сива на экран
    {
        cout << Dynamic_Array[i] << " ";
    }

    bubbleSort(Dynamic_Array, size); // вызовем функцию, которая отсортирует массив в
    порядке возрастания

    cout << "\nThe Array after sorting : \n";

    for (int i = 0; i < size; i++) // повторим цикл вывода еще раз для вывода уже отсорти-
    рованного массива
    {
        cout << Dynamic_Array[i] << " ";
    }
    delete[] Dynamic_Array;
    return 0;
}

```

```
Консоль отладки Microsoft V X + v
Enter the size of the array value: 15
The Array before sorting :
41 467 334 500 169 724 478 358 962 464 705 145 281 827 961
The Array after sorting :
41 145 169 281 334 358 464 467 478 500 705 724 827 961 962
```

Рисунок 61. Тестирование программного кода со значением

Также, с помощью динамического массива можно реализовать задачу с подсчетом суммы натуральных чисел, код реализации решения с использованием динамического массива приведен ниже в таблице 21, а результат на рисунке 62.

Таблица 21. Листинг программы решения задачи на подсчет суммы натуральных чисел с использованием динамических массивов

```
#include <iostream>
using namespace std;

int main()
{
    int N; // объявляем целочисленную переменную N, которая будет отвечать за размер массива, в котором мы будем хранить натуральные числа
    cout << "Enter the size of the Array: "; // вывод на экран сообщения о вводе данных в консоль
    cin >> N; // ввод значения переменной N
    int* Array = new int[N]; // объявление динамического массива целочисленных значений, размера 'N',
    for (int i = 0; i < N; i++) // объявляем цикл for, устанавливаем начальное значение счетчика равное нулю, цикл будет выполняться пока счетчик i '<' меньше значения 'N', увеличение счетчика идет с помощью операции инкремент '++'
    {
        Array[i] = i + 1; // в данном цикле мы заполняем массив натуральными числами, так как изначально он был пустой
    }
    int sum = 0; // инициализируем целочисленную переменную суммы значением 0
    for (int i = 0; i < N; i++) // объявляем цикл for, устанавливаем начальное значение счетчика равное нулю, цикл будет выполняться пока счетчик i '<' меньше значения 'N', увеличение счетчика идет с помощью операции инкремент '++'
    {
        sum += Array[i]; // с помощью оператора '+=', что эквивалентно 'sum = sum + Array[i]', производим подсчет суммы натуральных чисел от одного до 'N'
    }
    cout << "The sum is: " << sum; // вывод на экран значения получившейся суммы
    delete [] Array; // удаление динамического массива из памяти
    return 0;
}
```

```
Консоль отладки Microsoft V X + v
Enter the size of the Array: 10
The sum is: 55
```

Рисунок 62. Тестирование программного кода со значением

7.9. Строки и оператор sizeof

Строкой называют последовательность символов, заключенных в двойные кавычки.

Каждая из строк содержит на один символ больше, чем задано явно, так как они оканчиваются неявно нулевым символом (`'\0'`), имеющим значение 0. Типом строки считается “массив из соответствующего числа символов”, поэтому, например, строка “abcd” будет массивом типа `char` на 5 элементов (`char[5]`). Пустая строка записывается как “” и является массивом типа `char` на 1 элемент. Рисунок 63 иллюстрирует принцип записи строки с точки зрения массива.

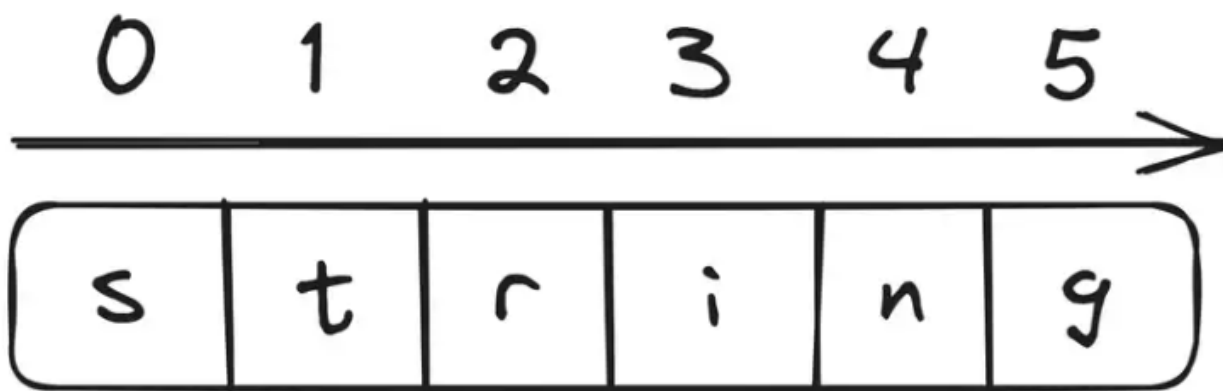


Рисунок 63. Строка как массив

При работе со строками в рамках процедурного программирования их можно конкатенировать (объединять) и получать их размер.

Конкатенация строк, как правило, предполагает использование оператора сложения. Объединять строки можно, как с использованием переменных и проведением действий над ними, так и с использованием строки в кавычках. Пример первого способа и результат объединения показан на рисунках 64 и 65, пример второго способа и результата на рисунках 66 и 67.

В C++ существуют специальные символы в строках. Все специальные символы в строках начинаются со слэша (`\`). Наиболее часто используемые специальные символы, следующие:

- `\n` - перенос строки;
- `\'` - одинарная кавычка;

- \" - двойная кавычка;
- \\ - обратный слеш.

```
string c = "C";
string pp = "++";
string cpp = c + pp;

cout << "cpp = " << cpp << endl;
```

Рисунок 64. Сегмент листинга кода для объединения строк с использованием переменных

```
cpp = C++
```

Рисунок 65. Результат объединения строк

```
string c = "C";
string cpp = c + "++";

cout << "cpp = " << cpp << endl;
```

Рисунок 66. Сегмент листинга кода для объединения строк с использованием сложения строки в кавычках

```
cpp = C++
```

Рисунок 67. Результат объединения строк

Для определения длины, как встроенных типов данных, так и типов данных, определенных программистом, используется оператор `sizeof`. Пример работы оператора `sizeof` с различными типами данных представлена в таблице 22, результат представлен на рисунке 68.

Таблица 22. Листинг программы с использованием оператора `sizeof`

```
#include <iostream>
using namespace std;

struct Example {}; // объявление структуры

int main()
{
    int a = 90; // инициализация целочисленной переменной 'a' значением '90'
```



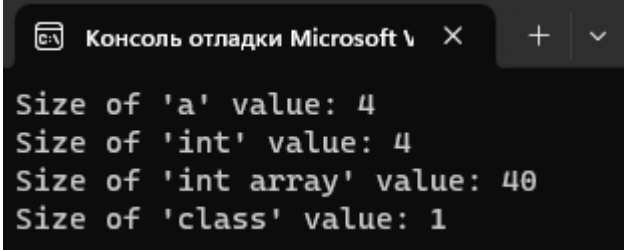
```

cout << "Size of 'a' value: " << sizeof(a) << "\n"; // вывод на экран количества памяти, зани-
маемой переменной 'a'
cout << "Size of 'int' value: " << sizeof(int) << "\n"; // вывод на экран количества памяти,
занимаемой типом данных 'int'

int array[10] = {'1','2','3','4','5','6','7','8','9','10'}; // инициализация целочисленного массива
'array' 10 элементами
cout << "Size of 'int array' value: " << sizeof(array) << "\n"; // вывод на экран количества
памяти, занимаемой массивом

Example test; // объявление структуры 'test'
cout << "Size of 'class' value: " << sizeof test << "\n"; // вывод на экран количества памяти,
занимаемой структурой 'test'
return 0;
}

```



```

Консоль отладки Microsoft v
Size of 'a' value: 4
Size of 'int' value: 4
Size of 'int array' value: 40
Size of 'class' value: 1

```

Рисунок 68. Тестирование программного кода со значением

Работает этот оператор на этапе компиляции программы. Оператор `sizeof` возвращает целочисленную константу без знака, которая равна длине операнда в байтах.

7.10. Функции, перегрузка функций

Причиной, из-за которой в свое время была создана концепция функций, стало стремление сократить размер программного кода. Любая последовательность операторов, встречающаяся в программе более одного раза, будучи вынесенной в отдельную функцию, сокращает размер программы.

Функция – это поименованная часть программы, которая может вызываться из других частей программы столько раз, сколько необходимо.

Несмотря на то, что функция в процессе выполнения программы исполняется не один раз, ее код хранится только в одной области памяти.

Если требуется, функция может определять входные параметры, позволяющие вызывающим объектам передавать ей аргументы. При необходимости функция также может возвращать значение как выходное. Функции полезны для сохранения основных операций в едином блоке, который может многократно использоваться.

Функции, как и переменные имеют различные типы. В зависимости от типа, функция должна вернуть то или иное значение или ничего не возвращать, если она типа `void`. Значения, передаваемые функции, являются аргументами, типы которых должны быть совместимы с типами параметров в определении функции.

На рисунке 69 показано, каким образом функция может вызываться из разных участков программы.

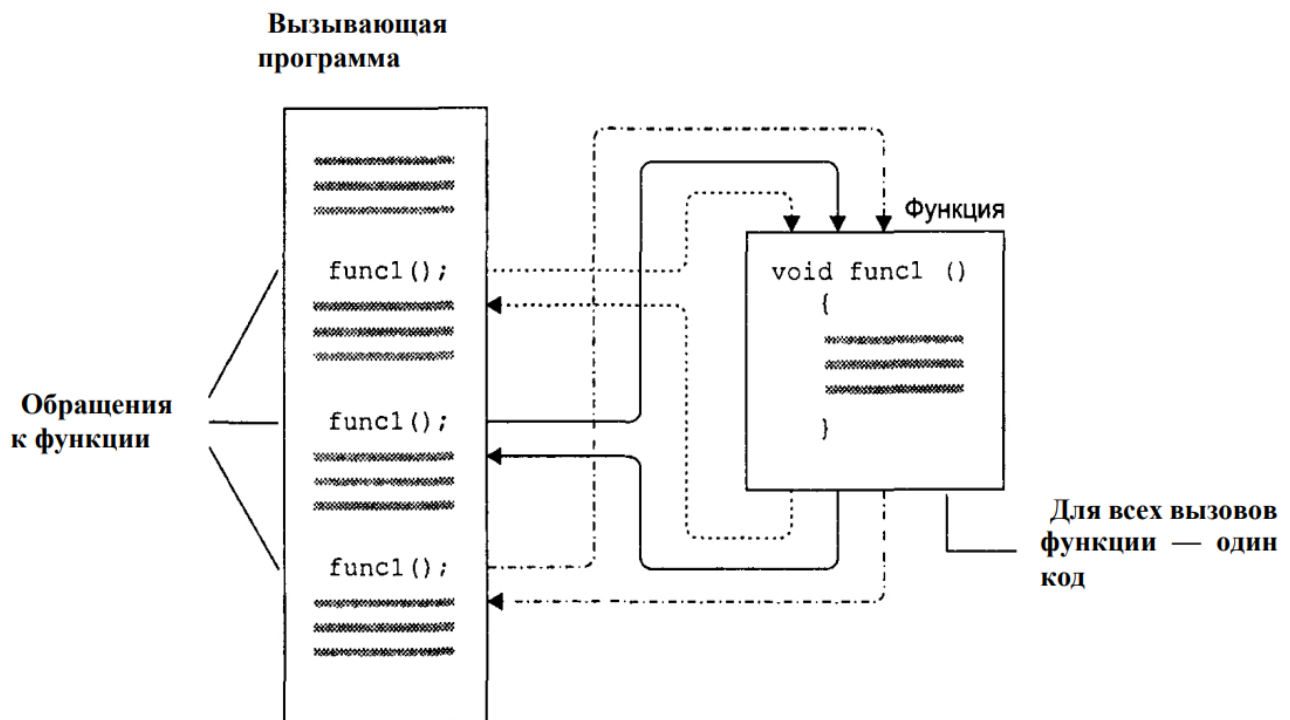


Рисунок 69. Принцип вызова функции

На рисунке 70 продемонстрирован синтаксис объявления, вызова и определения функции.

Когда происходит вызов функции, программа передает управление первому оператору тела функции. Затем исполняются операторы, находящиеся в теле функции, и когда достигается закрывающая фигурная скобка, управление передается обратно вызывающей программе.

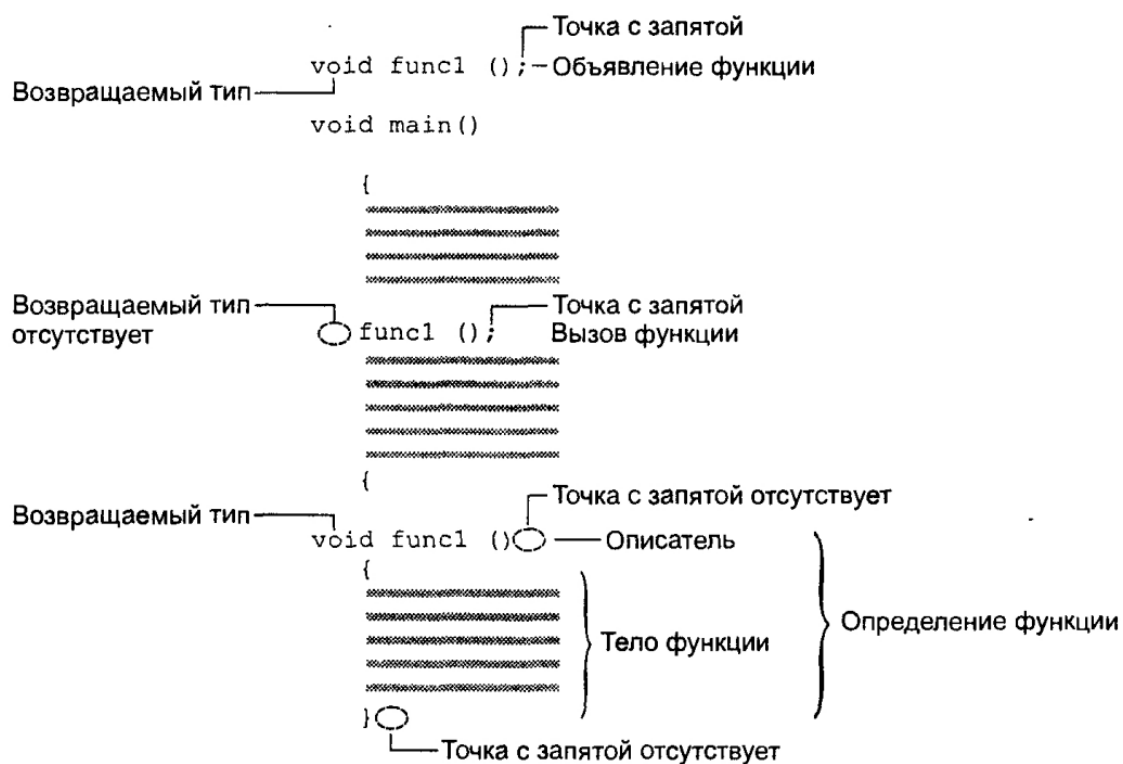


Рисунок 70. Синтаксис функции

Для рассмотрения синтаксиса функций, вновь обратимся к примеру, рассмотренному в начале. Блок-схемы алгоритма решения приведена на рисунках 71-74, результат тестирования на рисунке 75. Листинг кода представлен в таблице 23.

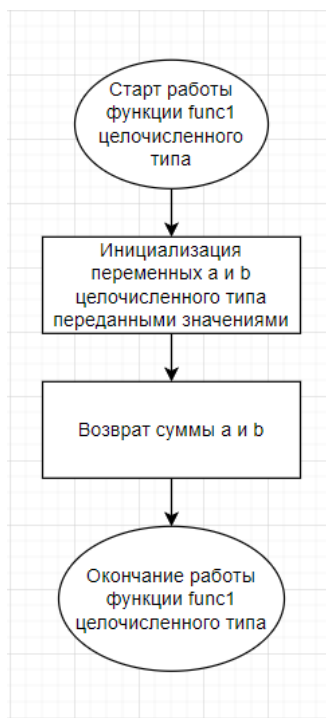


Рисунок 71. Блок-схема функции func1

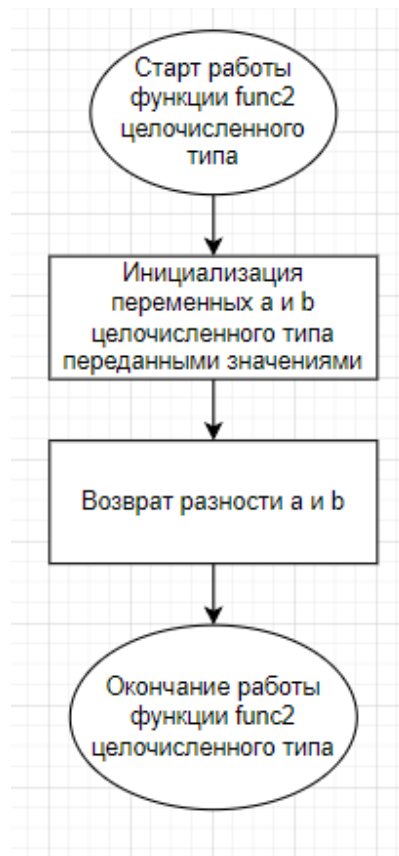


Рисунок 72. Блок-схема функции `func2`

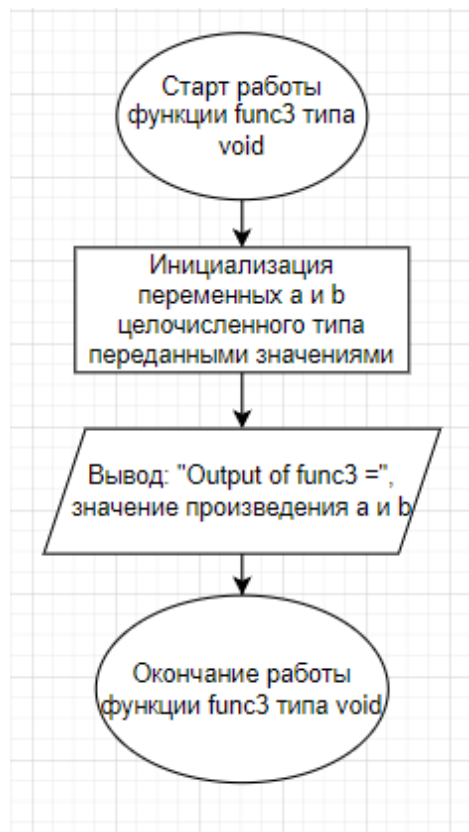


Рисунок 73. Блок-схема функции `func3`

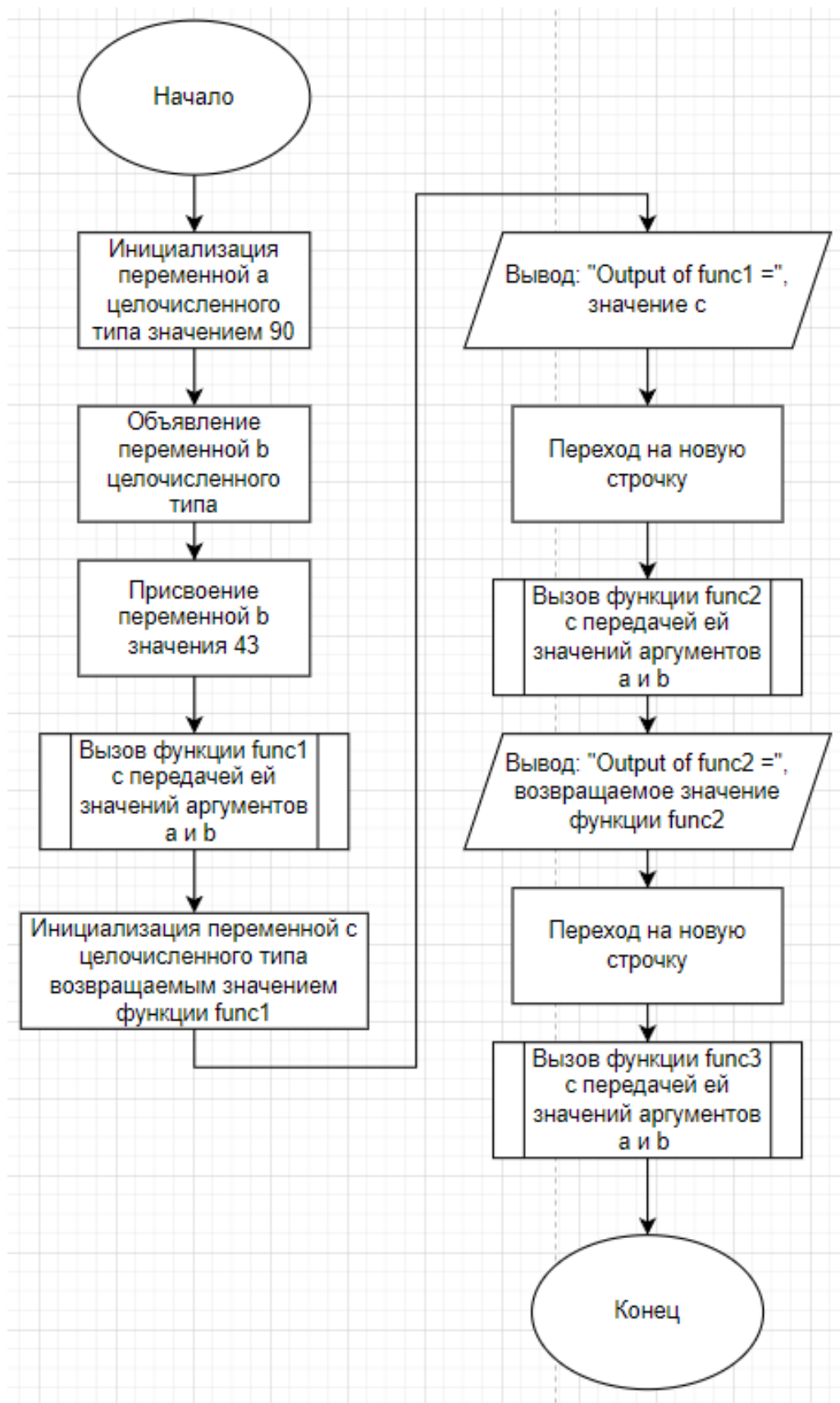


Рисунок 74. Блок-схема функции *main* задачи

Таблица 23. Листинг программы с использованием функций

```
#include <iostream>
using namespace std;

// объявляем несколько функций с разными именами
int func1(int a, int b)
{
    return a + b; // вернет значение суммы параметров 'a' и 'b'
}

int func2(int a, int b)
{
    return a - b; // вернет значение разности параметров 'a' и 'b'
}

void func3(int a, int b) // данная функция объявлена как void, то есть она ничего не возвра-
    щает, ключевое слово 'return', в таких функциях не требуется
{
    cout << "Output of func3 = " << a * b; // вывод в консоль значения произведения парамет-
    ров 'a' и 'b'
}

int main()
{
    int a = 90; // инициализация целочисленной переменной 'a' значением 90
    int b; // объявление целочисленной переменной 'b'
    b = 43; // присвоение переменной 'b' значения 43
    int c = func1(a, b); // инициализация целочисленной переменной 'c' значением, которое
    вернет функция 'func1' с аргументами 'a' и 'b'
    cout << "Output of func1 = " << c << "\n"; // вывод в консоль значения переменной 'c'
    cout << "Output of func2 = " << func2(a, b) << "\n"; // вывод в консоль значения которое
    вернет функция 'func2' с аргументами 'a' и 'b'
    func3(a, b); // вызов функции func3 с аргументами 'a' и 'b'
    return 0;
}
```

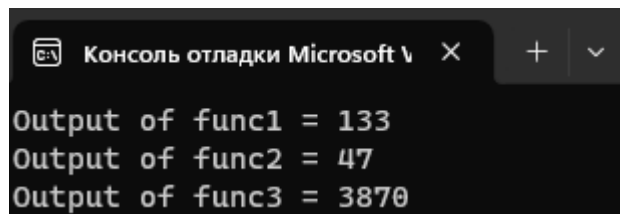


Рисунок 75. Тестирование программного кода со значением

Если в функции заранее задано значение аргумента, то оно называется **аргументом, передаваемым в функцию по умолчанию**. Такие аргументы, как правило, всегда стоят последними в очереди. Рассмотрим, как пример, ту же

реализацию, но с применением аргументов по умолчанию в таблице 24. Результат тестирования приведен на рисунке 76.

Таблица 24. Листинг программы с использованием функций и аргументов, передаваемых функции по умолчанию

```
#include <iostream>
using namespace std;

int func1(int a = 5, int b = 34) // при объявлении функции 'func1', укажем в параметрах значения для 'a' и 'b'
{
    return a + b;
}

int func2(int a, int b)
{
    return a - b;
}

void func3(int a, int b)
{
    cout << "Output of func3 = " << a * b;
}

int main()
{
    int a = 90;
    int b;
    b = 43;
    int c = func1(); // инициализация целочисленной переменной 'c' значением, которое вернет функция 'func1', аргументы при вызове, указывать не будем
    cout << "Output of func1 = " << c << "\n";
    cout << "Output of func2 = " << func2(a, b) << "\n";
    func3(a, b);
    return 0;
}
```

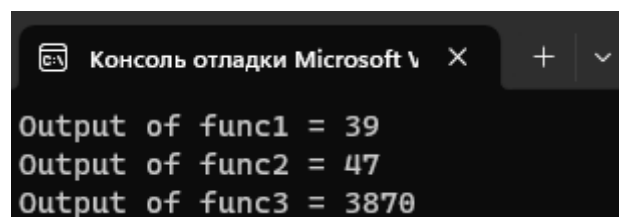


Рисунок 76. Тестирование программного кода со значением

Как можно заметить, программа не выдала ошибки при компиляции несмотря на то, что при вызове функции `func1`, не было указано никаких

аргументов. Результат работы программы также изменился, переменные `a` и `b`, в `main`-функции инициализированы значениями 90 и 43 соответственно, а результат сложения этих двух переменных явно меньше, что дополнительно свидетельствует о том, что для выполнения тела функции `func1`, были использованы значения по умолчанию, указанные при объявлении.

Как правило, разным функциям дают различные имена, но, когда функции выполняют концептуально аналогичные задачи для объектов различных типов, может оказаться удобным присвоить им одно и то же имя.

Перегрузка функций — это механизм C++, благодаря которому функции с разным количеством или типами параметров могут иметь одинаковое имя. При вызове функции, программа понимает к какой именно из многих нужно обратиться именно по передаваемым в нее аргументам. Добавим в предыдущую реализацию перегрузку функции `func2`. Листинг кода приведен в таблице 25 и результаты тестирования на рисунке 77.

Таблица 25. Листинг программы с использованием перегрузки функций

```
#include <iostream>
using namespace std;

int func1(int a = 5, int b = 34)
{
    return a + b;
}

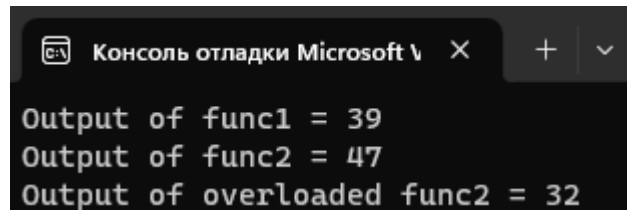
int func2(int a, int b) // объявляем функцию 'func2' с целочисленными параметрами 'a' и 'b'
{
    return a - b;
}

void func2(int a, int b, int c) // еще раз объявляем функцию 'func2' с целочисленными параметрами 'a', 'b' и 'c'
{
    cout << "Output of overloaded func2 = " << a * b / c;
}

int main()
{
    int a = 90;
    int b;
    b = 43;
    int c = func1();
    cout << "Output of func1 = " << c << "\n";
    cout << "Output of func2 = " << func2(a, b) << "\n"; // в первый раз, вызовем функцию с
    двумя аргументами 'a' и 'b'
```



```
func2(a, b, 120); // вызовем перегруженную функцию 'func2' с тремя аргументами 'a', 'b' и '120'
return 0;
}
```



```
Консоль отладки Microsoft V  X  +  v
Output of func1 = 39
Output of func2 = 47
Output of overloaded func2 = 32
```

Рисунок 77. Тестирование программного кода со значением

7.11. Структуры

Структурой называют совокупность элементов произвольных типов. Однако обычно структуры служат для хранения каких-то общедоступных данных в виде публичных переменных.

Для определения структуры применяется ключевое слово `struct`, а сам формат определения выглядит так, как показано на рисунке 78.

```
struct имя_структуры
{
    компоненты_структуры
};
```

Рисунок 78. Определение структуры

В качестве примера определения структуры, рассмотрим данный код. Здесь, объявляется некоторую структуру, которая будет содержать в себе несколько чисел. Далее создается экземпляр структуры 'Test1' и 'Test2', которым присваиваются разные значения, после чего будет попробовано присвоить одну структуру другой, и рассмотрен результат присвоения (Таблица 26). Тестирование приведено на рисунке 79.

Таблица 26. Листинг программы с использованием структуры

```
#include <iostream>
#include <string>
using namespace std;

struct Example // объявление структуры с названием 'Example'
```

```

{
    int param1; // объявление целочисленной переменной 'param1'
    char symbol; // объявление символьной переменной 'symbol'
};

int main()
{
    Example Test1; // объявление 'Test1' как структуры 'Example'
    Test1.param1 = 3; // присвоение переменной 'param1', находящейся в структуре, некото-
// рого значения
    Test1.symbol = 'G'; // присвоение переменной 'symbol', находящейся в структуре, неко-
// торого значения

    Example Test2; // объявление 'Test2' как структуры 'Example'
    Test2.param1 = 7;
    Test2.symbol = 'L';

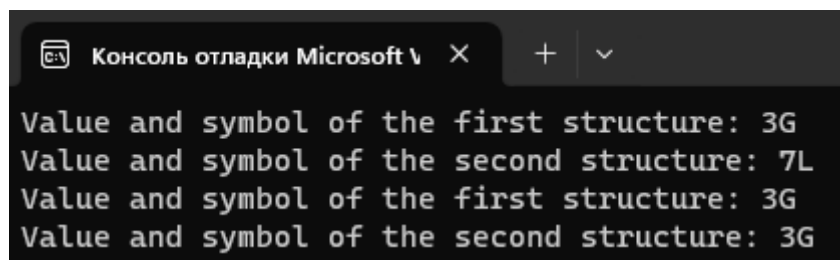
    cout << "Value and symbol of the first structure: " << Test1.param1 << Test1.symbol << "\n";
// вывод на экран значения переменной 'param1'
    cout << "Value and symbol of the second structure: " << Test2.param1 << Test2.symbol <<
    "\n"; // вывод на экран значения переменной 'symbol'

    Test2 = Test1; // структуры, равно как и переменные подвергаются присвоению

    cout << "Value and symbol of the first structure: " << Test1.param1 << Test1.symbol << "\n";
    cout << "Value and symbol of the second structure: " << Test2.param1 << Test2.symbol <<
    "\n";

    return 0;
}

```



```

Консоль отладки Microsoft V  X  +  v
Value and symbol of the first structure: 3G
Value and symbol of the second structure: 7L
Value and symbol of the first structure: 3G
Value and symbol of the second structure: 3G

```

Рисунок 79. Тестирование программного кода со значением

Более сложным вариантом создания структуры будет создание структуры с большим числом параметров и с использованием, так называемого, конструктора, который позволит создать структуру сразу с несколькими параметрами. Подробности работы конструкторов будут разобраны на странице 87 настоящего пособия. Пример создания структуры используя конструктор, описан ниже в таблице 27.

Таблица 27. Листинг программы с использованием в структуре конструктора

```
#include <iostream>
#include <string>
using namespace std;

struct Car // объявление структуры 'Car'
{
    string brand;
    string model;
    string engine;
    int productionYear;
    long price;
    Car(); // объявление конструктора без параметров
    Car(string, string, string, int, long); // объявление конструктора с параметрами
};

Car::Car(string br, string mod, string eng, int PY, long pr) // описание функционала конструктора с параметрами, объявленного ранее, в теле структуры 'Car'
{
    brand = br; // каждой переменной из структуры, присваиваем соответствующий аргумент из функции
    model = mod;
    engine = eng;
    productionYear = PY;
    price = pr;
}

int main()
{
    Car Car1; // объявление экземпляра 'Car1' структуры 'Car'
    Car1.brand = "Mercedes"; // обращение к параметру 'brand' структуры 'Car1' и присвоение ему значения 'Mercedes'
    Car1.model = "GLE Coupe 400d";
    Car1.engine = "330 H/P, 2.9L";
    Car1.productionYear = 2023;
    Car1.price = 14700000;

    Car Car2("BMW", "X6", "400 H/P, 3L", 2020, 8460000); // создание структуры 'Car2' через конструктор с желанными параметрами
    return 0;
}
```

В данном случае, мы используем не один, а два конструктора: первый, позволит просто создать пустую структуру, с параметрами, которые ничем не инициализированы (конструктор по умолчанию), второй, позволит при создании новой структуры, задать эти параметры сразу (параметризированный конструктор).

Для того, чтобы задать какие-нибудь значения параметрам из первой структуры, мы используем оператор ‘.’ (точка) через имя желанной структуры.

Однако это еще не все, объявить и заполнить структуру – это одно, а как вывести из нее какое-нибудь значение? Для этого добавим в наш код несколько вариантов вывода значений из структуры. Первый, это поочередное обращение к параметрам структуры и их вывод. Второй, более правильный подход, это использование собственной функции внутри самой структуры. Важно отметить, что функции внутри структур и классов, называются *методами*. Листинг кода приведен в таблице 28, а результат тестирования кода на рисунке 80.

Таблица 28. Листинг программы с использованием в структуре метода

```
#include <iostream>
#include <string>
using namespace std;

struct Car
{
    string brand;
    string model;
    string engine;
    int productionYear;
    long price;
    Car() {};
    Car(string, string, string, int, long);

    void Show(); // объявляем метод 'Show'
};

Car::Car(string br, string mod, string eng, int PY = 0, long pr = 0)
{
    brand = br;
    model = mod;
    engine = eng;
    productionYear = PY;
    price = pr;
}

void Car::Show() // описание функционала метода 'Show'
{
    cout << brand << " " << model << " " << engine << " " << productionYear << " " << price <<
    "\n"; // вывод на экран значений всех параметров структуры
}

int main()
{
    Car Car1;
    Car1.brand = "Mersedes";
```

```

Car1.model = "GLE Coupe 400d";
Car1.engine = "330H/P, 2.9L";
Car1.productionYear = 2023;
Car1.price = 14700000;

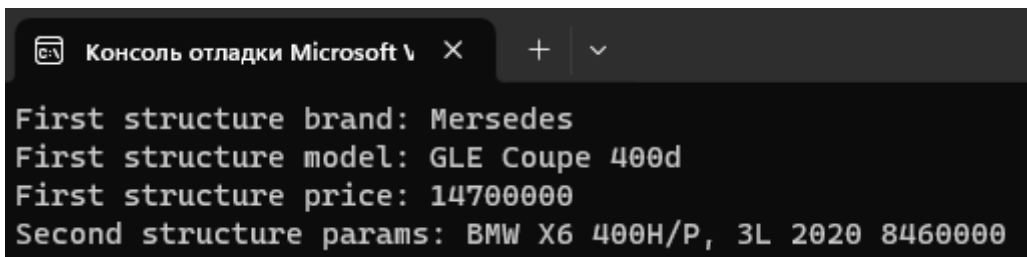
Car Car2("BMW", "X6", "400H/P, 3L", 2020, 8460000);

cout << "First structure brand: " << Car1.brand << "\n"; // вывод брэнда структуры
'Mercedes'
cout << "First structure model: " << Car1.model << "\n"; // вывод модели структуры
'Mercedes'
cout << "First structure price: " << Car1.price << "\n"; // вывод стоимости структуры
'Mercedes'

cout << "Second structure params: ";
Car2.Show(); // вызов метода 'Show' структуры BMW

return 0;
}

```



```

Консоль отладки Microsoft V
First structure brand: Mercedes
First structure model: GLE Coupe 400d
First structure price: 14700000
Second structure params: BMW X6 400H/P, 3L 2020 8460000

```

Рисунок 80. Тестирование программного кода со значением

7.12. Локальные и глобальные переменные

Область видимости переменных — это те части программы, в которой пользователь может изменять или использовать переменные в своих нуждах.

В C++ существуют отдельные блоки, которые начинаются с открывающей скобки ({) и заканчиваются соответственно закрывающей скобкой (}). Такими блоками являются циклы (for, while, do while) и функции.

Если переменная была создана в таком блоке, то ее областью видимости будет являться этот блок от его начала (от открывающей скобки - {) и до его конца (до закрывающей скобки - }) включая все дочерние блоки созданные в этом блоке.

Глобальными переменными называются те переменные, которые были созданы вне тела какого-то блока. Их можно всегда использовать во всей программе, вплоть до ее окончания работы.

Локальные переменные — это переменные, созданные в блоках. Областью видимости таких переменных является блоки (и все их дочерние), а также их область видимости не распространяется на другие блоки. Пример работы приведен в таблице 29.

Таблица 29. Листинг программы с использованием глобальных и локальных переменных

```
#include <iostream>
using namespace std;

const int globalConst = 935; // объявление глобальной константы

void func()
{
    cout << "Global Const = " << globalConst << "\n"; // вывод на экран значения глобальной
    константы с применением символьной константы '\n' для установки конца строки
    cout << "Local Const = " << localConst; // попытка вывода значения локальной константы
    из функции main
}

int main()
{
    globalConst = 9; // присвоение какого-либо значения данной переменной невозможно
    const float localConst = 9.0f; // объявление локальной константы

    return 0;
}
```

Если создать глобальную переменную и с таким же именем локальную, то получится, что там, где была создана локальная переменная будет использоваться именно локальная переменная, а не глобальная. Так как локальная переменная считается по приоритету выше глобальной. Пример работы приведен в таблице 30. Результат представлен на рисунке 81.

Таблица 30. Листинг программы с использованием одноименных глобальных и локальных переменных

```
#include <iostream>
using namespace std;

const int x = 100; // инициализация глобальной целочисленной константы 'x' со значением
100

int main()
```

```
{
    const int x = 1000; // инициализация локальной целочисленной константы 'x' со значением 1000
    cout << "x value is: " << x << "\n"; // вывод на экран значения локальной константы 'x'
    return 0;
}
```

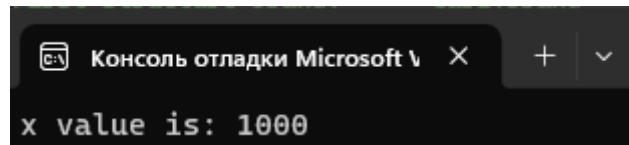


Рисунок 81. Тестирование программного кода со значением

В случае создания двух переменных с одинаковым именем (одна из которых является глобальной, а другая локальной) при использовании в блоке, в котором была объявлена локальная переменная, можно использовать и глобальную переменную. Для ее использования нужно всего лишь применить глобальный оператор разрешения.

Глобальный оператор разрешения — это два подряд поставленные двоеточия, с помощью которых можно объяснить компилятору, что требуется использовать глобальную переменную, а не локальную. Пример использования глобального оператора разрешения приведен в таблице 31, результат тестирования на рисунке 82.

Таблица 31. Листинг программы с использованием глобальных и локальных переменных и глобального оператора разрешения

```
#include <iostream>
using namespace std;

const int x = 100; // инициализация глобальной целочисленной константы 'x' со значением 100

int main()
{
    const int x = 1000; // инициализация локальной целочисленной константы 'x' со значением 1000
    cout << "x value is: " << x << "\n"; // вывод на экран значения локальной константы 'x'

    cout << "x value is: " << ::x; // вывод на экран значения глобальной константы 'x'
    return 0;
}
```

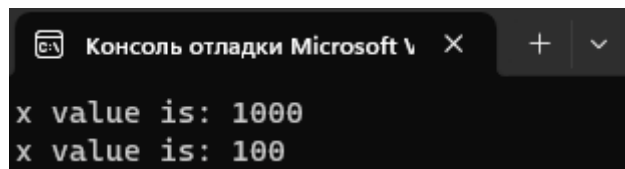


Рисунок 82. Тестирование программного кода со значением

8. ПАРАДИГМА ОБЪЕКТНО-ОРИЕНТИРОВАННОГО ПРОГРАММИРОВАНИЯ

8.1. Понятие парадигмы объектно-ориентированного программирования

В окончательном виде любая программа представляет собой набор инструкций процессора. Все, что написано на любом языке программирования, — более удобная, упрощенная запись этого набора инструкций, облегчающая написание, отладку и последующую модификацию программы. Чем выше уровень языка, тем в более простой форме записываются одни и те же действия

С ростом объема программы становится невозможным удерживать в памяти все детали, и становится необходимым структурировать информацию, выделять главное и отбрасывать несущественное. Этот процесс называется повышением степени абстракции программы.

Первым шагом к повышению абстракции является использование функций, позволяющее после написания и отладки функции отвлечься от деталей ее реализации, поскольку для вызова функции требуется знать только ее интерфейс. Если глобальные переменные не используются, интерфейс полностью определяется заголовком функции.

Следующий шаг — описание собственных типов данных, позволяющих структурировать и группировать информацию, представляя ее в более естественном виде. Например, можно представить с помощью одной структуры все разнообразные сведения, относящиеся к одному виду товара на складе.

Для работы с собственными типами данных требуются специальные функции. Естественно, сгруппировать их с описанием этих типов данных в одном месте программы, а также по возможности отделить от ее остальных частей. При этом для использования этих типов и функций не требуется полного знания того, как именно они написаны — необходимы только описания интерфейсов. Объединение в модули описаний типов данных и функций, предназначенных для работы с ними, со скрытием от пользователя модуля несущественных деталей, является дальнейшим развитием структуризации программы.

Все три описанных выше метода повышения абстракции преследуют цель упростить структуру программы, то есть представить ее в виде меньшего количества более крупных блоков и минимизировать связи между ними. Это позволяет управлять большим объемом информации и, следовательно, успешно отлаживать более сложные программы.

Как пример, представим, что есть три геометрические фигуры, как круг, квадрат и треугольник. Каждую из фигур можно, например, нарисовать или каждой можно задать цвет. Однако окружность по определению в рамках программирования — это такая фигура, которая имеет радиус. Изобразить ее можно с помощью функции, которая будет рисовать дугу и т.д., что явно не подходит для треугольника или квадрата.

Суть объектно-ориентированного программирования в том, что оно позволяет выражать эти различия и использует их. Здесь основную роль играет механизм наследования, который подразумевает, что есть суперкласс или родительский класс (в вышеописанном случае геометрических фигур) и есть подкласс или дочерний класс (в вышеописанном случае самой фигуры).

Так, *парадигма объектно-ориентированного программирования* звучит следующим образом: *определите, какой класс вам необходим; предоставьте полный набор операций для каждого класса; общность классов выразите явно с помощью наследования.*

8.2. Основные определения объекта и столпы объектно-ориентированного программирования

Объектом в природе принято называть то, что может быть индивидуально описано и распознано. Каждый объект имеет свои различные свойства.

Определение *объекта в программе* — экземпляр класса или его копия, где под *классом* подразумевается абстрактный тип данных, определяемый пользователем. Основными свойствами класса являются его *поля* (переменные-член класса) и его *методы* (функции-член класса).

Основное различие между классом и структурой в ООП заключается в том, что структуры, как правило, используют в качестве объединения данных, а классы — в качестве объединения данных и функций.

Множество объектов, находящихся в отношениях и связях друг с другом, которое образует определённую целостность, единство называется *системой*.

Основными столпами объектно-ориентированного программирования являются наследование, инкапсуляция и полиморфизм.

Наследование — это механизм, позволяющий строить иерархию типов. Механизм наследования предполагает определение базового типа (или родительского), а затем использование этого типа для построения производных типов (или потомков). Причем каждый производный тип наследует все свойства базового типа, включая характеристики (данные) и набор операций (функции).

Наследование призвано отобразить такое свойство реального мира, как иерархичность.

Например, из базового типа животные могут быть порождены типы: «млекопитающие», «рыбы», «птицы». Все эти типы будут содержать характеристики типа «животные» и иметь свои, отличные от других, характеристики: у типа «млекопитающие» – ноги, у типа «рыбы» – плавники, у типа «птицы» – крылья.

Инкапсуляция — этот принцип гласит, что вся важная информация, необходимая для работы объекта, в нем же и хранится. И только определенные данные доступны для внешних функций и объектов.

Инкапсуляция повышает степень абстракции программы: данные класса и реализация его функций находятся ниже уровня абстракции, и для написания программы информация о них не требуется. Кроме того, инкапсуляция позволяет изменить реализацию класса без модификации основной части программы, если интерфейс остался прежним (например, при необходимости сменить способ хранения данных с массива на стек).

Другими словами, инкапсуляция (сокрытие информации) – определение пользователем новых типов данных. Каждый такой тип содержит определение набора значений и операций, которые могут быть выполнены над этими значениями, и образует так называемый абстрактный тип данных. При этом никакие другие функции, кроме набора операций, не должны иметь доступа к значениям типа.

Например, новый тип – «животные» может содержать значения, характеризующие вес, размер, цвет, а также действия, такие как передвижение, размножение, насыщение.

Полиморфизм — возможность использовать в различных классах иерархии одно имя для обозначения сходных по смыслу действий и гибко выбирать требуемое действие во время выполнения программы.

Сущность полиморфизма, что в переводе с греческого означает много форм, заключается в обозначении общего действия одним именем (функцией), которое используется во всей иерархии типов. Причем каждый тип в этой иерархии реализует это действие своим собственным, пригодным для него способом.

Например, все типы в иерархии «животные» содержат операцию «передвижение». Но каждый из производных типов выполняет это действие своим способом:

- млекопитающие бегают;
- рыбы плавают;
- птицы летают.

Основные свойства ООП хорошо поясняются рисунком 83

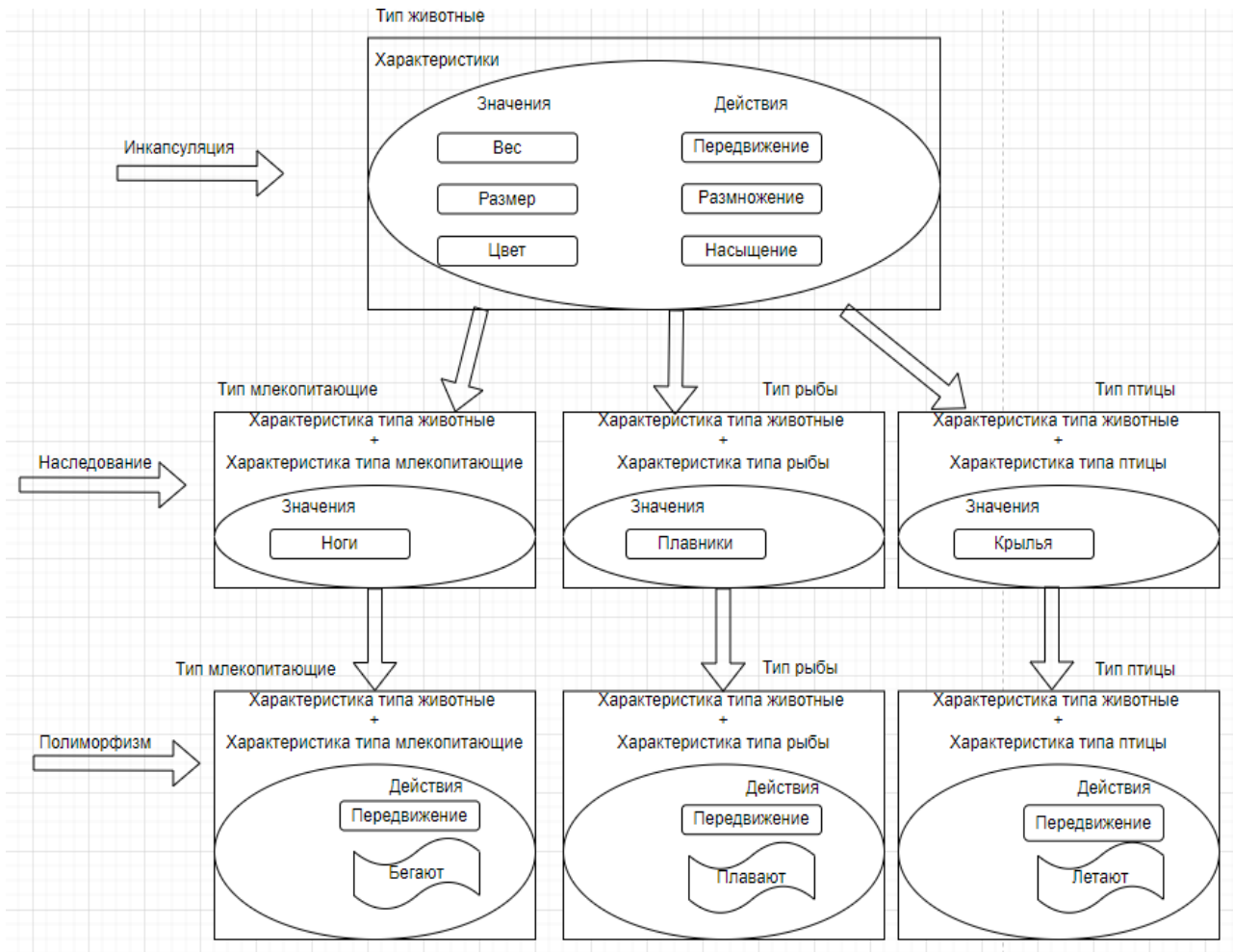


Рисунок 83. Основные свойства ООП

8.3. Время жизни объектов, взаимодействие объектов

Созданные человеком объекты имеют определенный жизненный цикл, который можно выразить некоторой схемой, представленной на рисунке 84

Жизненный цикл объекта

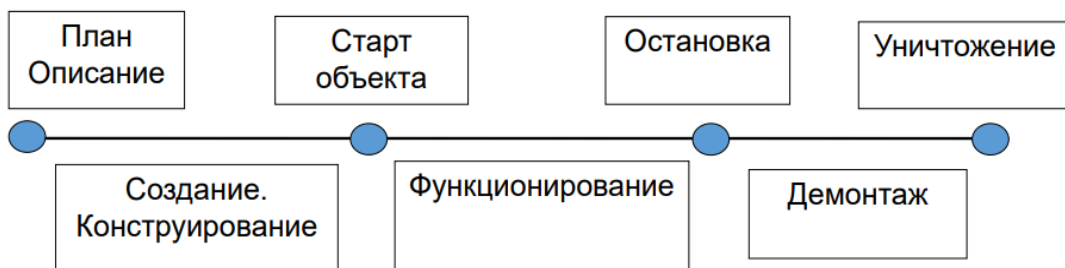


Рисунок 84. Жизненный цикл объекта

Примеры: дом, ПК, автомобиль и т.д.

Совершенствование объекта отображается в реализации новых, более совершенных, качественных, функционально развитых экземпляров.

Разработка любой системы, включает, в том числе выполнение следующих работ:

1. Определение цели, множества задач, для решения которых предназначена система.
2. Описание, создание необходимых объектов, составных частей (элементов) системы.
3. Конструктивная сборка системы.
4. Определения правил взаимодействия составных объектов системы. Построение соответствующих интерфейсов.
5. Запуск системы для функционирования, решения задач (согласно назначению).

В языке C++ реальному объекту из некоторой предметной области ставится в соответствии *виртуальный объект*. Язык обеспечивает жизненный цикл виртуального объекта по аналогии жизненного цикла объекта из предметной области. Схемы этих жизненных циклов совпадают.

Объекты могут взаимодействовать между собой. Объект при взаимодействии с другим объектом, отвечает обратной реакцией, выполняет определенные действия, в соответствии со своим функциональным назначением. Схематически это выглядит следующим образом (рисунок 85)

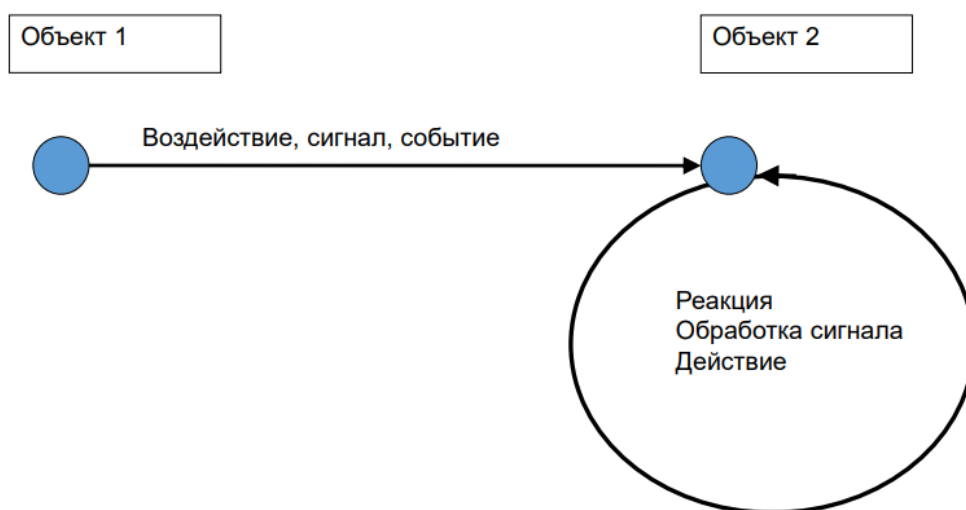


Рисунок 85. Схема взаимодействия объектов

8.4. Конструктор и деструктор

Как уже выше было сказано, основными свойствами класса являются его поля и методы. В соответствии с жизненным циклом в начале объект всегда создается и в конце всегда уничтожается. Для создания объекта используется особый метод под названием конструктор. Для уничтожения объекта соответственно деструктор.

Конструктор — это особый вид метода, который является членом класса и вызывается в момент создания объекта. Особенность данного метода заключается в том, что его имя совпадает с именем класса.

Как правило, конструктор используется для выделения памяти под объект класса и начальной инициализации внутренних данных класса.

Основные свойства конструктора:

- Конструктор не возвращает значение, даже типа void.
- Нельзя получить указатель на конструктор.
- Конструкторы не наследуются.
- Конструкторы нельзя описывать с модификаторами const, static, virtual.
- Класс может иметь несколько конструкторов с разными параметрами для разных видов инициализации.

Основные типы конструкторов: по умолчанию, с параметром, копирования, преобразования.

Конструктор по умолчанию – конструктор, который вызывается без параметров. Если класс не содержит явным образом определенный конструктор, тогда при создании объекта автоматически вызывается конструктор по умолчанию. Конструктор по умолчанию просто выделяет память для объекта класса, если он объявляется. Пример описания конструктора по умолчанию, в заголовочном файле некоторого класса Example приведен ниже в таблице 32:

Таблица 32. Листинг программы с использованием конструктора по умолчанию

```
#ifndef EXAMPLE_H
#define EXAMPLE_H
class Example // объявление класса Example
{
public:
    Example(); // объявление конструктора по-умолчанию класса Example
};

#endif
```

Конструктор с параметром (параметризованный конструктор) — конструктор, в который передаются параметры (аргументы). Пример описания параметризованного конструктора с двумя параметрами, в заголовочном файле некоторого класса Example приведен ниже в таблице 33:

Таблица 33. Листинг программы с использованием конструктора с параметром

```
#ifndef EXAMPLE_H
#define EXAMPLE_H
class Example
{
    int value1;
    char symbol;
public:
    Example(); // объявление конструктора по-умолчанию класса Example
    Example(int value1, char symbol); // объявление параметризованного конструктора
    класса Example
};
#endif
```

Конструктор копирования — это специальный вид конструктора, получающий в качестве единственного параметра указатель на объект этого же класса. Этот конструктор вызывается в тех случаях, когда новый объект создается путем копирования, существующего:

- при описании нового объекта с инициализацией другим объектом;
- при передаче объекта в функцию по значению;
- при возврате объекта из функции;

Пример кода, реализующего конструктор копии показан ниже в таблице 34.

Таблица 34. Листинг программы с использованием конструктора копирования

```
#ifndef EXAMPLE_H
#define EXAMPLE_H
class Example
{
    int value1;
    char symbol;
public:
    Example(); // объявление конструктора по-умолчанию класса Example
    Example(int value1, char symbol); // объявление параметризованного конструктора
    класса Example
    Example(const Example &obj); // объявление конструктора копирования класса
    Example
};
#endif
```

Любой конструктор класса, принимающий один параметр какого-либо другого типа, называется **конструктором преобразования**, поскольку он осуществляет преобразование из типа параметра в тип этого класса. В языке программирования C++ такие конструкторы имеют ключевое слово "explicit" и могут быть использованы для создания объектов с помощью неявного преобразования.

Пример кода, реализующего конструктор преобразования показан ниже в таблице 35.

Таблица 35. Листинг программы с использованием конструктора преобразования

```
#ifndef EXAMPLE_H
#define EXAMPLE_H

class Example
{
    int value1;
    char symbol;
public:
    Example(); // объявление конструктора по-умолчанию класса Example
    Example(int value1, char symbol); // объявление параметризованного конструктора
    класса Example
    Example(const Example &obj); // объявление конструктора копирования класса
    Example
    explicit Example(int); // объявление конструктора преобразования
};

#endif
```

Деструктор — это особый вид метода, применяющийся для освобождения памяти, занимаемой объектом.

Основные свойства деструктора:

- Деструктор не возвращает значение, даже типа void и не имеет аргументов.
- Может быть виртуальным.
- Конструкторы не наследуются.
- Деструкторы нельзя описывать с модификаторами const и static.

Пример кода, реализующего деструктор показан ниже в таблице 36.

Таблица 36. Листинг программы с использованием деструктора

```
#ifndef EXAMPLE_H
#define EXAMPLE_H
```



```

class Example
{
    int value1;
    char symbol;
public:
    Example(); // объявление конструктора по-умолчанию класса Example
    Example(int value1, char symbol); // объявление параметризованного конструктора
    класса Example
    Example(const Example &obj); // объявление конструктора копирования класса
    Example
    explicit Example(int); // объявление конструктора преобразования
    ~Example(); // объявление деструктора класса Example
};
#endif

```

8.5. Описание объектов, модификаторы доступа, константный объект

Класс предоставляет чертежи для объектов, поэтому объект создается из класса. Для доступа к элементам объекта используется оператор точка "." при обращении через имя объекта и оператор стрелка "->" при обращении через указатель.

Для демонстрации работы операторов точка и стрелка, будет добавлен еще один параметр value2 в публичный доступ в класс Example. Листинг кода представлен в таблице 37.

Таблица 37. Листинг программы заголовочного файла для демонстрации работы операторов точка и стрелка

```

#ifndef EXAMPLE_H
#define EXAMPLE_H
class Example
{
    int value1; // объявление приватного целочисленного свойства
    char symbol; // объявление приватного символьного свойства
public:
    int value2; // объявление публичного целочисленного свойства
    Example(); // объявление конструктора по умолчанию класса Example
    Example(int value1, char symbol); // объявление параметризованного конструктора класса Example
    Example(const Example &obj); // объявление конструктора копирования класса
    Example
    explicit Example(int); // объявление конструктора преобразования
    ~Example(); // объявление деструктора класса Example
};
#endif

```

Для того чтобы работать с созданным классом в main-функции, необходимо подключить заголовочный файл в основной файл, после чего в main-функции можно создать объект функции и использовать его методы и поля. Пример кода представлен в таблице 38:

Таблица 38. Листинг программы основного исполняемого файла для демонстрации работы операторов точка и стрелка

```
#include <iostream>
#include "Example.h" // подключаем заголовочный файл с описанием класса
using namespace std;

int main()
{
    Example Test; // создание объекта Test класса Example
    Test.value2 = 100; // полю value2 объекта Test присваиваем значение 100
    Example* Test2 = new Example(8000, 'F'); // создание второго объекта Test2 используя указатель и параметризованный конструктор
    Test2 -> value2 = 900; // присвоение значения 900 полю value2 объекта Test2
    delete Test2; // обязательно удаление указателя в конце программы

    return 0;
}
```

Обращение вышеуказанным способом может сработать только к элементам с модификатором доступа public. В языке C++ существуют три модификатора доступа: public, protected, private.

Модификатор доступа public используется для того, чтобы поле или метод класса (или его потомков) было доступно из любой точки программы.

Модификатор доступа protected используется, чтобы ограничить доступ к полям или методам класса. В таком случае доступ будет только из самого класса или производных от него, но не извне.

Модификатор доступа private — используется чтобы максимально ограничить доступ к полям или методам класса. В таком случае доступ будет только из самого класса. Описание модификаторов доступа, будет рассмотрено на примере уже существующего класса Example в таблице 39:

Таблица 39. Листинг программы заголовочного файла для демонстрации работы модификаторов доступа

```
#ifndef EXAMPLE_H
#define EXAMPLE_H
#include <string>

class Example
{
private: // применение приватного модификатора доступа
    int value1;
    char symbol;
public: // применение публичного модификатора доступа
    int value2;
    Example();
    Example(int value1, char symbol);
    Example(const Example& obj);
    explicit Example(int);
    ~Example();
protected: // применение защищенного модификатора доступа
    std::string name; // в случае, когда пространство имен не задано, нужно указывать
    конструкцию std::, для применения элемента этого пространства имен
};

#endif
```

Важно также отметить, что все объявленные в классе поля и методы, если не был указан модификатор доступа `public`, по умолчанию становятся приватными (`private`).

Отдельного внимания заслуживает так называемый **константный объект**, значения полей которого запрещается изменять. К такому объекту должны применяться только константные методы.

Особенности константного метода, следующие:

- Объявляется с ключевым словом `const`.
- Не может изменять значения полей класса.
- Может вызывать только константные методы.
- Может вызываться для любых (не только константных) объектов.

Пример работы константного объекта и константного метода показан в листингах в таблицах 40-42. Результат работы кода представлен на рисунке 86.

Таблица 40. Листинг программы заголовочного файла для демонстрации работы константного объекта и константного метода

```
#ifndef EXAMPLE2_H
#define EXAMPLE2_H
```

```

class Example2
{
public:
    int value;
    char symbol;
    Example2(char s, int v) { value = v; symbol = s; } // небольшие методы можно описы-
    вать непосредственно внутри класса

    void Show();

    void ShowSV() const;
};

#endif

```

Таблица 41. Листинг программы исполняемого файла для демонстрации ра-
боты константного объекта и константного метода

```

#include "Example2.h"
#include <iostream>

void Example2::Show()
{
    std::cout << "The value is: " << value << " and symbol is: " << symbol << "\n";
}

void Example2::ShowSV() const
{
    std::cout << "The value is: " << value << " and symbol is: " << symbol << "\n";
}

```

Таблица 42. Листинг программы файла основной функции для демонстрации
работы константного объекта и константного метода

```

#include <iostream>
#include "Example2.h" // подключаем заголовочный файл с описанием класса
using namespace std;
int main()
{
    const Example2 Test ('A', 1); // создание константного объекта Test класса Example2
    Test.value = 4; // попытка изменения значения поля приведет к ошибке в данном
    случае

    Test.Show(); // попытка вызова данного метода также приведет к ошибке
    Test.ShowSV(); // этот метод можно вызывать, так как он тоже константный
    return 0;
}

```

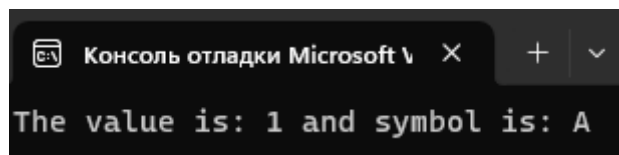


Рисунок 86. Тестирование программного кода со значением

8.6. Указатели на элементы классов

К объекту класса можно обращаться двумя способами непосредственно и с помощью указателя на объект.

Для того, чтобы получить доступ к отдельному члену объекта (члену класса, которому принадлежит объект) исключительно «силами» объекта, используется оператор «точка». Если для доступа к отдельному члену объекта (члену класса, которому принадлежит объект) служит указатель на этот объект, необходимо использовать оператор «стрелка».

Чтобы объявить указатель на объект, используется тот же синтаксис, как и для случаев объявления указателей на значения других типов.

Указатели на методы используются в таком случае, когда возникает необходимость вызвать метод, имя которого неизвестно. Однако в отличие от указателя на переменную или обычную функцию, указатель на метод не ссылается на определённый адрес памяти. Он больше похож на индекс в массиве, поскольку задает смещение. Сам же адрес получается путем сочетания указателя на метод с указателем на определенный объект.

Основные правила использования указателей на методы классов:

- Указателю на метод можно присваивать только адреса методов, имеющих соответствующий заголовок.
- Нельзя определить указатель на статический метод класса.
- Нельзя преобразовать указатель на метод в указатель на обычную функцию, не являющуюся членом класса.

Пример работы указателя на элементы класса показан в листингах в таблицах 43-45. Результат работы кода представлен на рисунке 87.

Таблица 43. Листинг программы заголовочного файла для демонстрации работы указателя на элементы класса

```
#pragma once
#include <string>

class Example4 // объявление класса Example4
{
    int a;
    std::string name;
public:
    Example4(std::string n, int num); // объявляем параметризованный конструктор
    void Change(std::string, int); // объявляем некоторый метод, который будет менять
    void Show();
};
```

Таблица 44. Листинг программы исполняемого файла для демонстрации работы указателя на элементы класса

```
#include "Example4.h"
#include <iostream>

Example4::Example4(std::string n, int num)
{
    name = n;
    a = num;
}

void Example4::Change(std::string n, int num)
{
    name = n;
    a = num;
}

void Example4::Show()
{
    std::cout << "The name is: " << name << " and the value is: " << a << "\n";
}
```

Таблица 45. Листинг программы файла основной функции для демонстрации работы указателя на элементы класса

```
#include <iostream>
#include "Example4.h" // подключаем заголовочный файл с описанием класса
using namespace std;

int main()
{
    Example4* Test = new Example4("Example", 57); // инициализация указателя Test, новым объектом класса Example4
    Test->Change("Example", 96); // вызов метода Change с двумя аргументами
    //Test->a = 1080; // выдаст ошибку, так как поля a - приватно
    Test->Show();
    delete Test;

    return 0;
}
```

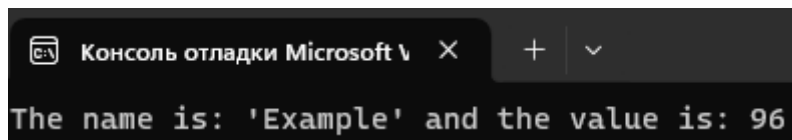


Рисунок 87. Тестирование программного кода со значением

8.7. Указатель this

Как показывает практика, многие реальные задачи из жизни требуют того, чтобы в протоколе описания класса существовала возможность вызова метода внутри определения другой метода. Если последовательно придерживаться парадигмы объектно-ориентированного программирования, то вложенный вызов метода должен интерпретироваться как передача сообщения объекту. Объект, принимающий вложенное сообщение – это тот же самый объект, который обрабатывает сообщение, представленное вызовом внешней функции.

Вложенный вызов метода в объектно-ориентированных языках интерпретируется как передача сообщения самому себе, что должно быть явно указано. В C++ для представления термина «сам себе» используется **ключевое слово this**.

Таким образом, ключевое слово this используется для обозначения указателя на объект, который сгенерировал обращение к функции-члену. Всем функциям-членам указатель this передается автоматически.

При каждом вызове метода ему автоматически передается указатель this, на объект, для которого он вызывался. Указатель this является неявным параметром, принимаемым всеми методами, следовательно, в любом методе указатель

this можно использовать для ссылки на вызывающий объект. Методы каждого объекта имеют доступ к указателю this.

Пример работы указателя this показан в листингах в таблицах 46-48. Результат работы кода представлен на рисунке 88.

Таблица 46. Листинг программы заголовочного файла для демонстрации работы указателя this

```
#ifndef EXAMPLE3_H
#define EXAMPLE3_H

class Example3 // объявление класса Example3
{
private:
    int a, b, c; // объявляем три целочисленных приватных поля
public:
    Example3(int, int, int); // объявляем параметризованный конструктор класса с
    тремя целочисленными параметрами
    void CountPerimeter(); // объявляем метод, который будет выводить значение пери-
    метра нашего треугольника
};
#endif
```

Таблица 47. Листинг программы исполняемого файла для демонстрации работы указателя this

```
#include "Example3.h" // подключаем файл с описанием класса
#include <iostream> // подключаем библиотеку для организации ввода/вывода

Example3::Example3(int a, int b, int c) // описываем тело конструктора
{
    this->a = a; // в данном случае, чтобы не увеличивать количество наименований пе-
    рменных используем указатель this
    this->b = b;
    this->c = c;
}

void Example3::CountPerimeter() // описываем тело метода для подсчета и вывода пери-
метра
{
    std::cout << "Perimeter is: " << this->a + this->b + this->c << "\n"; // также исполь-
    зуем указатель this для подсчета периметра
}

void Example3::CountPerimeter()
{
    std::cout << "Perimeter is: " << this->a + this->b + this->c << "\n";
}
```


Таблица 48. Листинг программы файла основной функции для демонстрации работы указателя this

```
#include <iostream>
#include "Example3.h" // подключаем заголовочный файл с описанием класса
using namespace std;

int main()
{
    Example3 Triangle (3,4,5); // создание объекта Triangle класса Example3 с заданными
    сторонами
    Triangle.CountPerimeter(); // вызываем метод для подсчета и вывода значения пери-
    метра треугольника

    return 0;
}
```

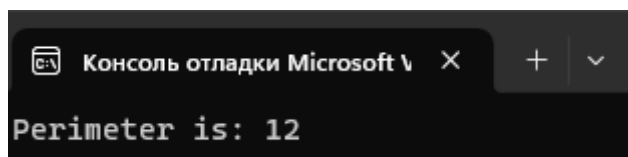


Рисунок 88. Тестирование программного кода со значением

8.8. Статические поля и методы класса

Статические поля применяются для хранения данных, общих для всех объектов класса. Они существуют в единственном экземпляре, то есть не дублируются.

У статических полей есть следующие особенности:

- Память под статическое поле выделяется один раз при его инициализации вне зависимости от числа созданных объектов и инициализируется с помощью операции доступа к области действия, а не операции выбора
- Статические поля доступны как через имя класса, так и через имя объекта
- На статические поля распространяется действие спецификатора доступа
- Память, занимаемая статическим полем, не учитывается при определении размера объекта с помощью операции sizeof

Статические методы предназначены для обращения к статическим полям класса. Они могут обращаться непосредственно только к статическим полям и вызывать только другие статические методы класса, потому что им не передается скрытый указатель this. Обращение к статическим методам производится так же,

как к статическим полям — либо через имя класса, либо, если хотя бы один объект класса уже создан, через имя объекта. Статические методы не могут быть константными и виртуальными.

Пример работы статических полей и методов показан в листингах в таблицах 49-51. Результат работы кода представлен на рисунке 89.

Таблица 49. Листинг программы заголовочного файла для демонстрации работы статических полей и методов

```
#pragma once
class Example5
{
private:
    static int staticParam; // объявление статической целочисленной переменной
    int nonStaticParam;
public:
    Example5(int, int); // объявление параметрического конструктора
    static void staticMethod(); // объявление статического метода
    void nonStaticMethod();
    void Show();
};
```

Таблица 50. Листинг программы исполняемого файла для демонстрации работы статических полей и методов

```
#include "Example5.h"
#include <iostream>
int Example5::staticParam; // выделение памяти под статическую переменную

Example5::Example5(int SP, int NSP) // обычные не статические методы могут работать со
статическими переменными
{
    staticParam = SP;
    nonStaticParam = NSP;
}

void Example5::staticMethod()
{
    staticParam += 50;
    //this->staticParam += 50; // приведет к ошибке, так как в статических методах, до-
ступа к this нет
    //nonStaticParam += 100; // также приведе к ошибке, так как статический метод рабо-
тает только со статическими элементами
}

void Example5::nonStaticMethod()
```

```

{
    this->nonStaticParam += 20; // можно использовать this, так как это нестатический
    // член класса
    staticParam += 90; // можно обращаться к статическому члену класса и менять его
    this->staticParam += 85; // не приведет к ошибке так как указатель this можно ис-
    // пользовать в нестатических методах
}

void Example5::Show()
{
    std::cout << "The staticParam value: " << staticParam << "\n";
    std::cout << "The nonStaticParam value: " << nonStaticParam << "\n";
}

```

Таблица 51. Листинг программы файла основной функции для демонстрации работы статических полей и методов

```

#include <iostream>
#include "Example5.h" // подключаем заголовочный файл с описанием класса
using namespace std;

int main()
{
    int a, b;
    cout << "Enter two values for new classes: ";
    cin >> a >> b;
    Example5 Test(a,b);
    Test.staticMethod(); // вызов статического метода staticMethod() объекта Test
    Test.Show(); // вызов метода Show объекта Test
    Test.nonStaticMethod(); // вызов метода nonStaticMethod() объекта Test
    Test.Show(); // вызов метода Show объекта Test

    return 0;
}

```

```

Консоль отладки Microsoft V  X  +  v
Enter two values for new classes: 10 15
The staticParam value: 60
The nonStaticParam value: 15
The staticParam value: 235
The nonStaticParam value: 35

```

Рисунок 89. Тестирование программного кода со значением

9. ОСНОВНЫЕ ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Парадигмой программирования называют набор идей и понятий, которые определяют стиль написания программ, подход к программированию.

Программа — это описание на языке программирования структур данных и алгоритма решения задачи, автоматически переводимое, при помощи специальной программы-транслятора (компилятора или интерпретатора), на язык команд компьютера для последующего выполнения.

Основной точкой входа в программу является функция `main`.

Переменная — это именованная область памяти.

Описание — это оператор, который вводит имя в программу.

Объявление переменной — это действие, в котором сначала указывается тип данных, а затем название этой переменной.

Присвоение значения переменной — это действие, в котором ранее объявленной переменной присваивается некоторое значение (значение записывается в именованную область памяти).

Инициализация переменной — это действие, в котором одновременно происходит и объявление переменной, и присвоение ей значения.

Константа, которая начинается с комбинации `0x` — является **шестнадцатеричной константой**.

Константа, которая начинается с комбинации `0` — является **восьмеричной константой**.

Символьной константой является символ, заключенный в одинарные кавычки, например `'a'` или `'0'`.

Условные операторы — операторы, которые указывают, следует ли выполнять какие-то связанные с ними инструкции

Цикл — это повторение одного и того же участка кода в программе.

Тело цикла — последовательность действий, которые повторяются, внутри цикла

Шаг цикла или **итерация** — один проход цикла.

Параметрами цикла — переменные, изменяющиеся внутри цикла и влияющие на его окончание.

Указатель — это самостоятельный тип данных, который может хранить адреса других переменных в памяти.

Ссылочная переменная (ссылка) — это псевдоним, то есть другое имя для уже существующей переменной.

Массив – это набор однотипных переменных, к которому можно обратиться по одному имени, но они имеют разные индексы.

Статические массивы – массивы, которые объявляются с указанием конкретных значений диапазонов своих индексов.

Динамическим называется такой массив, размер которого меняется во время исполнения программы.

Строкой называют последовательность символов, заключенных в двойные кавычки.

Функция – это поименованная часть программы, которая может вызываться из других частей программы столько раз, сколько необходимо.

Аргументом, передаваемым в функцию по умолчанию — называется такой аргумент, значение которого в функции заранее задано.

Перегрузка функций — это механизм C++, благодаря которому функции с разным количеством или типами параметров могут иметь одинаковое имя.

Структурой называют совокупность элементов произвольных типов.

Область видимости переменных — это те части программы, в которой пользователь может изменять или использовать переменные в своих нуждах.

Глобальными переменными называются те переменные, которые были созданы вне тела какого-то блока.

Локальные переменные — это переменные, созданные в блоках. Областью видимости таких переменных является блоки (и все их дочерние), а также их область видимости не распространяется на другие блоки.

Глобальный оператор разрешения — это два подряд поставленные двоеточия, с помощью которых можно объяснить компилятору, что требуется использовать глобальную переменную, а не локальную.

Парадигма объектно-ориентированного программирования звучит следующим образом: *определите, какой класс вам необходим; предоставьте полный набор операций для каждого класса; общность классов выразите явно с помощью наследования.*

Объектом в природе принято называть то, что может быть индивидуально описано и распознано.

Объект в программе – экземпляр класса или его копия.

Класс — абстрактный тип данных, определяемый пользователем. Основными свойствами класса являются его **поля** (переменные-член класса) и его **методы** (функции-член класса).

Система — множество объектов, находящихся в отношениях и связях друг с другом, которое образует определённую целостность, единство.

Наследование — это механизм, позволяющий строить иерархию типов. Механизм наследования предполагает определение базового типа (или родительского), а затем использование этого типа для построения производных типов (или потомков).

Инкапсуляция — этот принцип гласит, что вся важная информация, необходимая для работы объекта, в нем же и хранится. И только определенные данные доступны для внешних функций и объектов.

Полиморфизм — возможность использовать в различных классах иерархии одно имя для обозначения сходных по смыслу действий и гибко выбирать требуемое действие во время выполнения программы.

Конструктор — это особый вид метода, который является членом класса и вызывается в момент создания объекта. Особенность данного метода заключается в том, что его имя совпадает с именем класса.

Конструктор по умолчанию — конструктор, который вызывается без параметров. Если класс не содержит явным образом определенный конструктор, тогда при создании объекта автоматически вызывается конструктор по умолчанию.

Конструктор с параметром (параметризированный конструктор) — конструктор, в который передаются параметры (аргументы).

Конструктор копирования — это специальный вид конструктора, получающий в качестве единственного параметра указатель на объект этого же класса.

Конструктором преобразования — любой конструктор класса, принимающий один параметр какого-либо другого типа, поскольку он осуществляет преобразование из типа параметра в тип этого класса.

Деструктор — это особый вид метода, применяющийся для освобождения памяти, занимаемой объектом.

Модификатор доступа *public* используется для того, чтобы поле или метод класса (или его потомков) было доступно из любой точки программы.

Модификатор доступа *protected* используется, чтобы ограничить доступ к полям или методам класса. В таком случае доступ будет только из самого класса или производных от него, но не извне.

Модификатор доступа *private* — используется чтобы максимально ограничить доступ к полям или методам класса. В таком случае доступ будет только из самого класса.

Константный объект, значения полей которого запрещается изменять. К такому объекту должны применяться только константные методы.

Ключевое слово *this* используется для обозначения указателя на объект, который сгенерировал обращение к функции-члену. Всем функциям-членам указатель *this* передается автоматически.

СПИСОК ИСТОЧНИКОВ И ЛИТЕРАТУРЫ

1. ГОСТ 19.701-90 Единая система программной документации схемы алгоритмов, программ, данных и систем обозначения условные и правила выполнения / СТАНДАРТИНФОРМ, 2010
2. Лафоре Р. Объектно-ориентированное программирование в C++. / пер. с англ. А. Кузнецов, М. Назаров, В. Шрага — СПб.: Питер, 2020. — 928 с.: ил..
3. Бьёрн Страуструп. Язык программирования C++. Специальное издание. Бином-Пресс, 2007. — 1104 с.
4. Павловская Т.А. C/C++. Программирование на языке высокого уровня /— СПб.: Питер, 2018. — 461 с: ил..
5. Эллайн А. Э47 C++. От ламера до программера. — СПб.: Питер, 2015. — 480 с.: ил..
6. Ритчи Деннис М., Керниган Брайан У. Язык программирования C, 2019/ ISBN 978-5-907144-14-9
7. Унгер А.Ю. Объектно-ориентированное программирование [Электронный ресурс]: Учебное пособие / Унгер А.Ю. — М.: МИРЭА – Российский технологический университет, 2022.

СВЕДЕНИЯ ОБ АВТОРАХ

Лозовский Владимир Валерьевич, кандидат технических наук, доцент, доцент кафедры вычислительной техники Института информационных технологий РТУ МИРЭА.

Морозов Василий Алексеевич, ассистент кафедры вычислительной техники Института информационных технологий РТУ МИРЭА.

Гололобов Александр Алексеевич, ассистент кафедры вычислительной техники Института информационных технологий РТУ МИРЭА.